The Pragmatic Programmer

Chapter 1 — A Pragmatic Philosophy (လက်တွေ့ကျသော အတွေးအခေါ်)

Pragmatic Programmer တွေကို သီးခြားခွဲခြားပေးတာက Programming Skill တစ်ခုတည်း မဟုတ်ပါဘူး။

သူတို့ရဲ့—

- သဘောထား
- အလုပ်လုပ်ပုံ
- ပြဿနာတွေကို ချဉ်းကပ်ပုံ
- ဖြေရှင်းချက်တွေကို စဉ်းစားပုံ

တို့ပဲ ဖြစ်ပါတယ်။

Pragmatic Programmer တွေဟာ လက်ရှိပြဿနာတစ်ခုတည်းကိုပဲ မကြည့်ဘဲ **အခြေအနေတစ်ခုလုံးရဲ့ Context နဲ့ Bigger Picture** ကိုပါ ကြည့်တတ်ကြပါတယ်။ ဒီလိုလုပ်နိုင်မှသာ အကျိုးရှိတဲ့ Compromise တွေ ပြုလုပ်ပြီး မှန်ကန်တဲ့ Decision တွေ ချမှတ်နိုင်မှာ ဖြစ်ပါတယ်။

1. The Cat Ate My Source Code - "ကြောင်က Source Code ကို စားသွားတယ်" ဆိုတဲ့ ဆင်ခြေ

Pragmatic Programmer တစ်ယောက်ရဲ့ အရေးကြီးဆုံး အရည်အချင်းတွေထဲက တစ်ခုက—

တာဝန်ယူမှု (Responsibility) ဖြစ်ပါတယ်။

တစ်ခုခုမှားသွားတဲ့အခါ—

"ဒါက ကျွန်တော့်အမှားမဟုတ်ဘူး"

ဆိုပြီး အကြောင်းပြချက်ရှာဖို့ မကြိုးစားသင့်ပါဘူး။

Project မှာ ဖြစ်ပျက်နေတဲ့ အရာတွေအတွက် ကိုယ်တတ်နိုင်သမျှ **တာဝန်ယူပြီး ပြဿနာကို ဖြေရှင်းဖို့** လိုပါတယ်။

Responsibility ကို လက်ခံပါ

ကိုယ်လုပ်နိုင်တာနဲ့ မလုပ်နိုင်တာကို ခွဲခြားသိဖို့လိုပါတယ်။

ဥပမာ—

- Requirements မရှင်းဘူး
- Manager က Deadline တိုတောင်းပေးတယ်
- Third-party Library မှာ Bug ရှိတယ်
- Server ပျက်သွားတယ်

စတာတွေ ဖြစ်နိုင်ပါတယ်။

ဒီလိုအခြေအနေတွေမှာ "ဒါက ကျွန်တော့်အပြစ်မဟုတ်ဘူး" လို့ပြောပြီး ရပ်မနေသင့်ပါဘူး။

အဲဒီအစား—

"အခုအခြေအနေမှာ ငါဘာလုပ်နိုင်သလဲ?"

ဆိုတာကို စဉ်းစားရပါမယ်။

Broken Window Theory

Software Project တစ်ခုဟာ တဖြည်းဖြည်းနဲ့ ပျက်စီးသွားနိုင်ပါတယ်။

အသေးစား Problem တစ်ခုကို မပြင်ဘဲထားလိုက်ရင် နောက်ထပ် Problem တွေ ထပ်ပေါ်လာနိုင်ပါတယ်။

နောက်ဆုံးမှာ—

"ဒီ Project က ဒီလိုပဲ"

ဆိုတဲ့ သဘောထား ဖြစ်လာနိုင်ပါတယ်။

ဒါကြောင့် Problem သေးသေးလေးတွေကိုလည်း လျစ်လျူမရှုသင့်ပါဘူး။

No Broken Windows

Project ထဲမှာ—

- Warning တွေ
- Bad Code
- Temporary Fix တွေ
- မလိုအပ်တဲ့ Code
- မသန့်ရှင်းတဲ့ Design

တွေကို "နောက်မှ ပြင်မယ်" ဆိုပြီး မထားသင့်ပါဘူး။

ပထမဆုံး Broken Window ကို ချက်ချင်းပြင်ပါ။

အဓိကသင်ခန်းစာ

ကိုယ့်အလုပ်အတွက် တာဝန်ယူပါ။ အကြောင်းပြချက်ပေးမယ့်အစား ဖြေရှင်းချက်ရှာပါ။

2. Software Entropy - Software ရဲ့ ပျက်စီးယိုယွင်းမှု

Entropy ဆိုတာ Disorder သို့မဟုတ် ပျက်စီးယိုယွင်းမှုကို ဆိုလိုပါတယ်။

Software Project တွေမှာလည်း အချိန်ကြာလာတာနဲ့အမျှ—

- Code ပိုရှုပ်လာတယ်
- Architecture ပိုပျက်လာတယ်
- Documentation မကိုက်တော့ဘူး
- Temporary Fix တွေ များလာတယ်
- Design ရဲ့ မူလရည်ရွယ်ချက် ပျောက်လာတယ်

ဆိုတဲ့ အခြေအနေတွေ ဖြစ်နိုင်ပါတယ်။

ဒီလိုဖြစ်လာတာကို **Software Entropy** လို့ စဉ်းစားနိုင်ပါတယ်။

Broken Windows နဲ့ Software Entropy

အဆောက်အအုံတစ်ခုမှာ ပြတင်းပေါက်တစ်ချပ် ပျက်နေပြီး ဘယ်သူမှ မပြင်ဘူးဆိုရင်—

"ဒီနေရာကို ဘယ်သူမှ ဂရုမစိုက်ဘူး"

လို့ လူတွေထင်လာနိုင်ပါတယ်။

နောက်ထပ် ပြတင်းပေါက်တွေ ပျက်လာပြီး နောက်ဆုံးမှာ အဆောက်အအုံတစ်ခုလုံး ပျက်စီးသွားနိုင်ပါတယ်။

Software မှာလည်း အလားတူပါပဲ။

အမှားသေးသေးလေးတွေကို လက်ခံလာတာနဲ့ အမှားကြီးတွေကိုလည်း လက်ခံလာနိုင်ပါတယ်။

Entropy ကို တားဆီးပါ

Project ရဲ့ Quality ကျလာတာကို တွေ့ရင်—

"ဒါလေးပဲ"

လို့ မထားပါနဲ့။

ချက်ချင်းပြင်ပါ။

Code ကို Refactor လုပ်ပါ။

Documentation ကို Update လုပ်ပါ။

မလိုအပ်တဲ့ Code ကို ဖယ်ရှားပါ။

အဓိကသင်ခန်းစာ

Project ကို သန့်ရှင်းနေအောင် အမြဲထိန်းသိမ်းပါ။

3. Stone Soup and Boiled Frogs - ကျောက်တုံးဟင်းနှင့် ရေနွေးထဲက ဖား

ဒီ Section မှာ **Change** ကို ဘယ်လိုကိုင်တွယ်ရမလဲဆိုတာ ဆွေးနွေးထားပါတယ်။

Project တစ်ခုမှာ Change မဖြစ်အောင် တားလို့မရပါဘူး။

ဒါကြောင့် Change ကို—

ဖြည်းဖြည်း၊ လက်တွေ့ကျကျ၊ လူတွေပါဝင်လာအောင်

စီမံနိုင်ဖို့လိုပါတယ်။

Stone Soup

ဇာတ်လမ်းထဲမှာ စစ်သားတွေဟာ ရွာတစ်ရွာကို ရောက်လာပြီး—

"ကျောက်တုံးဟင်းချက်မယ်"

လို့ ပြောပါတယ်။

အစမှာ ရွာသားတွေက မယုံကြည်ကြပါဘူး။

ဒါပေမယ့် စစ်သားတွေက—

"နည်းနည်း ဆားထည့်ရင် ပိုကောင်းမယ်"

"အာလူးနည်းနည်းရှိရင် ပိုကောင်းမယ်"

"မုန်လာဥနည်းနည်းရှိရင် ပိုကောင်းမယ်"

ဆိုပြီး တဖြည်းဖြည်းနဲ့ ရွာသားတွေကို ပါဝင်လာအောင် လုပ်ပါတယ်။

နောက်ဆုံးမှာ ရွာသားတွေ ကိုယ်တိုင် ပါဝင်ပြီး အရသာရှိတဲ့ ဟင်းတစ်အိုး ဖြစ်လာပါတယ်။

Be a Catalyst for Change

Software Project မှာလည်း Change ကြီးတစ်ခုကို တစ်ခါတည်း အားလုံးကို လက်ခံခိုင်းတာထက်—

အောင်မြင်နိုင်တဲ့ အသေးစားအဆင့်တစ်ခုကို အရင်ပြပါ။

လူတွေက အောင်မြင်နေတဲ့အရာထဲကို ဝင်ပါဝင်ရတာ ပိုလွယ်ပါတယ်။

TIP 5
Be a Catalyst for Change
ပြောင်းလဲမှုကို ဖြစ်ပေါ်လာအောင် လှုံ့ဆော်ပေးတဲ့သူ ဖြစ်ပါ။

အနာဂတ်မှာ ဘာဖြစ်လာနိုင်တယ်ဆိုတာကို လူတွေကို အနည်းငယ် ပြသပေးနိုင်ရင် သူတို့ကိုယ်တိုင် ပါဝင်လာနိုင်ပါတယ်။

Boiled Frogs

တစ်ဖက်မှာတော့ **Boiled Frog** ဇာတ်လမ်းရှိပါတယ်။

အေးတဲ့ရေထဲမှာ ဖားတစ်ကောင်ကို ထည့်ပြီး ရေကို တဖြည်းဖြည်းပူအောင်လုပ်ရင် ဖားက အပူချိန်တိုးလာတာကို သတိမထားမိဘဲ နောက်ဆုံးမှာ အန္တရာယ်ဖြစ်နိုင်တယ်ဆိုတဲ့ ဥပမာပါ။

ဒီဥပမာကို Software Project ရဲ့ **Gradual Change** နဲ့ နှိုင်းယှဉ်ထားပါတယ်။

Project တွေဟာ တစ်နေ့တည်းနဲ့ မပျက်စီးပါဘူး။

- Feature တစ်ခု
- Patch တစ်ခု
- Requirement တစ်ခု
- Deadline တစ်ခု

စီစီနဲ့ တဖြည်းဖြည်း ပြောင်းလာပြီး နောက်ဆုံးမှာ မူလ Project နဲ့ လုံးဝမတူတော့နိုင်ပါတယ်။

TIP 6
Remember the Big Picture

ကိုယ်လုပ်နေတဲ့ အလုပ်တစ်ခုတည်းကိုပဲ မကြည့်ဘဲ Project တစ်ခုလုံးရဲ့ Bigger Picture ကို အမြဲကြည့်ပါ။

4. Good-Enough Software - လုံလောက်အောင်ကောင်းတဲ့ Software

Software ကို Perfect ဖြစ်အောင်လုပ်ဖို့ ကြိုးစားတာဟာ အမြဲတမ်း အကောင်းဆုံးနည်းလမ်း မဟုတ်ပါဘူး။

Real World မှာ—

- Time ကန့်သတ်ချက်ရှိတယ်
- Technology ကန့်သတ်ချက်ရှိတယ်
- Budget ကန့်သတ်ချက်ရှိတယ်
- User Requirements ရှိတယ်
- Deadline ရှိတယ်

ဒါကြောင့် **အခြေအနေအရ လုံလောက်အောင်ကောင်းတဲ့ Software** ကို ပေးနိုင်ဖို့လိုပါတယ်။

“Good Enough” ဆိုတာ Poor Quality မဟုတ်ပါ

Good Enough ဆိုတာ—

“အရည်အသွေးမကောင်းလည်း ရတယ်”

လို့ ဆိုလိုတာ မဟုတ်ပါဘူး။

Software ဟာ User ရဲ့ Requirements တွေကို ဖြည့်ဆည်းရပါမယ်။

စာအုပ်ရဲ့ အဓိကအယူအဆက—

ဘယ်အချိန်မှာ Software က “Good Enough” ဖြစ်ပြီလဲဆိုတာ ဆုံးဖြတ်တဲ့အခါ Users တွေကိုပါ ပါဝင်စေပါ။

User ကို Trade-Off ထဲမှာ ပါဝင်စေပါ

တစ်ခါတလေ Developer တွေက—

“ဒီ Feature နည်းနည်းထပ်ထည့်မယ်”

“ဒီ Code ကို နည်းနည်းထပ် Polish လုပ်မယ်”

ဆိုပြီး Release ကို ရွှေ့တတ်ကြပါတယ်။

ဒါပေမယ့် User အတွက် အချိန်မီရရှိဖို့က ပိုအရေးကြီးနိုင်ပါတယ်။

တစ်ဖက်မှာလည်း Deadline ရှိလို့—

Basic Engineering Quality ကို ဖြတ်တောက်ပြီး Software ကို အမြန်ထုတ်တာ

ကိုလည်း မလုပ်သင့်ပါဘူး။

အဓိကအယူအဆ

Perfect Software ကို စောင့်နေတာထက် User အတွက် လုံလောက်အောင်ကောင်းတဲ့ Software ကို အချိန်မှန်ပေးနိုင်ဖို့က ပို အရေးကြီးနိုင်ပါတယ်။

5. Your Knowledge Portfolio

Programmer တစ်ယောက်ရဲ့ အရေးအကြီးဆုံး Professional Assets တွေက—

Knowledge နဲ့ Experience

ဖြစ်ပါတယ်။

ဒါပေမယ့် Knowledge ဟာ အမြဲတမ်း တန်ဖိုးရှိနေမယ့်အရာ မဟုတ်ပါဘူး။

Technology အသစ်တွေ၊ Language အသစ်တွေ၊ Environment အသစ်တွေ ပေါ်လာတာနဲ့ ကိုယ့်ရဲ့ Knowledge ဟာ တဖြည်းဖြည်း **Out of Date** ဖြစ်လာနိုင်ပါတယ်။

Knowledge Portfolio ဆိုတာဘာလဲ?

Programmer တစ်ယောက် သိထားတဲ့—

- Programming Languages
- Technologies
- Tools
- Application Domains
- Development Techniques
- Experience

အားလုံးကို **Knowledge Portfolio** လို့ စဉ်းစားနိုင်ပါတယ်။

ဒါကို Financial Portfolio တစ်ခုလို စီမံခန့်ခွဲသင့်ပါတယ်။

Knowledge Portfolio ရဲ့ စည်းမျဉ်းများ

1. Invest Regularly

Knowledge ထဲကို ပုံမှန်ရင်းနှီးမြှုပ်နှံပါ။

တစ်နေ့နည်းနည်းပဲဖြစ်ဖြစ်—

အမြဲလေ့လာတဲ့ Habit

က အရေးကြီးပါတယ်။

2. Diversify

Technology တစ်မျိုးတည်းကိုပဲ မမှီခိုပါနဲ့။

Technologies မျိုးစုံကို သိထားရင် Change ဖြစ်လာတဲ့အခါ ပိုပြီး လိုက်လျောညီထွေဖြစ်နိုင်ပါတယ်။

3. Manage Risk

Technology အားလုံးကို High-Risk Technology တွေချည်း မရွေးပါနဲ့။

တစ်ဖက်မှာလည်း Technology အသစ်တွေကို လုံးဝရှောင်မနေပါနဲ့။

Risk နဲ့ Opportunity ကို Balance လုပ်ပါ။

4. Buy Low, Sell High

Technology အသစ်တစ်ခု Popular မဖြစ်ခင် လေ့လာထားရင် နောက်ပိုင်းမှာ အကျိုးရှိနိုင်ပါတယ်။

5. Review and Rebalance

ကိုယ့် Knowledge Portfolio ကို အမြဲပြန်စစ်ပါ။

ပြီးခဲ့တဲ့လက Hot ဖြစ်တဲ့ Technology က ဒီနေ့မှာ အသုံးမဝင်တော့နိုင်ပါတယ်။

ဒါကြောင့် Knowledge ကို **Review → Update → Rebalance** လုပ်ရပါမယ်။

TIP 8
Invest Regularly in Your Knowledge Portfolio

ကိုယ့်ရဲ့ Knowledge ထဲကို ပုံမှန် ရင်းနှီးမြှုပ်နှံပါ။

လက်တွေ့လုပ်နိုင်တဲ့ Goals

စာအုပ်က အောက်ပါအရာတွေကို အကြံပြုထားပါတယ်—

- နှစ်စဉ် Programming Language အသစ် အနည်းဆုံးတစ်ခု လေ့လာပါ
- သုံးလတစ်ကြိမ် Technical Book တစ်အုပ်ဖတ်ပါ
- Technical မဟုတ်တဲ့ စာအုပ်တွေလည်း ဖတ်ပါ
- Course တွေတက်ပါ
- Local/Professional Groups တွေထဲ ပါဝင်ပါ
- ကိုယ်သိထားတဲ့အရာကို တခြားသူတွေကို သင်ပေးပါ
- ကိုယ်မသိတဲ့အရာတွေကို လေ့လာပါ

ရည်ရွယ်ချက်က **Knowledge ကို အမြဲတမ်း လတ်ဆတ်နေအောင် ထိန်းသိမ်းဖို့** ဖြစ်ပါတယ်။

6. Communicate!

Programmer တစ်ယောက်ဟာ တစ်ယောက်တည်း အလုပ်လုပ်နေတာ မဟုတ်ပါဘူး။

နေ့စဉ်—

- Users
- Managers
- Team Members
- Clients
- Other Developers
- Technical Staff

တွေနဲ့ ဆက်သွယ်နေရပါတယ်။

ဒါကြောင့် Communication Skill ဟာ Programming Skill လောက်ပဲ အရေးကြီးပါတယ်။

Chapter 1 ရဲ့ နောက်ဆုံး Section ကလည်း ဒီအချက်ကို အထူးအလေးပေးထားပါတယ်။

Know What You Want to Say

တစ်ခုခုကို မပြောခင်—

"ငါဘာပြောချင်တာလဲ?"

ဆိုတာကို အရင်ရှင်းအောင်လုပ်ပါ။

ကိုယ့်အတွေးကို ကိုယ်တိုင် မရှင်းလင်းသေးရင် တခြားသူတွေကိုလည်း ရှင်းလင်းစွာ ပြောနိုင်မှာမဟုတ်ပါဘူး။

Know Your Audience

တစ်ယောက်နဲ့တစ်ယောက် Information လိုအပ်ချက် မတူပါဘူး။

ဥပမာ—

Manager

Project Status, Cost, Schedule ကို သိချင်နိုင်ပါတယ်။

Developer

Technical Details ကို သိချင်နိုင်ပါတယ်။

User

သူ့အတွက် System က ဘာလုပ်ပေးမလဲဆိုတာ သိချင်ပါတယ်။

ဒါကြောင့် **Audience နဲ့ ကိုက်ညီအောင် ပြောဆိုပါ။**

Choose Your Timing

Information မှန်ပေမယ့် အချိန်မမှန်ရင် Communication က အလုပ်မဖြစ်နိုင်ပါဘူး။

အရေးကြီးတဲ့ Message ကို အချိန်မီ ပြောပါ။

Problem ဖြစ်ပြီးမှ မပြောဘဲ—

Problem ဖြစ်နိုင်တဲ့အချိန်မှာ စောစောပြောပါ။

Choose Your Style

Communication ဆိုတာ Meeting ထဲမှာ စကားပြောတာတစ်ခုတည်း မဟုတ်ပါဘူး။

- Email
- Documentation
- Diagram
- Presentation
- Code Comment
- Report
- Chat

စတာတွေလည်း Communication ပုံစံတွေပါ။

Message နဲ့ Audience အလိုက် သင့်တော်တဲ့ Medium ကို ရွေးပါ။

Make It Look Good

Content မှန်တာတင် မလုံလောက်ပါဘူး။

Document တစ်ခု၊ Presentation တစ်ခု၊ Report တစ်ခုဟာ—

- ဖတ်ရလွယ်ရမယ်
- စနစ်တကျရှိရမယ်
- ရှင်းလင်းရမယ်
- Consistent ဖြစ်ရမယ်

ဒါမှ လူတွေက Information ကို ပိုလွယ်ကူစွာ နားလည်နိုင်မှာပါ။

The Broken Window Effect in Communication

Communication မကောင်းရင် Project မှာ—

- Misunderstanding
- Duplicate Work
- Wrong Assumptions
- Delays
- Conflicts

တွေ ဖြစ်လာနိုင်ပါတယ်။

ဒါကြောင့် Communication ကို Project ရဲ့ အစိတ်အပိုင်းတစ်ခုအဖြစ် သဘောထားပါ။

အဓိကသင်ခန်းစာ

Code ကောင်းကောင်းရေးနိုင်ရုံနဲ့ မလုံလောက်ပါဘူး။ ကိုယ့်အတွေးကို လူတွေ နားလည်အောင် ရှင်းလင်းစွာ ဆက်သွယ်နိုင်ရပါမယ်။

Chapter 2 — A Pragmatic Approach (လက်တွေ့ကျသော ချဉ်းကပ်နည်း)

ဒီ Chapter မှာ Software Development ရဲ့ အဆင့်တိုင်းမှာ အသုံးဝင်တဲ့ အခြေခံအယူအဆတွေ၊ နည်းလမ်းတွေကို စုစည်းထားပါတယ်။

ပထမဆုံး **The Evils of Duplication** နဲ့ **Orthogonality** က တစ်ခုနဲ့တစ်ခု ဆက်စပ်ပါတယ်။ တစ်ခုက System ထဲမှာ Knowledge ကို မထပ်ရေးဖို့ သတိပေးပြီး၊ နောက်တစ်ခုက Knowledge တစ်ခုတည်းကို System Component အများကြီးထဲ မခွဲပစ်ဖို့ အလေးပေးပါတယ်။

Reversibility ကတော့ Environment ပြောင်းလဲလာတဲ့အခါ Project ကို ပြောင်းလဲရလွယ်အောင် ကာကွယ်ပေးတဲ့ နည်းလမ်းတွေကို ဆွေးနွေးပါတယ်။

Tracer Bullets နဲ့ **Prototypes and Post-it Notes** ကလည်း ဆက်စပ်ပါတယ်။ Tracer Bullets က Requirements စုဆောင်းခြင်း၊ Design စမ်းသပ်ခြင်းနဲ့ Code Implementation ကို တစ်ချိန်တည်းမှာ လုပ်နိုင်အောင် ကူညီပေးပါတယ်။ မသင့်တော်တဲ့အခြေအနေတွေမှာတော့ Prototype တွေကို အသုံးပြုပြီး Architecture, Algorithm, Interface နဲ့ Idea တွေကို စမ်းသပ်နိုင်ပါတယ်။

Domain Languages က Problem Domain နဲ့ ပိုနီးစပ်တဲ့ Language တွေကို ကိုယ်တိုင်တည်ဆောက်အသုံးပြုနိုင်တဲ့ နည်းလမ်းတွေကို ပြောပြပါတယ်။

နောက်ဆုံး **Estimating** မှာ အချိန်နဲ့ Resource ကန့်သတ်ချက်တွေကြားမှာ အလုပ်တစ်ခု ဘယ်လောက်ကြာမလဲဆိုတာ လက်တွေ့ကျကျ ခန့်မှန်းနိုင်ဖို့ ဆွေးနွေးထားပါတယ်။

7. The Evils of Duplication - ထပ်ခါထပ်ခါ ရေးသားခြင်းရဲ့ ဆိုးကျိုးများ

Programmer တွေဟာ Knowledge တွေကို—

- စုဆောင်းတယ်
- စီစဉ်တယ်
- ထိန်းသိမ်းတယ်
- အသုံးချတယ်

Requirements ကို Specification ထဲမှာ ရေးထားတယ်။ Code ထဲမှာ အဲဒီ Knowledge ကို အကောင်အထည်ဖော်တယ်။ Testing မှာလည်း အဲဒီ Knowledge ကို အသုံးပြုတယ်။

ပြဿနာက **Knowledge ဟာ မတည်ငြိမ်ပါဘူး။**

Client နဲ့ ဆွေးနွေးပြီး Requirements ပြောင်းနိုင်တယ်။ Government Regulation ပြောင်းနိုင်တယ်။ ရွေးထားတဲ့ Algorithm က မအောင်မြင်နိုင်ဘူး။ Environment ပြောင်းနိုင်တယ်။

ဒါကြောင့် Software Development မှာ Maintenance ဆိုတာ Release ပြီးမှ စတာမဟုတ်ပါဘူး။ **Development လုပ်နေတဲ့ အချိန်တစ်လျှောက်လုံး Maintenance Mode ထဲမှာ ရှိနေပါတယ်။**

DRY Principle

Knowledge တစ်ခုကို နေရာအများကြီးမှာ ထပ်ရေးထားရင် Maintenance လုပ်တဲ့အခါ ပြဿနာဖြစ်ပါတယ်။

စာအုပ်က အဓိကစည်းမျဉ်းအဖြစ်—

EVERY PIECE OF KNOWLEDGE MUST HAVE A SINGLE, UNAMBIGUOUS, AUTHORITATIVE REPRESENTATION WITHIN A SYSTEM.

လို့ ဆိုပါတယ်။

TIP 11
DRY — Don't Repeat Yourself

အဓိပ္ပာယ်က—

Knowledge တစ်ခုကို တစ်နေရာထက်ပိုပြီး မထပ်ရေးပါနဲ့။

တစ်နေရာမှာ ပြောင်းရင် တခြားနေရာတွေကိုလည်း ပြန်ပြောင်းရမယ်ဆိုရင် တစ်နေ့နေ့မှာ တစ်နေရာကို မေ့သွားမှာ သေချာပါတယ်။ အဲဒီကနေ Contradictory Information ဖြစ်ပြီး Program ပျက်နိုင်ပါတယ်။

Duplication ဘယ်လိုဖြစ်လာသလဲ?

စာအုပ်က Duplication ကို အဓိကအားဖြင့် ၄ မျိုးခွဲပါတယ်—

1. **Imposed Duplication** — Environment ကြောင့် မဖြစ်မနေ ထပ်ရေးရသလို ဖြစ်နေခြင်း
2. **Inadvertent Duplication** — ကိုယ်တိုင် Duplicate ဖြစ်နေတာ မသိခြင်း
3. **Impatient Duplication** — လွယ်တယ်ထင်လို့ Copy/Paste လုပ်ပြီး ထပ်ရေးခြင်း
4. **Interdeveloper Duplication** — Developer အများကြားမှာ Knowledge တစ်ခုတည်းကို ထပ်ရေးခြင်း

Reuse လုပ်ရလွယ်အောင် ပြုလုပ်ပါ

တခြား Developer ရေးထားတဲ့ Code ကို ပြန်ရှာပြီး Reuse လုပ်ရတာ ခက်နေမယ်ဆိုရင် လူတွေက ကိုယ်တိုင်ပြန်ရေးကြပါလိမ့်မယ်။

ဒါကြောင့်—

TIP 12
Make It Easy to Reuse

ရှိပြီးသား Code, Utility, Script, Knowledge တွေကို ရှာပြီး Reuse လုပ်ရတာ ကိုယ်တိုင်အသစ်ရေးတာထက် လွယ်အောင် Environment ကို တည်ဆောက်ပါ။

8. Orthogonality - (အစိတ်အပိုင်းများကို လွတ်လပ်စွာ အလုပ်လုပ်နိုင်အောင် Design လုပ်ခြင်း)

Orthogonality ဆိုတာ Geometry ကနေ ယူထားတဲ့ Term ဖြစ်ပါတယ်။

Computer Programming မှာတော့—

တစ်ခုကို ပြောင်းလဲလိုက်တဲ့အခါ တခြားတစ်ခုကို မထိခိုက်စေခြင်း

ဆိုတဲ့ အဓိပ္ပာယ်နဲ့ အသုံးပြုပါတယ်။

ဥပမာ—

Database Code နဲ့ User Interface Code ဟာ Orthogonal ဖြစ်ရင်—

- UI ကို ပြောင်းလဲလို့ Database မထိခိုက်ဘူး
- Database ကို ပြောင်းလဲလို့ UI မထိခိုက်ဘူး

ဖြစ်သင့်ပါတယ်။

Nonorthogonal System

Nonorthogonal System မှာ Component တွေဟာ တစ်ခုနဲ့တစ်ခု အလွန်အကျွံ မှီခိုနေပါတယ်။

တစ်ခုကို ပြောင်းလိုက်တာနဲ့—

Component A
↓
Component B
↓
Component C
↓
Component D

လိုမျိုး ပြဿနာတွေ တစ်ဆင့်ပြီးတစ်ဆင့် ပြန့်သွားနိုင်ပါတယ်။

စာအုပ်မှာ Helicopter Controls ကို ဥပမာပေးထားပါတယ်။ Control တစ်ခုကို ပြောင်းလိုက်ရင် တခြား Control တွေကိုပါ ပြန်ညှိရတဲ့အတွက် System က အလွန်ရှုပ်ထွေးလာပါတယ်။

TIP 13
Eliminate Effects Between Unrelated Things

ဆက်စပ်မှုမရှိတဲ့ အရာတွေကြားမှာ တစ်ခုကတစ်ခုကို မထိခိုက်စေအောင် Design လုပ်ပါ။

Component တစ်ခုချင်းစီဟာ—

- Independent ဖြစ်ရမယ်
- Self-contained ဖြစ်ရမယ်
- တိကျတဲ့ Responsibility တစ်ခုရှိရမယ်

ဒီလိုဆိုရင် Component တစ်ခုကို ပြောင်းတဲ့အခါ System တစ်ခုလုံးကို မထိခိုက်စေဘဲ ပြင်နိုင်ပါတယ်။

Orthogonality ရဲ့ အကျိုးကျေးဇူး

1. Productivity တိုးတက်ခြင်း

Change တွေကို Localized လုပ်နိုင်တဲ့အတွက်—

- Development Time လျော့တယ်
- Testing Time လျော့တယ်
- Reuse လုပ်ရလွယ်တယ်
- Component အသစ်တွေနဲ့ ပေါင်းစပ်ရလွယ်တယ်

ဖြစ်လာပါတယ်။

2. Risk လျော့နည်းခြင်း

Component တစ်ခုမှာ ပြဿနာဖြစ်ရင် အဲဒီနေရာမှာပဲ ကန့်သတ်ထားနိုင်ပါတယ်။

ဒါကြောင့် System တစ်ခုလုံးက ပိုပြီး Stable ဖြစ်လာပါတယ်။ Testing လည်း ပိုလွယ်လာပါတယ်။ Vendor/Product/Platform တစ်ခုကို အလွန်အကျွံ မမှီခိုတော့ပါဘူး။

9. Reversibility - ပြန်လည်ပြောင်းလဲနိုင်မှု

Software Project တွေမှာ အစောပိုင်းက ချမှတ်လိုက်တဲ့ Decision တွေဟာ နောက်ပိုင်းမှာ ပြန်ပြင်ဖို့ အလွန်ခက်နိုင်ပါတယ်။

ဥပမာ—

- Database Vendor ရွေးချယ်မှု
- Architecture
- Deployment Model
- Third-party Product
- Technology

စတာတွေပါ။

Requirements, Users နဲ့ Hardware တွေဟာ Software ရေးပြီးစီးတာထက် ပိုမြန်မြန် ပြောင်းလဲနိုင်ပါတယ်။

ဒါကြောင့် Architecture ကို အစကတည်းက Flexible ဖြစ်အောင် Design လုပ်ဖို့လိုပါတယ်။

Database Vendor Example

အစမှာ Database Vendor A ကို ရွေးထားတယ်။

နောက်ပိုင်း Performance Testing လုပ်တဲ့အခါ Vendor A Database က နှေးနေပြီး Vendor B Database က ပိုမြန်တယ်ဆိုပါစို့။

Database Calls တွေကို Code တစ်ခုလုံးထဲမှာ ပြန့်ကျဲပြီးရေးထားရင် Vendor ပြောင်းဖို့ အလွန်ခက်ပါမယ်။

Database ကို—

"Persistence Service"
အဖြစ် Abstract လုပ်ထားရင် Database Vendor ပြောင်းရတာ ပိုလွယ်လာပါတယ်။

Deployment ကိုလည်း Flexible ဖြစ်အောင်ထားပါ

Project က Client-Server အဖြစ် စတင်ခဲ့ပေမယ့် နောက်ပိုင်းမှာ Stand-alone Version လိုလာနိုင်ပါတယ်။

Architecture က Flexible ဖြစ်ရင် Deployment Configuration ပြောင်းရုံနဲ့ ပြောင်းနိုင်ပါတယ်။

Stand-alone ကနေ Client-Server / N-tier ကို ပြောင်းတာလည်း အလွယ်တကူ ဖြစ်သင့်ပါတယ်။

TIP 14
There Are No Final Decisions
"ဒီဆုံးဖြတ်ချက်က အပြီးသတ်ပဲ" လို့ မယူဆပါနဲ့။

Decision တွေကို ကျောက်တုံးပေါ်မှာ ထွင်းထားသလို မစဉ်းစားဘဲ—

သဲပေါ်မှာ ရေးထားသလို စဉ်းစားပါ။

လိုအပ်လာရင် ပြန်ဖျက်ပြီး ပြန်ရေးနိုင်အောင် Design လုပ်ထားပါ။

10. Tracer Bullets - Tracer Bullets ဖြင့် Target ကို ရှာဖွေခြင်း

Project အသစ်တစ်ခုကို စတဲ့အခါ Target ကို အတိအကျ မသိနိုင်ပါဘူး။

- User Requirements မရှင်းနိုင်ဘူး
- New Technology ဖြစ်နိုင်တယ်
- Algorithm အသစ်ဖြစ်နိုင်တယ်
- Library အသစ်ဖြစ်နိုင်တယ်
- Environment က ပြောင်းလဲနိုင်တယ်

ဒီလိုအခြေအနေမှာ အစကတည်းက စာရွက်ပေါ်မှာ အရာအားလုံးကို အတိအကျတွက်ပြီး နောက်ဆုံးမှ Code ရေးတာဟာ Risk များပါတယ်။

Machine Gun မှာ **Tracer Bullet** တွေကို သုံးပြီး ကျည်ဆန်တွေ ဘယ်ကိုသွားနေတယ်ဆိုတာ ချက်ချင်းမြင်နိုင်သလို Software မှာလည်း **Tracer Code** ကို အသုံးပြုနိုင်ပါတယ်။

TIP 15
Use Tracer Bullets to Find the Target
Tracer Code ရဲ့ ရည်ရွယ်ချက်က—

Requirement ကနေ Final System ရဲ့ တစ်စိတ်တစ်ပိုင်းအထိ မြန်မြန်၊ မြင်သာအောင်၊ ထပ်ခါတလဲလဲ စမ်းသပ်နိုင်ခြင်း ဖြစ်ပါတယ်။

Tracer Code ဆိုတာဘာလဲ?

Tracer Code ဟာ Throw-away Code မဟုတ်ပါဘူး။

Final Production Code ထဲမှာ ပါဝင်မယ့် Code ဖြစ်ပါတယ်။

အစပိုင်းမှာ Functionality မပြည့်စုံသေးပေမယ့်—

- Error Checking
- Structure
- Documentation
- Self-checking

တွေကို ထည့်ထားပါတယ်။

ဥပမာ System တစ်ခုမှာ—

User Interface
↓
Business Logic
↓
Database

ဆိုတဲ့ End-to-End Connection ကို အစောပိုင်းမှာ အရင်အလုပ်လုပ်အောင် လုပ်ထားနိုင်ပါတယ်။

Functionality နည်းနည်းပဲရှိသေးပေမယ့် System တစ်ခုလုံး ဘယ်လိုချိတ်ဆက်ထားတယ်ဆိုတာ မြင်နိုင်ပါပြီ။

Tracer Bullets ရဲ့ အကျိုးကျေးဇူး

- User တွေ Early Working Version ကို မြင်နိုင်တယ်
- Feedback စောစောရနိုင်တယ်
- Developer တွေ Architecture Skeleton ရလာတယ်
- Integration ကို နေ့တိုင်းလုပ်နိုင်တယ်
- Debugging နဲ့ Testing ပိုလွယ်တယ်
- Progress ကို တိုင်းတာရလွယ်တယ်
- Big-bang Integration ကို ရှောင်နိုင်တယ်

11. Prototypes and Post-it Notes

Car Manufacturer တွေက Car အသစ်တစ်စီးမထုတ်ခင် Prototype အမျိုးမျိုး ပြုလုပ်ကြပါတယ်။

တစ်ခုက Aerodynamics စမ်းဖို့၊ တစ်ခုက Styling စမ်းဖို့၊ တစ်ခုက Structural Characteristics စမ်းဖို့ ဖြစ်နိုင်ပါတယ်။

Software မှာလည်း အလားတူ—

Risk ရှိတဲ့အရာ၊ မသေချာတဲ့အရာ၊ မစမ်းဖူးသေးတဲ့အရာတွေကို Prototype လုပ်နိုင်ပါတယ်။

TIP 16
Prototype to Learn

Prototype ရဲ့ တန်ဖိုးက Code ကို ထုတ်လုပ်ဖို့ မဟုတ်ပါဘူး။

Prototype ကနေ သင်ယူရတဲ့ Knowledge က အဓိကပါ။

ဘာတွေကို Prototype လုပ်နိုင်သလဲ?

စာအုပ်မှာ—

- Architecture
- Existing System ထဲက New Functionality
- External Data ရဲ့ Structure/Content
- Third-party Tools/Components
- Performance Issues
- User Interface Design

တွေကို Prototype လုပ်နိုင်တယ်လို့ ဖော်ပြထားပါတယ်။

Prototype မှာ ဘာတွေကို လျစ်လျူရှုနိုင်သလဲ?

Prototype ဆိုတာ Final Product မဟုတ်တဲ့အတွက်—

Correctness

Dummy Data သုံးနိုင်ပါတယ်။

Completeness

Functionality အနည်းငယ်ပဲ ရှိနိုင်ပါတယ်။

Robustness

Error Checking မပြည့်စုံနိုင်ပါတယ်။

Style

Comments/Documentation အများကြီး မလိုအပ်နိုင်ပါဘူး။

Post-it Notes နဲ့ Prototype

Code မရေးဘဲလည်း Prototype လုပ်နိုင်ပါတယ်။

ဥပမာ—

- Whiteboard
- Post-it Notes
- Index Cards
- Simple UI Drawing

တွေကို အသုံးပြုနိုင်ပါတယ်။

Workflow နဲ့ Application Logic လို Dynamic Things တွေအတွက် Post-it Notes က အထူးအသုံးဝင်ပါတယ်။

Prototype နဲ့ Tracer Code မတူပါ

Prototype

→ Specific Problem တစ်ခုကို စမ်းသပ်ဖို့

→ Learning အတွက်

→ Disposable Code ဖြစ်နိုင်တယ်

Tracer Code
→ System တစ်ခုလုံး ဘယ်လိုချိတ်ဆက်မလဲ စမ်းဖို့

→ End-to-End Structure တည်ဆောက်ဖို့

→ Final System ရဲ့ Skeleton အဖြစ် ဆက်အသုံးပြုမယ်

ဖြစ်ပါတယ်။

12. Problem Domain အတွက် သီးခြား Language များ

Programming Language တစ်ခုကို အသုံးပြုတဲ့အခါ Implementation Details တွေထဲမှာ အချိန်အများကြီး ကုန်နိုင်ပါတယ်။

Domain Language ရဲ့ ရည်ရွယ်ချက်က—

Programmer ကို Problem Domain နဲ့ ပိုနီးကပ်အောင် လုပ်ပေးခြင်း

ဖြစ်ပါတယ်။

TIP 17
Program Close to the Problem Domain

Higher Level of Abstraction နဲ့ Code ရေးနိုင်ရင် Implementation Details တွေကို အလွန်အကျွံ စဉ်းစားစရာမလိုတော့ဘဲ

Domain Problem ကို ဖြေရှင်းဖို့ ပိုအာရုံစိုက်နိုင်ပါတယ်။

Mini-Languages

Application အတွက် ကိုယ်ပိုင် Mini-Language တစ်ခု ဖန်တီးနိုင်ပါတယ်။

ဥပမာ Business Rule တွေကို—

```
IF customer.status = GOLD
THEN discount = 20%
```

လိုမျိုး Domain နဲ့ နီးစပ်တဲ့ Syntax နဲ့ ရေးနိုင်ပါတယ်။

ဒီလိုဆိုရင် Programmer တင်မကဘဲ Domain ကို နားလည်တဲ့ User/Operator တွေလည်း ပိုလွယ်ကူစွာ နားလည်နိုင်ပါတယ်။

Domain-Specific Error Messages

General Programming Language သုံးရင် Error က—

```
Syntax error: undeclared identifier
```

လိုမျိုး ဖြစ်နိုင်ပါတယ်။

Domain Language သုံးရင်—

```
"AB123" is not a format.
Known formats are ABC123, XYZ43B, PDQB, and 42.
```

လိုမျိုး User နားလည်တဲ့ Domain Vocabulary နဲ့ Error ပြနိုင်ပါတယ်။

Mini-Language ကို ဘယ်လိုတည်ဆောက်မလဲ?

ရိုးရှင်းတဲ့ Mini-Language ဆိုရင်—

- switch statements
- Regular Expressions
- Scripting Languages

နဲ့ Parse လုပ်နိုင်ပါတယ်။

ပိုရှုပ်ထွေးတဲ့ Language ဆိုရင်—

BNF (Backus-Naur Form)
နဲ့ Syntax ကို အရင်သတ်မှတ်ပြီး Parser Generator တွေကို အသုံးပြုနိုင်ပါတယ်။

Data Languages နှင့် Imperative Languages

Domain Language တွေကို အဓိကအားဖြင့် နှစ်မျိုး စဉ်းစားနိုင်ပါတယ်—

Data Language

Application က အသုံးပြုမယ့် Data Structure/Configuration ကို ဖော်ပြဖို့ အသုံးပြုပါတယ်။

Imperative Language

ဘာလုပ်ရမလဲ ဆိုတဲ့ Procedure/Action တွေကို ဖော်ပြပါတယ်။

အဓိကရည်ရွယ်ချက်က—

Problem Domain နဲ့ ပိုနီးစပ်တဲ့ Language ကို အသုံးပြုပြီး Implementation Details တွေကို လျှော့ချခြင်း ဖြစ်ပါတယ်။

13. Estimating - အချိန်နှင့် Resource ကို ခန့်မှန်းခြင်း

Programming မှာ မေးခွန်းအများကြီးက Estimate လုပ်ဖို့လိုပါတယ်။

ဥပမာ—

- File တစ်ခု Download လုပ်ဖို့ ဘယ်လောက်ကြာမလဲ?
- Data ၁ သန်းကို သိမ်းဖို့ ဘယ်လောက် Disk Space လိုမလဲ?
- Network ကနေ Data တစ်ခု ပို့ဖို့ ဘယ်လောက်ကြာမလဲ?
- Project ပြီးဖို့ ဘယ်နှလောက်ကြာမလဲ?

ဒီမေးခွန်းတွေမှာ Information အပြည့်အစုံ မရှိရင် Exact Answer မပေးနိုင်ပါဘူး။

ဒါပေမယ့် **Estimate** လုပ်နိုင်ပါတယ်။

TIP 18
Estimate to Avoid Surprises

Estimate လုပ်တတ်ရင် System တစ်ခုရဲ့ Feasibility ကို ကြိုတင်သိနိုင်ပြီး ဘယ်နေရာမှာ Optimization လုပ်ဖို့လိုမလဲဆိုတာလည်း ပိုကောင်းကောင်း ဆုံးဖြတ်နိုင်ပါတယ်။

Estimate ဘယ်လောက်တိကျရမလဲ?

Estimate ရဲ့ Accuracy က Context ပေါ်မူတည်ပါတယ်။

ဥပမာ—

လူတစ်ယောက်က "ဘယ်အချိန်ရောက်မလဲ?" လို့ မေးရင် နာရီဝက်လောက်အထိ ခန့်မှန်းရုံနဲ့ လုံလောက်နိုင်ပါတယ်။

ဒါပေမယ့် အရေးပေါ်အခြေအနေမှာတော့ Second အထိ တိကျဖို့လိုနိုင်ပါတယ်။

ဒါကြောင့် Estimate ပေးတဲ့အခါ—

"ဒီ Estimate ကို ဘယ်လောက်တိကျဖို့ လိုသလဲ?"

ဆိုတာကို အရင်စဉ်းစားပါ။

Estimate ရဲ့ Unit ကိုလည်း ရွေးပါ

Project Duration ကို—

- 1–15 days → **days**
- 3–8 weeks → **weeks**
- 8–30 weeks → **months**
- 30+ weeks → **အရမ်းသေချာစဉ်းစားပြီးမှ Estimate ပေးပါ**

လို့ စာအုပ်က အကြံပြုထားပါတယ်။

ဥပမာ 125 Working Days ဆိုရင် "125 days" လို့ ပြောတာထက် **"about six months"** လို့ ပြောတာက ကိုယ်ပေးလိုတဲ့ Accuracy Level ကို ပိုကောင်းစွာ ဖော်ပြနိုင်ပါတယ်။

Estimate ဘယ်ကရမလဲ?

အခြေခံနည်းလမ်းတစ်ခုက—

အဲဒီအလုပ်ကို အရင်လုပ်ဖူးတဲ့သူကို မေးပါ။

Exact Match မဖြစ်နိုင်ပေမယ့် အရင်အတွေ့အကြုံတွေက Estimate အတွက် အလွန်အသုံးဝင်ပါတယ်။

1. Understand What's Being Asked

Estimate မလုပ်ခင်—

- ဘာကို Estimate လုပ်နေတာလဲ?
- Scope ဘယ်လောက်လဲ?
- ဘယ်လို Conditions တွေရှိလဲ?
- Accuracy ဘယ်လောက်လိုလဲ?

ဆိုတာတွေကို ရှင်းအောင်လုပ်ပါ။

2. Build a Model

Problem ကို ရိုးရှင်းတဲ့ Mental Model တစ်ခုအဖြစ် တည်ဆောက်ပါ။

ဥပမာ Project Schedule Estimate လုပ်မယ်ဆိုရင်—

Requirements
↓
Design
↓
Implementation
↓
Testing
↓
Integration

လိုမျိုး အကြမ်းဖျင်း Model တစ်ခု တည်ဆောက်နိုင်ပါတယ်။

Model တည်ဆောက်တဲ့အခါ Problem ထဲက မမြင်ရသေးတဲ့ Pattern တွေကိုပါ ရှာတွေ့နိုင်ပါတယ်။

3. Break the Model into Components

Model ကို အစိတ်အပိုင်းငယ်တွေ ခွဲပါ။

Component တစ်ခုချင်းစီမှာ—

- Parameter
- Input
- Contribution

တွေကို သတ်မှတ်ပါ။

4. Give Each Parameter a Value

Parameter တစ်ခုချင်းစီကို Value ပေးပါ။

ဒါပေမယ့် Parameter အားလုံးက Result ကို တူညီတဲ့အတိုင်းအတာနဲ့ မထိခိုက်ပါဘူး။

အထူးသဖြင့် Multiply/Divide လုပ်တဲ့ Parameter တွေက Result ကို ပိုပြီး ထိခိုက်နိုင်ပါတယ်။

ဒါကြောင့် **Result ကို အများဆုံးသက်ရောက်စေတဲ့ Parameter တွေကို အထူးအာရုံစိုက်ပါ။**

5. Calculate the Answers

Complex Estimate တစ်ခုမှာ Answer တစ်ခုတည်း မဟုတ်နိုင်ပါဘူး။

Critical Parameters တွေကို အမျိုးမျိုးပြောင်းပြီး Calculation အများကြီး လုပ်ပါ။

ဥပမာ—

48 MB memory → 1.0 second
64 MB memory → 0.75 second

လိုမျိုး Conditions အလိုက် Estimate ပေးနိုင်ပါတယ်။

Estimate ကို Record လုပ်ပါ

ကိုယ်ပေးခဲ့တဲ့ Estimate တွေကို မှတ်တမ်းတင်ထားပါ။

နောက်ပိုင်း Actual Result နဲ့ ပြန်နှိုင်းယှဉ်ပါ။

Estimate မှားခဲ့ရင်—

“ဘာကြောင့် မှားသွားတာလဲ?”

ဆိုတာကို ရှာပါ။

Model မှားတာလား၊ Parameter မှားတာလား၊ Assumption မှားတာလား သိလာရင် နောက်တစ်ကြိမ် Estimate က ပိုကောင်းလာပါမယ်။

Project Schedule Estimating

Project ကြီးတွေမှာ အစကတည်းက Final Schedule တစ်ခုတည်းကို တိတိကျကျ သတ်မှတ်ဖို့ ခက်ပါတယ်။

စာအုပ်က Incremental Development နဲ့—

1. Requirements စစ်ပါ
2. Risk Analyze လုပ်ပါ
3. Design / Implement / Integrate လုပ်ပါ
4. User နဲ့ Validate လုပ်ပါ

ဆိုတဲ့ Cycle ကို ထပ်ခါထပ်ခါ လုပ်ဖို့ အကြံပြုပါတယ်။

TIP 19
Iterate the Schedule with the Code

Code တိုးတက်လာတာနဲ့အမျှ Schedule Estimate ကိုလည်း ပြန်လည်ပြင်ဆင်ပါ။

အစက Estimate က Guess ပိုများနိုင်ပေမယ့် Iteration တစ်ခုပြီးတိုင်း ရလာတဲ့ Actual Experience ကို အသုံးပြုပြီး Schedule ကို ပိုတိကျအောင် ပြင်ဆင်နိုင်ပါတယ်။

အခန်း ၃ — အခြေခံ Tools များ

Craftsman တစ်ယောက်ဟာ အစပိုင်းမှာ အရည်အသွေးကောင်းတဲ့ အခြေခံ Tools တွေနဲ့ စတင်ရပါတယ်။

Woodworker တစ်ယောက်အတွက်—

- Ruler
- Gauge
- Saw
- Plane
- Chisel
- Drill
- Mallet
- Clamp

စတာတွေက အခြေခံ Tools တွေ ဖြစ်ပါတယ်။

အဲဒီ Tools တွေကို အချိန်ကြာလာတာနဲ့အမျှ ကျွမ်းကျင်လာပြီး ကိုယ့်လက်ရဲ့ အစိတ်အပိုင်းတစ်ခုလို အသုံးပြုလာနိုင်ပါတယ်။

Programmer တွေလည်း အတူတူပါပဲ။

Tools က ကိုယ့်ရဲ့ Talent ကို တိုးမြှင့်ပေးပါတယ်။

Tool ကောင်းကောင်းရှိရုံနဲ့ မလုံလောက်ပါဘူး။ အဲဒီ Tool ကို ဘယ်လိုအသုံးပြုရမလဲဆိုတာလည်း ကျွမ်းကျင်ရပါမယ်။

14. The Power of Plain Text

Pragmatic Programmer တွေရဲ့ အခြေခံကုန်ကြမ်းဟာ သစ်သား၊ သံ မဟုတ်ပါဘူး။

Knowledge — အသိပညာ ဖြစ်ပါတယ်။

Requirements တွေကို Knowledge အဖြစ် စုဆောင်းပြီး—

- Design
- Implementation
- Tests
- Documentation

တွေထဲမှာ ဖော်ပြကြပါတယ်။

အဲဒီ Knowledge ကို ရေရှည်သိမ်းဆည်းဖို့ အကောင်းဆုံး Format က **Plain Text** ဖြစ်တယ်လို့ စာအုပ်က ဆိုပါတယ်။

What Is Plain Text?

Plain Text ဆိုတာ လူတွေက တိုက်ရိုက်ဖတ်ပြီး နားလည်နိုင်တဲ့ Printable Characters တွေနဲ့ ရေးထားတဲ့ Text ဖြစ်ပါတယ်။

ဥပမာ—

Field19=467abe

ဆိုရင် 467abe က ဘာကိုဆိုလိုတယ်ဆိုတာ လူက ခန့်မှန်းဖို့ခက်ပါတယ်။

ဒီလိုရေးမယ့်အစား—

DrawingType=UMLActivityDrawing

ဆိုရင် အဓိပ္ပာယ်ကို လူက တိုက်ရိုက်နားလည်နိုင်ပါတယ်။

Plain Text ဆိုတာ Structure မရှိတဲ့ Text လို့ မဆိုလိုပါဘူး။

XML, SGML, HTML တို့ဟာ Plain Text ဖြစ်ပေမယ့် Structure ကောင်းကောင်း သတ်မှတ်ထားပါတယ်။

Plain Text vs Binary

ဥပမာ uses_menus ဆိုတဲ့ Property တစ်ခုကို TRUE/FALSE သိမ်းမယ်ဆိုရင်—

myprop.uses_menus=FALSE

လို ရေးနိုင်ပါတယ်။

Binary Format နဲ့ဆို—

0010010101110101

လို ဖြစ်နိုင်ပါတယ်။

Binary Data ရဲ့ ပြဿနာက Data ကို နားလည်ဖို့လိုအပ်တဲ့ Context ဟာ Data ကိုယ်တိုင်နဲ့ သီးခြားဖြစ်နေတာပါ။

Plain Text ကတော့ **Self-describing Data** ဖြစ်အောင် လုပ်နိုင်ပါတယ်။

Plain Text ရဲ့ အားသာချက်များ

Plain Text ကို အသုံးပြုရင်—

- လူတွေ တိုက်ရိုက်ဖတ်နိုင်တယ်
- Version Control လုပ်နိုင်တယ်
- Diff Tools အသုံးပြုနိုင်တယ်
- Search လုပ်ရလွယ်တယ်
- Script တွေနဲ့ Process လုပ်ရလွယ်တယ်
- Test Data ပြင်ဆင်ရလွယ်တယ်
- System ပျက်သွားရင်လည်း အလွယ်တကူ ပြန်ဖတ်နိုင်တယ်

Production Configuration File ကို Plain Text နဲ့ထားရင် Source Control ထဲထည့်ပြီး ပြောင်းလဲမှုတိုင်းကို History သိမ်းထားနိုင်ပါတယ်။ diff လို Tools တွေနဲ့ ဘာတွေပြောင်းလဲသွားလဲ ကြည့်နိုင်ပါတယ်။

Drawbacks

Plain Text ရဲ့ အဓိကအားနည်းချက် ၂ ခုက—

1. Binary Format ထက် Storage Space ပိုယူနိုင်တယ်။
2. Read/Process လုပ်ရာမှာ Computational Cost ပိုများနိုင်တယ်။

ဒါပေမယ့် ရေရှည်မှာ Human Readability နဲ့ Flexibility က အလွန်အသုံးဝင်ပါတယ်။

TIP 20 - Knowledge ကို Plain Text ထဲမှာ သိမ်းပါ

15. Command Shell ကို အသုံးချခြင်း

Woodworker တစ်ယောက်အတွက် Workbench က အရေးကြီးသလို Programmer အတွက် **Command Shell** က Workbench တစ်ခုလို ဖြစ်ပါတယ်။

Shell Prompt ကနေ—

- Program တွေ Run နိုင်တယ်
- Tools တွေ ခေါ်သုံးနိုင်တယ်
- Pipe နဲ့ Tools တွေ ပေါင်းစပ်နိုင်တယ်
- File တွေ Search လုပ်နိုင်တယ်
- System Status စစ်နိုင်တယ်
- Output ကို Filter လုပ်နိုင်တယ်
- Debugger, Browser, Editor တွေ Launch လုပ်နိုင်တယ်
- ကိုယ်ပိုင် Automation Command တွေ ရေးနိုင်တယ်

ဆိုတဲ့ အလုပ်တွေ လုပ်နိုင်ပါတယ်။

GUI တစ်ခုတည်းကို မမှီခိုပါနဲ့

GUI နဲ့ Point-and-Click လုပ်ရတာ အဆင်ပြေပါတယ်။

ဒါပေမယ့် Programmer တစ်ယောက်အနေနဲ့ GUI ရဲ့ ကန့်သတ်ချက်ထဲမှာပဲ မနေသင့်ပါဘူး။

Command Shell ကို ကျွမ်းကျင်ရင်—

Command တွေကို ပေါင်းစပ်ပြီး အလုပ်ကြီးတွေကို အလိုအလျောက် လုပ်နိုင်ပါတယ်။

ဥပမာ—

find
grep
sort
filter
redirect
pipe

စတဲ့ Tools တွေကို ပေါင်းစပ်အသုံးပြုနိုင်ပါတယ်။

Shell ကို Programming လုပ်ပါ

Shell ကို Command တစ်ခုချင်းစီ Run ဖို့အတွက်ပဲ မသုံးပါနဲ့။

မကြာခဏလုပ်ရတဲ့ အလုပ်တွေကို Script အဖြစ် ရေးထားပါ။

ဥပမာ—

build
test
package
deploy

ဆိုတဲ့ အလုပ်တွေကို Script တစ်ခုနဲ့ Run လုပ်နိုင်ပါတယ်။

ဒီလိုဆိုရင်—

Computer ကို ကိုယ်လုပ်စရာမလိုဘဲ ကိုယ့်အတွက် အလုပ်လုပ်ခိုင်းနိုင်ပါတယ်။

Portability

Environment အသစ်တစ်ခုကို ပြောင်းသွားတဲ့အခါ အဲဒီ Environment မှာ ဘယ် Shell တွေ ရှိသလဲ လေ့လာပါ။

ကိုယ်သုံးနေကျ Shell ကို Environment အသစ်မှာလည်း ယူသုံးနိုင်မလား စမ်းကြည့်ပါ။

အစိကသင်ခန်းစာ

Shell ဟာ Programmer ရဲ့ Workbench ဖြစ်ပါတယ်။

Shell ကို ကောင်းကောင်းကျွမ်းကျင်ရင် နေ့စဉ်အလုပ်တွေကို အလွန်မြန်မြန်ဆန်ဆန် လုပ်နိုင်ပါတယ်။

16. Editor ကို အစွမ်းကုန် အသုံးချခြင်း

Programming ရဲ့ အခြေခံကုန်ကြမ်းဟာ Text ဖြစ်တဲ့အတွက် **Editor** ဟာ အရေးကြီးဆုံး Tools တွေထဲက တစ်ခု ဖြစ်ပါတယ်။

Text ကို အလွယ်တကူ Manipulate လုပ်နိုင်ရပါမယ်။

Editor ကို ကိုယ့်လက်ရဲ့ အစိတ်အပိုင်းတစ်ခုလို အသုံးပြုနိုင်တဲ့အထိ ကျွမ်းကျင်သင့်ပါတယ်။

One Editor

Editor တစ်ခုကို ကျွမ်းကျင်အောင် သုံးပါ

စာအုပ်က Editor အများကြီး သုံးမယ့်အစား **Editor တစ်ခုကို အလွန်ကျွမ်းကျင်အောင် လေ့လာပြီး အလုပ်အမျိုးမျိုး အတွက် အသုံးပြုဖို့** အကြံပြုပါတယ်။

ဥပမာ—

- Code
- Documentation
- Memo
- System Administration
- Email
- Configuration Files

စတာတွေအတွက် Editor တစ်ခုတည်းကို အသုံးပြုနိုင်ပါတယ်။

Editor မျိုးစုံသုံးရင် Keyboard Shortcuts နဲ့ Editing Conventions တွေ မတူတော့ဘဲ Efficiency ကျနိုင်ပါတယ်။

Keyboard ကို ကျွမ်းကျင်ပါ

Mouse နဲ့—

Click → Select → Copy → Paste
လုပ်နေရုံနဲ့ မလုံလောက်ပါဘူး။

Powerful Editor တစ်ခုမှာ Keyboard Shortcuts တွေကို ကျွမ်းကျင်ထားရင် Text Manipulation ကို အလွန်မြန်မြန် လုပ်နိုင်ပါတယ်။

ဥပမာ Cursor ကို Line ရဲ့အစကို ရွှေ့ဖို့ Backspace အကြိမ်များစွာနှိပ်တာထက် Shortcut တစ်ခုသုံးတာက ပိုမြန်ပါတယ်။

TIP 22

Editor တစ်ခုကို ကောင်းကောင်းအသုံးပြုပါ

Editor တစ်ခုကို ရွေးပါ။

အဲဒီ Editor ရဲ့—

- Commands
- Shortcuts
- Search
- Replace
- Macros
- Configuration
- Extensions

တွေကို ကျွမ်းကျင်အောင် လေ့လာပါ။

ရည်ရွယ်ချက်က—

Editor ကို စဉ်းစားပြီး အသုံးပြုနေရတာမဟုတ်ဘဲ လက်က အလိုအလျောက် လုပ်နိုင်တဲ့အဆင့်ရောက်ဖို့ပါ။

17. Source Code ကို ထိန်းချုပ်ခြင်း

Programmer တစ်ယောက်အတွက် Source Code ဟာ တန်ဖိုးရှိတဲ့ အရာပါ။

ဒါကြောင့် Source Code ကို—

အမြဲ Source Code Control System အောက်မှာ ထားသင့်ပါတယ်။

ကိုယ့်ရဲ့ Personal Address Book လို အရာတွေတောင် Version Control လုပ်နိုင်ပါတယ်။

Source Code Control ရဲ့ အားသာချက်

Source Code Control System က—

- ပြောင်းလဲမှု History သိမ်းပေးတယ်
- ဘယ်သူက ဘာပြောင်းလဲတယ်ဆိုတာ သိနိုင်တယ်
- အရင် Version ကို ပြန်သွားနိုင်တယ်
- Branch ခွဲနိုင်တယ်
- Team နဲ့ အတူတကွ အလုပ်လုပ်နိုင်တယ်
- အမှားလုပ်မိရင် ပြန်ပြင်နိုင်တယ်

အထူးသဖြင့် Source Code Control ဟာ **Time Machine** တစ်ခုလို ဖြစ်ပါတယ်။

မှားယွင်းမှုဖြစ်သွားရင် အရင်အခြေအနေကို ပြန်သွားနိုင်ပါတယ်။

Everything Under Source Control

Source Control အောက်မှာ Code တစ်ခုတည်းကိုပဲ မထားသင့်ပါဘူး။

လိုအပ်ရင်—

- Documentation
- Configuration
- Build Scripts
- Test Data
- Database Schema
- Deployment Scripts

စတာတွေကိုပါ ထည့်သွင်းစဉ်းစားသင့်ပါတယ်။

TIP 23

Source Code Control ကို အမြဲအသုံးပြုပါ

မိမိရေးထားတဲ့ အလုပ်တန်ဖိုးရှိသမျှကို Version Control ထဲမှာ ထားပါ။

18. Debugging

Programmer တစ်ယောက်အနေနဲ့ Bug မဖြစ်အောင် ကြိုးစားရပါမယ်။

ဒါပေမယ့် Bug တွေ မဖြစ်ဘူးလို့ မယူဆသင့်ပါဘူး။

Bug ဖြစ်လာတဲ့အခါ မြန်မြန်နဲ့ မှန်မှန် Debug လုပ်နိုင်ရပါမယ်။

Fix the Problem, Not the Blame

အပြစ်ရှိသူကို မရှာဘဲ Problem ကို ဖြေရှင်းပါ

Bug က ကိုယ့်အမှားဖြစ်ဖြစ်၊ တခြားသူ့အမှားဖြစ်ဖြစ်—

အခုချိန်မှာ အဲဒါဟာ ကိုယ့်ရဲ့ Problem ဖြစ်နေပါတယ်။

ဒါကြောင့် အပြစ်တင်မယ့်အစား Root Cause ကို ရှာပြီး ဖြေရှင်းပါ။

TIP 24
Fix the Problem, Not the Blame

Bug တစ်ခုတွေ့တဲ့အခါ—

ရပ်ပါ။ အသက်ရှူပါ။ စဉ်းစားပါ။

ချက်ချင်း Code ကို Random ပြင်မနေပါနဲ့။

အရင်ဆုံး—

- ဘာဖြစ်နေတာလဲ?
- ဘာကြောင့်ဖြစ်တာလဲ?
- ဘယ်အခြေအနေမှာဖြစ်တာလဲ?
- ဘယ်အချိန်ကစဖြစ်တာလဲ?
- ဘယ် Data နဲ့ဖြစ်တာလဲ?

ဆိုတာတွေကို သိအောင်လုပ်ပါ။

Find the Root Cause

Bug ရဲ့ လက္ခဏာကိုပဲ Fix မလုပ်ပါနဲ့။

Root Cause ကို ရှာပါ။

စာအုပ်ကလည်း Problem ရဲ့ ဒီ Particular Appearance တစ်ခုတည်းကို မဟုတ်ဘဲ Root Cause ကို ရှာဖို့ အလေးပေးထားပါတယ်။

Where to Start

Debugging မစခင်—

Code ကို Warning မရှိဘဲ Compile ဖြစ်အောင်လုပ်ပါ။

Compiler က ရှာပေးနိုင်တဲ့ Problem ကို ကိုယ်တိုင် အချိန်ကုန်ခံပြီး မရှာသင့်ပါဘူး။ Compiler Warning Level ကို ဖြစ်နိုင်သမျှ မြင့်ထားဖို့ စာအုပ်က အကြံပြုပါတယ်။

Gather Accurate Data

Bug Report တစ်ခုကို ကြားလိုက်တာနဲ့ ချက်ချင်း Conclusion မချပါနဲ့။

သက်ဆိုင်ရာ Data အားလုံးကို စုဆောင်းပါ။

တစ်ခါတလေ User ကိုယ်တိုင် Bug ဖြစ်ပုံကို ပြန်ကြည့်ဖို့ လိုနိုင်ပါတယ်။

Artificial Test တစ်ခုတည်းနဲ့ မလုံလောက်ပါဘူး။

Boundary Conditions နဲ့ Realistic User Usage Pattern နှစ်ခုလုံးကို စနစ်တကျ စမ်းသပ်ရပါမယ်။

Reproduce the Bug

Bug ကို Fix မလုပ်ခင်—

ပြန်ဖြစ်အောင်လုပ်နိုင်ဖို့ ကြိုးစားပါ။

Bug ကို ပြန်မဖြစ်အောင် မလုပ်နိုင်ရင် Fix ဖြစ်သွားပြီလားဆိုတာ Verify လုပ်ဖို့ ခက်ပါတယ်။

အကောင်းဆုံးက Bug ကို Step ၁၅ ဆင့် လုပ်ပြီးမှ ဖြစ်လာတာမျိုးမဟုတ်ဘဲ—

Command တစ်ခုတည်းနဲ့ Reproduce လုပ်နိုင်တဲ့အဆင့် ဖြစ်အောင် လျှော့ချနိုင်ဖို့ ကြိုးစားပါ။

Visualize Your Data

Program ဘာလုပ်နေတယ်ဆိုတာ နားလည်ဖို့ Data ကို မြင်အောင်လုပ်တာ အလွန်အသုံးဝင်ပါတယ်။

ဥပမာ—

variable_name = value
လိုမျိုး Output ထုတ်ပြီး Program ရဲ့ State ကို ကြည့်နိုင်ပါတယ်။

TIP 25

Debugging လုပ်တဲ့အခါ မထိတ်လန့်ပါနဲ့

အေးအေးဆေးဆေး စဉ်းစားပါ။

Root Cause ကိုရှာပါ။

Debugging Checklist

Bug ဖြစ်လာရင်—

- Report လုပ်ထားတဲ့ Problem က Root Cause လား၊ Symptom လား?
- Compiler လား?
- OS လား?
- ကိုယ့် Code လား?
- Coworker တစ်ယောက်ကို ရှင်းပြရမယ်ဆိုရင် ဘယ်လိုရှင်းပြမလဲ?
- Unit Tests က လုံလောက်ရဲ့လား?
- ဒီ Data နဲ့ Unit Test Run လုပ်ရင် ဘာဖြစ်မလဲ?
- ဒီ Bug ဖြစ်စေတဲ့ Condition က System ရဲ့ တခြားနေရာမှာ ရှိနိုင်သလား?

ဆိုတာတွေကို စစ်ဆေးပါ။

19. Text Manipulation (Text ကို ပြုပြင်အသုံးချခြင်း)

Pragmatic Programmer တွေဟာ Woodworker က Wood ကို ပုံဖော်သလို **Text ကို Manipulate လုပ်ကြပါတယ်။**

Shell၊ Editor၊ Debugger တွေဟာ သီးခြားအလုပ်တွေကို ကောင်းကောင်းလုပ်ပေးတဲ့ Tools တွေပါ။

ဒါပေမယ့် တစ်ခါတလေ Basic Tools တွေနဲ့ မလွယ်တဲ့ Transformation တွေ လုပ်ဖို့လိုလာပါတယ်။

အဲဒီအခါ **Text Manipulation Language** တွေက အသုံးဝင်ပါတယ်။

Text Manipulation Languages

အသုံးများတဲ့ Tools/Languages တွေထဲမှာ—

- awk
- sed
- Python
- Tcl
- Ruby
- Perl

စတာတွေ ပါပါတယ်။

ဒီ Tools တွေကို အသုံးပြုရင် Conventional Programming Language နဲ့ ရေးရင် အချိန် ၅ ဆ၊ ၁၀ ဆ ကြာနိုင်တဲ့ အလုပ်တွေကို အလွန်မြန်မြန် ပြုလုပ်နိုင်ပါတယ်။

ဥပမာ Experiment တစ်ခုကို—

၅ နာရီရေးမယ့်အစား ၃၀ မိနစ်နဲ့ စမ်းကြည့်နိုင်တာ

ဟာ အလွန်အရေးကြီးပါတယ်။

What Can You Do?

Text Manipulation Language တွေကို—

- Database Schema ထုတ်လုပ်ခြင်း
- Test Data ပြင်ဆင်ခြင်း
- Source Code ပြောင်းလဲခြင်း
- Documentation Generate လုပ်ခြင်း
- Configuration ပြင်ဆင်ခြင်း
- Files အများကြီးကို တစ်ပြိုင်နက် Process လုပ်ခြင်း

စတာတွေအတွက် အသုံးပြုနိုင်ပါတယ်။

ဥပမာ Database Schema တစ်ခုကနေ—

- SQL
- Data Dictionary
- C Libraries
- Integrity Check Scripts
- Web Documentation
- XML

တွေကို Automatically Generate လုပ်နိုင်ပါတယ်။

TIP 28

Text Manipulation Language တစ်ခုကို လေ့လာပါ

နေ့စဉ်အလုပ်ထဲမှာ Text နဲ့ အများကြီး အလုပ်လုပ်နေရရင်—

Computer ကို အဲဒီအလုပ်တချို့ လုပ်ခိုင်းပါ။

20. Code Generators

Woodworker တစ်ယောက်က တူညီတဲ့အရာကို အကြိမ်ကြိမ် ပြုလုပ်ရမယ်ဆိုရင် **Jig** ဒါမှမဟုတ် **Template** တစ်ခုလုပ်ထားတတ်ပါတယ်။

တစ်ကြိမ်မှန်အောင်လုပ်ထားရင် နောက်ပိုင်းမှာ ထပ်ခါထပ်ခါ အသုံးပြုနိုင်ပါတယ်။

Programmer တွေလည်း အလားတူ—

Code က Code ကို ရေးပေးနိုင်ပါတယ်။

ဒါကို **Code Generator** လို့ ခေါ်ပါတယ်။

TIP 29

Code ရေးတဲ့ Code ကို ရေးပါ

Code Generator ရဲ့ အဓိကအမျိုးအစား ၂ ခု ရှိပါတယ်။

1. Passive Code Generators

တစ်ကြိမ် Run လုပ်ပြီး Output ထုတ်ပေးပါတယ်။

ပြီးရင် ထွက်လာတဲ့ Output ဟာ Project ရဲ့ ပုံမှန် Source File တစ်ခုလို ဖြစ်သွားပါတယ်။

ဥပမာ—

- New Source File Template
- Copyright Header
- Standard Comment Block
- Lookup Tables
- Language Conversion

စတာတွေကို Generate လုပ်နိုင်ပါတယ်။

2. Active Code Generators

လိုအပ်တဲ့အချိန်တိုင်း Run လုပ်ပြီး Output ကို ပြန် Generate လုပ်နိုင်ပါတယ်။

Generated Result က Temporary ဖြစ်ပါတယ်။

အဓိကအားသာချက်က **DRY Principle** ကို လိုက်နာနိုင်ခြင်းပါ။

Knowledge တစ်ခုကို နေရာအများကြီးမှာ ထပ်ရေးမယ့်အစား Source တစ်နေရာတည်းကနေ လိုအပ်တဲ့ Forms အားလုံးကို Generate လုပ်နိုင်ပါတယ်။

Example — Database Schema

Database Schema မှာ Column တစ်ခုထပ်လာရင်—

Database ဘက်မှာ တစ်နေရာပြင်ရုံနဲ့ Programming Code ထဲမှာလည်း ပြန်ပြင်ရမယ်ဆိုရင် Knowledge ကို နှစ်နေရာမှာ သိမ်းထားရပါတယ်။

ဒါဟာ **Duplication** ဖြစ်ပါတယ်။

Active Code Generator သုံးရင်—

Database Schema
↓
Code Generator
↓
Program Structures

လိုမျိုး Schema တစ်ခုတည်းကို Source of Truth အဖြစ်ထားနိုင်ပါတယ်။

Schema ပြောင်းသွားတဲ့အခါ Code ကို ပြန် Generate လုပ်နိုင်ပါတယ်။

Chapter 3 ရဲ့ အဓိကအယူအဆ

Tools တွေဟာ Programmer ရဲ့ Talent ကို မြှင့်တင်ပေးတဲ့ အရာတွေပါ။ Tool ကောင်းကောင်းရှိရုံမက အဲဒီ Tool တွေကို ကျွမ်းကျင်အောင် လေ့ကျင့်ပြီး ကိုယ့်လက်ရဲ့ Extension တစ်ခုလို အသုံးပြုနိုင်အောင်လုပ်ရပါမယ်။

Chapter 4 — Pragmatic Paranoia (သတိကြီးကြီးထားပြီး Programming လုပ်ခြင်း)

Software ရေးတဲ့အခါ **"ဘာမှမမှားနိုင်ဘူး"** လို့ မယူဆသင့်ပါဘူး။

ဘယ်သူမှ Perfect Software မရေးနိုင်ပါဘူး။ ကိုယ်တိုင်လည်း အမှားလုပ်နိုင်ပါတယ်။ အခြားသူတွေရဲ့ Code၊ မမှန်နိုင်တဲ့ Input၊ မမျှော်လင့်ထားတဲ့ Environment တွေနဲ့ အမြဲတမ်း အလုပ်လုပ်နေရတာဖြစ်တဲ့အတွက် **Defensive Programming** လုပ်ဖို့လိုပါတယ်။

Pragmatic Programmer တွေက တခြားသူတွေကိုပဲ မယုံတာမဟုတ်ပါဘူး။ **ကိုယ့်ကိုယ်ကိုတောင် မယုံကြည်ဘဲ ကိုယ့်အမှားတွေကို ကာကွယ်ဖို့ Code ထဲမှာ Defense တွေ ထည့်ထားကြပါတယ်။**

ဒီ Chapter မှာ—

1. Design by Contract
2. Dead Programs Tell No Lies
3. Assertive Programming
4. When to Use Exceptions
5. How to Balance Resources

ဆိုတဲ့ Section ၅ ခုကို ဆွေးနွေးထားပါတယ်။

21. Design by Contract (Contract အပေါ်အခြေခံပြီး Design လုပ်ခြင်း)

လူတွေအကြားမှာ Contract တစ်ခုဆိုတာ နှစ်ဖက်စလုံးရဲ့—

- Rights
- Responsibilities
- Obligations
- မလိုက်နာရင် ဖြစ်လာမယ့် အကျိုးဆက်

တွေကို သတ်မှတ်ပေးပါတယ်။

Software Module တွေကြားမှာလည်း ဒီ Concept ကို အသုံးပြုနိုင်ပါတယ်။

Design by Contract (DBC)

Bertrand Meyer က Eiffel Language အတွက် **Design by Contract** ဆိုတဲ့ Concept ကို ဖန်တီးခဲ့ပါတယ်။

အဓိကရည်ရွယ်ချက်က Software Module တစ်ခုချင်းစီရဲ့—

"ဘာကို လက်ခံမလဲ၊ ဘာကို လုပ်ပေးရမလဲ"

ဆိုတာကို ရှင်းလင်းစွာ သတ်မှတ်ထားခြင်း ဖြစ်ပါတယ်။

Correct Program ဆိုတာ—

ကိုယ်က လုပ်မယ်လို့ ပြောထားတာထက် ပိုမလုပ်ဘဲ၊ လျော့လည်းမလုပ်တဲ့ Program

ဖြစ်ပါတယ်။

Preconditions (ကြိုတင်လိုအပ်ချက်များ)

Precondition ဆိုတာ Function/Method တစ်ခုကို မခေါ်ခင် မှန်နေရမယ့် အခြေအနေ ဖြစ်ပါတယ်။

ဥပမာ—

addItem(item)
ဆိုတဲ့ Function တစ်ခုမှာ item ဟာ null မဖြစ်ရဘူးဆိုရင်—

Precondition:
item must not be null
လို့ သတ်မှတ်နိုင်ပါတယ်။

Precondition ကို ဖြည့်ဆည်းပေးဖို့က **Caller ရဲ့ တာဝန်** ဖြစ်ပါတယ်။

Postconditions (လုပ်ဆောင်ပြီးနောက် အာမခံချက်များ)

Postcondition ဆိုတာ Function က အလုပ်ပြီးသွားတဲ့အခါ မှန်နေရမယ့် အခြေအနေ ဖြစ်ပါတယ်။

ဥပမာ—

addItem(item)
ပြီးသွားတဲ့အခါ—

item must exist in the list
ဆိုတာကို Guarantee လုပ်နိုင်ပါတယ်။

Postcondition ရှိတယ်ဆိုတာ Function ဟာ နောက်ဆုံးမှာ ပြီးဆုံးရမယ်ဆိုတဲ့ အဓိပ္ပာယ်လည်း ပါပါတယ်။ Infinite Loop မဖြစ်သင့်ပါဘူး။

Class Invariants (Class တစ်ခုရဲ့ အမြဲမှန်ရမယ့် အခြေအနေ)

Class Invariant ဆိုတာ Caller ရဲ့ အမြင်အရ Class တစ်ခုအတွက် အမြဲမှန်နေရမယ့် အခြေအနေ ဖြစ်ပါတယ်။

Method တစ်ခုအတွင်း Processing လုပ်နေချိန်မှာ Invariant ပျက်နိုင်ပါတယ်။

ဒါပေမယ့် Method ပြီးပြီး Control က Caller ဆီပြန်ရောက်တဲ့အချိန်မှာတော့ Invariant ပြန်မှန်နေရပါမယ်။

TIP

Contract ကို Design လုပ်တဲ့အချိန်ကတည်းက သတ်မှတ်ပါ

Function တစ်ခုရေးတဲ့အခါ—

- ဘာ Input လက်ခံမလဲ?
- ဘာကို Guarantee လုပ်မလဲ?
- ဘယ်အခြေအနေတွေမှာ မလုပ်နိုင်ဘူးလဲ?

ဆိုတာတွေကို အရင်သတ်မှတ်ပါ။

22. Dead Programs Tell No Lies (သေသွားတဲ့ Program က မလိမ်ဘူး)

Program တစ်ခုမှာ မမျှော်လင့်ထားတဲ့ Error ဖြစ်လာတဲ့အခါ—

ဆက်ပြီး Run နေတာက အမြဲတမ်း ကောင်းတာမဟုတ်ပါဘူး။

တစ်ခါတလေ **Program ကို ရပ်လိုက်တာက အကောင်းဆုံး ဖြေရှင်းနည်း** ဖြစ်နိုင်ပါတယ်။

ဘာကြောင့်လဲဆိုတော့ Error ဖြစ်ပြီးနောက် ဆက်ပြီး Run နေရင်—

- Data ပျက်နိုင်တယ်
- Database ထဲကို Corrupted Data ရေးနိုင်တယ်
- System State ပိုဆိုးလာနိုင်တယ်
- နောက်ထပ် Error တွေ ဆက်ဖြစ်နိုင်တယ်

စာအုပ်က **"Dead program" ဟာ "crippled program" ထက် ပိုနည်းတဲ့ Damage ပဲ လုပ်နိုင်တယ်** ဆိုတဲ့ အယူအဆကို တင်ပြထားပါတယ်။

Crash Early (အမှားတွေ့တာနဲ့ စောစောရပ်ပါ)

Program တစ်ခုမှာ—

"ဒါက ဘယ်တော့မှ မဖြစ်နိုင်ဘူး"

လို့ သတ်မှတ်ထားတဲ့ အခြေအနေတစ်ခု တကယ်ဖြစ်လာပြီဆိုရင် Program ရဲ့ လက်ရှိ State ကို မယုံနိုင်တော့ပါဘူး။

အဲဒီအချိန်မှာ ဆက်လုပ်သမျှဟာ မှန်မမှန် မသေချာတော့ပါဘူး။

ဒါကြောင့်—

မဖြစ်နိုင်ဘူးလို့ သတ်မှတ်ထားတဲ့ အရာ တကယ်ဖြစ်လာရင် Program ကို အမြန်ဆုံး ရပ်ပါ။

Don't Just Hide Errors

Error တစ်ခုကို—

ignore

လုပ်ပြီး ဆက်သွားတာက အန္တရာယ်ရှိပါတယ်။

အထူးသဖြင့်—

- Invalid Data
- Impossible State
- Broken Invariant
- Unexpected Runtime Error

တွေကို ဖုံးကွယ်မထားသင့်ပါဘူး။

Error ကို စောစောတွေ့ရင် အကြောင်းရင်းနဲ့ နီးစပ်နေတဲ့အတွက် Debugging ပိုလွယ်ပါတယ်။

TIP
Crash Early

Program က ဆက်ပြီး အန္တရာယ်ဖြစ်စေမယ့်အစား ပြဿနာကို စောစောဖော်ထုတ်ပြီး ရပ်တန့်ပါ။

23. Assertive Programming (Assertion တွေသုံးပြီး Programming လုပ်ခြင်း)

Programmer တွေဟာ မကြာခဏ—

“ဒါက ဘယ်တော့မှ ဖြစ်မှာမဟုတ်ဘူး”

လို့ ပြောတတ်ကြပါတယ်။

ဥပမာ—

- count က Negative မဖြစ်နိုင်ဘူး။
- ဒီ Function ကို null နဲ့ ဘယ်တော့မှ မခေါ်ဘူး။
- ဒီ File က အမြဲရှိမယ်။
- ဒီ Calculation က အမြဲမှန်မယ်။

ဒီလိုယူဆချက်တွေကို မယုံဘဲ **Code နဲ့ တိုက်ရိုက်စစ်ဆေးသင့်ပါတယ်။**

Assertions

Assertion ဆိုတာ—

“ဒီအခြေအနေက မှန်နေရမယ်”

လို့ Code ထဲမှာ ပြောထားပြီး တကယ်မှန်မမှန် စစ်ဆေးတာ ဖြစ်ပါတယ်။

ဥပမာ—

```
assert(count >= 0)
```

ဆိုရင် count ဟာ Negative ဖြစ်လာတာနဲ့ Assertion က အမှားကို ဖော်ပြပေးပါမယ်။

TIP 33
မဖြစ်နိုင်ဘူးဆိုရင် Assertion သုံးပြီး မဖြစ်အောင်သေချာစစ်ပါ

ကိုယ့်စိတ်ထဲမှာ—

“ဒါက ဘယ်တော့မှ မဖြစ်နိုင်ဘူး”

လို့ တွေးမိတိုင်း Code ထဲမှာ အဲဒီ Assumption ကို စစ်ဆေးနိုင်တဲ့ Check တစ်ခု ထည့်ပါ။

Assertions ကို Production မှာ ပိတ်သင့်သလား?

စာအုပ်က Assertions အားလုံးကို Production မှာ ပိတ်လိုက်တာကို မထောက်ခံပါဘူး။

Testing က Bug အားလုံးကို ရှာတွေ့နိုင်တယ်လို့ မယူဆသင့်ပါဘူး။

Production Environment မှာ—

- Memory ကုန်သွားနိုင်တယ်
- Disk ပြည့်သွားနိုင်တယ်
- Network Cable ပျက်နိုင်တယ်
- Unexpected Input ရောက်လာနိုင်တယ်

စတဲ့ အခြေအနေတွေ ဖြစ်နိုင်ပါတယ်။

ဒါကြောင့် Assertions တွေက Production မှာလည်း အရေးကြီးနိုင်ပါတယ်။ Performance ပြဿနာရှိရင်တော့ တကယ် Performance ကို ထိခိုက်စေတဲ့ Assertion တွေကိုပဲ Selectively ပိတ်ဖို့ စာအုပ်က အကြံပြုထားပါတယ်။

Assertion Side Effects

Assertion ထဲမှာ Program ရဲ့ State ကို ပြောင်းလဲစေတဲ့ Operation မထည့်သင့်ပါဘူး။

ဥပမာ—

```
assert(iter.nextElement() != null)
```

လိုမျိုး Assertion က Iterator ကို ရွှေ့သွားစေတယ်ဆိုရင် Assertion ကို ဖယ်လိုက်တဲ့အခါ Program ရဲ့ Behavior ပြောင်းသွားနိုင်ပါတယ်။

ဒီလို Debugging Code ကိုယ်တိုင်က Program ရဲ့ Behavior ကို ပြောင်းစေတဲ့ ပြဿနာကို **Heisenbug** လို့ ခေါ်ပါတယ်။

အဓိကစည်းမျဉ်း

Assertion က Error ကို စစ်ရမယ်၊ Program ရဲ့ State ကို မပြောင်းရဘူး။

24. Exception ကို ဘယ်အချိန်သုံးမလဲ

Error Handling Code တွေ များလာရင် Program ရဲ့ Normal Logic ကို ဖတ်ရခက်လာနိုင်ပါတယ်။

ဥပမာ if/else တွေကို Error တစ်ခုချင်းစီအတွက် ထပ်ခါထပ်ခါ Nested လုပ်ထားရင် Main Logic က Error Handling ထဲမှာ ပျောက်သွားနိုင်ပါတယ်။

Exception ရှိတဲ့ Language တွေမှာ Error Handling ကို သီးခြားနေရာတစ်ခုမှာ စုထားနိုင်တဲ့အတွက် Normal Flow ကို ပို ရှင်းလင်းစေနိုင်ပါတယ်။

What Is Exceptional?

ဘယ်အရာက Exceptional ဖြစ်သလဲ?

စာအုပ်ရဲ့ အဓိကအယူအဆက—

Exception ကို Normal Program Flow အတွက် မသုံးသင့်ပါဘူး။

Exception ဟာ **Unexpected Event** တွေအတွက် ဖြစ်သင့်ပါတယ်။

ဥပမာ File တစ်ခုကို ဖွင့်တဲ့အခါ—

Case 1

Program က အဲဒီ File ရှိရမယ်လို့ မျှော်လင့်ထားတယ်။

ဒါပေမယ့် File ပျောက်နေတယ်။

ဒါဆို—

Exception သုံးသင့်ပါတယ်။

ဘာကြောင့်လဲဆိုတော့ မမျှော်လင့်ထားတဲ့ အရာဖြစ်သွားလို့ပါ။

Case 2

User က ကိုယ်တိုင် File Name ထည့်ပေးတာဖြစ်ပြီး အဲဒီ File ရှိမရှိ မသေချာဘူး။

File မရှိတာက Normal ဖြစ်နိုင်ပါတယ်။

ဒီအခြေအနေမှာ—

Normal Error Return သုံးတာ ပိုသင့်တော်ပါတယ်။

TIP 34

Exceptional Problems အတွက်ပဲ Exception သုံးပါ

Exception ကို Normal Processing အတွက် သုံးရင်—

- Readability ကျနိုင်တယ်
- Maintainability ကျနိုင်တယ်
- Encapsulation ပျက်နိုင်တယ်
- Caller နဲ့ Routine ကြား Coupling တိုးနိုင်တယ်

ဒါကြောင့်—

Exception = Unexpected / Exceptional Event
ဆိုတဲ့ စည်းမျဉ်းကို မှတ်ထားပါ။

25. How to Balance Resources (Resources တွေကို ဘယ်လို စီမံမလဲ)

Programming လုပ်တဲ့အခါ Resource အမျိုးမျိုးကို အသုံးပြုရပါတယ်။

ဥပမာ—

- Memory
- Files
- Transactions
- Threads
- Timers
- Database Connections
- Network Resources

စတာတွေပါ။

Resource အသုံးပြုတဲ့ Pattern က အများအားဖြင့်—

Allocate
↓
Use
↓
Deallocate
ဆိုတဲ့ ပုံစံဖြစ်ပါတယ်။

ပြဿနာက Resource ကို ဘယ်သူက ဖန်တီးတာလဲ၊ ဘယ်သူက ပြန်လွှတ်ရမလဲဆိုတာ မရှင်းရင် Resource Leak တွေ ဖြစ်လာနိုင်ပါတယ်။

TIP 35
Finish What You Start

Resource ကို Allocate လုပ်တဲ့ Routine/Object ကပဲ အဲဒီ Resource ကို Deallocate လုပ်ဖို့ တာဝန်ယူသင့်ပါတယ်။

ဥပမာ—

open file
↓
read file
↓
update data
↓
write data
↓
close file

File ကို ဘယ် Function က Open လုပ်သလဲဆိုတာ မရှင်းဘဲ Global Variable ထဲမှာ သိမ်းထားပြီး အခြား Function က Close လုပ်ရင် Coupling ဖြစ်လာပါတယ်။

စာအုပ်ရဲ့ Example မှာ readCustomer() က File ကို Open လုပ်ပြီး Global cFile ထဲမှာ သိမ်းထားပါတယ်။ writeCustomer() က အဲဒီ Global Variable ကို အသုံးပြုပြီး File ကို Close လုပ်ပါတယ်။ ဒီလိုပုံစံဟာ Routine နှစ်ခုကြား Tight Coupling ဖြစ်စေပါတယ်။

Resource Ownership

Resource တစ်ခုရဲ့ Owner ကို ရှင်းရှင်းလင်းလင်း သတ်မှတ်ပါ။

ဥပမာ—

Object A creates resource
↓
Object A owns resource
↓
Object A releases resource

ဒီလိုဆိုရင် Resource ဘယ်အချိန်မှာ ပြန်လွှတ်ရမလဲဆိုတာ ရှင်းပါတယ်။

Automatic Resource Management

Language ရဲ့ Feature တွေကို အသုံးပြုပြီး Resource Cleanup ကို အလိုအလျောက်လုပ်နိုင်ပါတယ်။

C++ မှာ Object Wrapper တွေက Object ပျက်သွားတဲ့အခါ Resource ကို ပြန်လွှတ်နိုင်ပါတယ်။

Java မှာတော့ finally Clause ကို အသုံးပြုပြီး Resource Cleanup လုပ်နိုင်ပါတယ်။

finally ထဲက Code ဟာ try Block ထဲက Code Run ဖြစ်ခဲ့ရင် Exception ဖြစ်ဖြစ်၊ return ဖြစ်ဖြစ် Execute လုပ်ပေးပါတယ်။

Resource Balance ကို စစ်ဆေးပါ

Resource တွေကို Free လုပ်ပြီးပြီလို့ မယုံပါနဲ့။

Program ထဲမှာ—

“Resource တွေက တကယ်ပဲ ပြန်လွှတ်ပြီးပြီလား?”

ဆိုတာကို စစ်ဆေးနိုင်တဲ့ Mechanism တွေ ထည့်ပါ။

Long-running Program တွေမှာ Loop တစ်ပတ်ပြီးတိုင်း Resource Usage တိုးလာသလားဆိုတာ စစ်ဆေးနိုင်ပါတယ်။

Memory Leak စစ်ဆေးတဲ့ Tools တွေလည်း အသုံးပြုနိုင်ပါတယ်။

Chapter 4 ရဲ့ အဓိကအယူအဆ

Perfect Software မရှိပါဘူး။ ဒါကြောင့် ကိုယ့် Code ကိုတောင် အပြည့်အဝ မယုံဘဲ Assumption တွေကို စစ်ဆေး၊ Error တွေကို စောစောဖော်ထုတ်၊ Resource တွေကို သေချာစီမံပြီး Defensive ဖြစ်အောင် Programming လုပ်ပါ။

Chapter 5 — Bend, or Break (ကွေးနိုင်ရင် မကျိုးဘူး)

ဘဝဟာ တစ်နေရာတည်းမှာ ရပ်မနေပါဘူး။

အဲဒီလိုပဲ ကျွန်တော်တို့ရေးတဲ့ Code တွေလည်း ရပ်တန့်နေမှာ မဟုတ်ပါဘူး။ အမြဲတမ်း မြန်မြန်ဆန်ဆန် ပြောင်းလဲနေတဲ့ ဒီခေတ်မှာ လိုက်လျောညီထွေဖြစ်နိုင်ပြီး Flexible ဖြစ်တဲ့ Code ရေးနိုင်ဖို့ အစွမ်းကုန် ကြိုးစားရပါမယ်။

မဟုတ်ရင် Code ဟာ မကြာခင် ခေတ်နောက်ကျသွားနိုင်သလို ပြင်ဆင်ဖို့ အရမ်းခက်တဲ့ Brittle Code ဖြစ်လာနိုင်ပါတယ်။

နောက်ဆုံးမှာ အနာဂတ်ရဲ့ အရှိန်အဟုန်နောက်မှာ ကျန်ခဲ့နိုင်ပါတယ်။

ဒီ Chapter မှာ မသေချာတဲ့ အနာဂတ်အခြေအနေတွေအတွက် Code ကို Flexible ဖြစ်အောင် ဘယ်လို Design လုပ်မလဲဆိုတာ ဆွေးနွေးထားပါတယ်။ အဓိကအကြောင်းအရာတွေကတော့—

1. Decoupling and the Law of Demeter
2. Metaprogramming
3. Temporal Coupling
4. It’s Just a View
5. Blackboards

တို့ ဖြစ်ပါတယ်။

26. Decoupling and the Law of Demeter

“Good fences make good neighbors.”

Programming မှာ Module တစ်ခုနဲ့တစ်ခု အလွန်အကျွံ သိကျွမ်းပြီး အပြန်အလှန် မှီခိုနေခြင်းဟာ ပြဿနာဖြစ်စေနိုင်ပါတယ်။

ဒါကြောင့် Code ကို **သီးခြား Module (cells)** တွေအဖြစ် ခွဲပြီး Module တစ်ခုနဲ့တစ်ခုကြား Interaction ကို ကန့်သတ်သင့်ပါတယ်။

Module တစ်ခုကို ပြောင်းလဲခြင်း၊ အစားထိုးခြင်း ပြုလုပ်ရတဲ့အခါ တခြား Module တွေက အလွယ်တကူ ဆက်လက်အလုပ်လုပ်နိုင်ရပါမယ်။

Minimize Coupling

Coupling ဆိုတာ Module တစ်ခုက တခြား Module တစ်ခုအပေါ် မှီခိုနေမှုကို ဆိုလိုပါတယ်။

Coupling များလာရင်—

- Module တစ်ခုကို ပြင်တဲ့အခါ တခြား Module တွေပါ ထိခိုက်နိုင်တယ်။
- Testing ပိုခက်လာတယ်။
- Reuse လုပ်ရခက်လာတယ်။
- System တစ်ခုလုံးက Brittle ဖြစ်လာတယ်။

ဒါကြောင့်—

Module တစ်ခုချင်းစီကို ကိုယ်ပိုင်တာဝန်ရှိတဲ့ အစိတ်အပိုင်းတစ်ခုလိုထားပြီး အခြား Module တွေနဲ့ ဆက်သွယ်မှုကို လျှော့ချပါ။

Law of Demeter

Law of Demeter ရဲ့ အဓိကအယူအဆက—

"ကိုယ့်နဲ့ နီးစပ်တဲ့ Object တွေနဲ့ပဲ ဆက်သွယ်ပါ။"

Object တစ်ခုက တခြား Object တစ်ခုရဲ့ Object ကို ထပ်ပြီးသွားရှာပြီး အဲဒီထဲက Object နဲ့ ထပ်ဆက်သွယ်တာမျိုးကို ရှောင်သင့်ပါတယ်။

ဥပမာ—

```
customer.getOrder().getPayment().getAccount().getBalance()
```

ဒီလို Chain အရှည်ကြီးနဲ့ Object တွေကို တစ်ဆင့်ပြီးတစ်ဆင့် လိုက်သွားရင် Objects တွေကြား Coupling အရမ်းများလာနိုင်ပါတယ်။

အစား—

```
customer.getPaymentBalance()
```

လိုမျိုး Customer ကို သူ့ရဲ့လိုအပ်တဲ့ Operation ကို ကိုယ်တိုင် ပေးထားနိုင်ပါတယ်။

ဒီလိုဆိုရင် အတွင်းမှာ ဘယ် Object တွေရှိတယ်ဆိုတာကို Caller က သိစရာမလိုတော့ပါဘူး။

Shy Code (ရှက်တတ်တဲ့ Code)

စာအုပ်က **"Shy Code"** ဆိုတဲ့ အယူအဆကို သုံးပါတယ်။

Shy Code ဆိုတာ—

- ကိုယ့်အတွင်းပိုင်းကို မလိုအပ်ဘဲ မဖော်ပြခြင်း
- အခြား Object တွေကို အလွန်အကျွံ မဝင်ရောက်ခြင်း
- လိုအပ်သလောက်ပဲ ဆက်သွယ်ခြင်း

ဖြစ်ပါတယ်။

TIP 36 - Module တွေကြား Coupling ကို လျှော့ပါ၊ Shy Code ရေးပြီး Law of Demeter ကို အသုံးပြုပါ။

27. Metaprogramming

“No amount of genius can overcome a preoccupation with detail.”
အသေးစိတ်အချက်အလက်တွေဟာ Code ကို ရှုပ်ထွေးစေနိုင်ပါတယ်။

အထူးသဖြင့် အဲဒီ Details တွေ မကြာခဏပြောင်းလဲနေရင် ပိုဆိုးပါတယ်။

Business Logic ပြောင်းတဲ့အခါ၊ ဥပဒေပြောင်းတဲ့အခါ၊ Management ရဲ့ လိုအပ်ချက်ပြောင်းတဲ့အခါတိုင်း Code ထဲကို ဝင်ပြင်ရမယ်ဆိုရင် System ပျက်သွားနိုင်ပြီး Bug အသစ်တွေ ဖြစ်လာနိုင်ပါတယ်။

ဒါကြောင့် စာအုပ်က—

“Details တွေကို Code ထဲကနေ ထုတ်ပစ်ပါ”

လို့ အကြံပြုပါတယ်။

Dynamic Configuration ပြုလုပ်ခြင်း

System ကို Configuration ပြောင်းလဲနိုင်အောင် တည်ဆောက်သင့်ပါတယ်။

Configuration ဆိုတာ Screen Color နဲ့ Prompt Text လို အရာလေးတွေပဲ မဟုတ်ပါဘူး။

ဥပမာ—

- Algorithm ရွေးချယ်မှု
- Database Product
- Middleware Technology
- User Interface Style

စတာတွေကိုပါ Configuration အဖြစ် သတ်မှတ်နိုင်ပါတယ်။

TIP 37
Configure, Don’t Integrate

Application ရဲ့ Configuration Options တွေကို ဖော်ပြဖို့ **Metadata** ကို အသုံးပြုပါ။

Metadata ဆိုတာ ရိုးရိုးပြောရရင်—

“Data အကြောင်းကို ဖော်ပြတဲ့ Data”

ဖြစ်ပါတယ်။

ဥပမာ Database Schema က Database ထဲက Field တွေရဲ့—

- Name
- Storage Length
- Attribute

စတာတွေကို ဖော်ပြပေးပါတယ်။

Put Details in Metadata

အဓိကစိတ်ကူးက—

General Case ကို Code ထဲမှာ ရေးပါ။ Specific Details တွေကို Code အပြင်ဘက်မှာ ထားပါ။

ဒီလိုလုပ်ထားရင် Requirement ပြောင်းတဲ့အခါ Program Code ကို အမြဲပြန် Compile လုပ်ပြီး ပြင်ဆင်စရာမလိုဘဲ Configuration/Metadata ကို ပြောင်းနိုင်ပါတယ်။

TIP 38

Code ထဲမှာ Abstraction ထားပြီး Details ကို Metadata ထဲမှာထားပါ

Program ကို General Case အတွက် ရေးပြီး Specific Information တွေကို Compiled Code အပြင်ဘက်မှာ ထားပါ။

28. Temporal Coupling (အချိန်အပေါ် မှီခိုမှု)

Temporal Coupling ဆိုတာ System ထဲက အရာတွေဟာ အချိန်အစီအစဉ်တစ်ခုအပေါ် မှီခိုနေရခြင်း ဖြစ်ပါတယ်။

ဥပမာ—

A ကို အရင်လုပ်ရမယ်

ပြီးမှ B ကိုလုပ်ရမယ်

ပြီးမှ C ကိုလုပ်ရမယ်

ဆိုရင် B က A ပြီးမှသာ လုပ်လို့ရပြီး C ကလည်း B ပြီးမှသာ လုပ်လို့ရပါတယ်။

ဒီလို Time Dependency များလေလေ System က Flexible ဖြစ်ဖို့ ခက်လေလေပါပဲ။

Concurrency

တစ်ချို့ Task တွေဟာ တစ်ခုနဲ့တစ်ခု မမှီခိုဘူးဆိုရင် Parallel လုပ်နိုင်ပါတယ်။

ဥပမာ Piña Colada လုပ်တဲ့ Workflow မှာ—

- Blender ဖွင့်ခြင်း
- Rum တိုင်းခြင်း
- Glass ယူခြင်း
- Umbrella ယူခြင်း

စတဲ့ အလုပ်တွေကို တစ်ခုပြီးမှတစ်ခု လုပ်စရာမလိုဘဲ တစ်ပြိုင်နက် လုပ်နိုင်ပါတယ်။

ဒီလို Workflow ကို Analyze လုပ်ခြင်းအားဖြင့် Concurrency တိုးပြီး အလုပ်လုပ်ရတဲ့အချိန်ကို လျှော့နိုင်ပါတယ်။

TIP 39

Workflow ကို Analyze လုပ်ပြီး Concurrency တိုးပါ

User ရဲ့ Workflow ထဲမှာ Parallel လုပ်လို့ရတဲ့ အလုပ်တွေကို ရှာဖွေပါ။

Design Using Services

System ကို Independent Services တွေအဖြစ် Design လုပ်နိုင်ပါတယ်။

Service တစ်ခုချင်းစီဟာ—

- Independent ဖြစ်တယ်
- Concurrent ဖြစ်နိုင်တယ်
- ရှင်းလင်းတဲ့ Interface ရှိတယ်

ဆိုရင် System တစ်ခုလုံးကို ပိုပြီး Flexible ဖြစ်အောင် ပြုလုပ်နိုင်ပါတယ်။

TIP 40
Services အဖြစ် Design လုပ်ပါ

Well-defined၊ Consistent Interface တွေနောက်က Independent, Concurrent Objects/Services တွေအဖြစ် System ကို Design လုပ်ပါ။

Always Design for Concurrency

Concurrency ကို နောက်မှ ထည့်ဖို့ကြိုးစားတာထက် အစကတည်းက ထည့်စဉ်းစားတာ ပိုလွယ်ပါတယ်။

Concurrency အတွက် Design လုပ်ထားရင်—

- Scalability ပိုကောင်းလာနိုင်တယ်
- Performance Requirement တွေကို ဖြည့်ဆည်းရလွယ်တယ်
- Interface တွေ ပိုသန့်ရှင်းလာတယ်
- Time Dependency တွေ လျော့လာတယ်

စာအုပ်က Non-concurrent Application ထဲကို နောက်မှ Concurrency ထည့်ဖို့ ကြိုးစားတာဟာ ပိုခက်တယ်လို့ ရှင်းပြထားပါတယ်။

TIP 41
Concurrency အတွက် အမြဲ Design လုပ်ပါ

29. It's Just a View

Program တစ်ခုကို အစိတ်အပိုင်းကြီးတစ်ခုတည်းအဖြစ် မရေးဘဲ Module တွေအဖြစ် ခွဲရေးသင့်ပါတယ်။

Module တစ်ခုချင်းစီမှာ ကိုယ်ပိုင် Responsibility ရှိသင့်ပါတယ်။

ဒါပေမယ့် Module တွေ ခွဲပြီးတဲ့အခါ မေးခွန်းအသစ်တစ်ခု ပေါ်လာပါတယ်—

Runtime မှာ ဒီ Module တွေ တစ်ခုနဲ့တစ်ခု ဘယ်လိုဆက်သွယ်မလဲ?

Module တစ်ခုက တခြား Module တစ်ခုအကြောင်း အလွန်အကျွံ သိနေမယ်ဆိုရင် Coupling တိုးလာပါမယ်။

ဒါကြောင့် Module တစ်ခုက သူ့အတွက် လိုအပ်တဲ့ Information ကိုပဲ သိနိုင်အောင် Design လုပ်သင့်ပါတယ်။

Events

Event ဆိုတာ—

"စိတ်ဝင်စားစရာတစ်ခုခု ဖြစ်သွားပြီ"

လို့ ပြောတဲ့ Message တစ်ခုလို ဖြစ်ပါတယ်။

Object တစ်ခုမှာ State ပြောင်းလဲသွားရင် အဲဒီ Event ကို စိတ်ဝင်စားတဲ့ အခြား Object တွေကို အသိပေးနိုင်ပါတယ်။

အရေးကြီးတာက Event ပို့တဲ့ Object က Event လက်ခံတဲ့ Object ကို တိုက်ရိုက် သိထားစရာ မလိုပါဘူး။

ဒါကြောင့် Coupling လျော့သွားပါတယ်။

Publish/Subscribe

Object တွေကို Event တစ်ခုချင်းစီအတွက် **Subscribe** လုပ်ခွင့်ပေးနိုင်ပါတယ်။

ဥပမာ—

Publisher
↓
Event
↓
Subscriber 1
Subscriber 2
Subscriber 3

Publisher က Event တစ်ခုထုတ်တဲ့အခါ အဲဒီ Event ကို Subscribe လုပ်ထားတဲ့ Object တွေကို Notify လုပ်ပေးပါတယ်။

ဒီနည်းနဲ့ Object တွေဟာ တစ်ခုနဲ့တစ်ခု အလွန်အကျွံ မှီခိုစရာမလိုတော့ပါဘူး။

TIP 42

View တွေကို Model တွေနဲ့ ခွဲထားပါ

Application ကို **Model** နဲ့ **View** အဖြစ် ခွဲခြား Design လုပ်ခြင်းအားဖြင့် ကုန်ကျစရိတ်အများကြီး မလိုဘဲ Flexibility ရနိုင်ပါတယ်။

30. Blackboards

Blackboard ဆိုတာ လူအများကြီး ဒါမှမဟုတ် Agent အများကြီးက Information တွေကို တစ်နေရာတည်းမှာ ထားပြီး အဲဒီ Information တွေကို အခြေခံကာ ကိုယ့်တာဝန်ကိုယ် လုပ်ဆောင်နိုင်တဲ့ နည်းလမ်းတစ်ခု ဖြစ်ပါတယ်။

စာအုပ်က Detective တွေက Murder Investigation လုပ်ရာမှာ Blackboard တစ်ခုကို အသုံးပြုတဲ့ ဥပမာနဲ့ စတင်ရှင်းပြထားပါတယ်။

ဥပမာ Blackboard ပေါ်မှာ—

H. DUMPTY
MALE, EGG

ACCIDENT OR MURDER?

လို Information တစ်ခုကို ရေးထားနိုင်ပါတယ်။

Detective တစ်ယောက်က အချက်အလက်တစ်ခု ထည့်မယ်။

နောက် Detective တစ်ယောက်က အဲဒီ Information ကို ကြည့်ပြီး သူ့အပိုင်းကို ဆက်လက်စုံစမ်းမယ်။

ဒီလိုနဲ့ Information တွေ တဖြည်းဖြည်း စုစည်းလာပါတယ်။

Blackboard in Software

Software System မှာလည်း အလားတူပါပဲ။

Agent တစ်ခုက Information တစ်ခုကို Blackboard ပေါ်မှာ ထည့်နိုင်ပါတယ်။

အခြား Agent တွေက အဲဒီ Information ကို ဖတ်ပြီး သူတို့လုပ်ဆောင်ရမယ့် အလုပ်ကို ဆုံးဖြတ်နိုင်ပါတယ်။

ဒီနည်းလမ်းရဲ့ အားသာချက်က Agent တစ်ခုက အခြား Agent တစ်ခုကို တိုက်ရိုက် သိထားစရာ မလိုခြင်းပါ။

ဒါကြောင့် System ထဲက Participant တွေကို Independent ဖြစ်အောင် ထားနိုင်ပါတယ်။

Workflow Coordination

Blackboard ဟာ အထူးသဖြင့် Data တွေဟာ—

- အချိန်မတူဘဲ ရောက်လာနိုင်တဲ့အခါ
- Source မတူတဲ့အခါ
- Agent မတူတဲ့အခါ
- အစီအစဉ်တိတိကျကျ မသတ်မှတ်နိုင်တဲ့အခါ

အသုံးဝင်ပါတယ်။

ဥပမာ Loan Application တစ်ခုမှာ—

- Name
- Address
- Credit Check
- Ownership Information
- Insurance Information

စတဲ့ Data တွေဟာ အချိန်မတူဘဲ ရောက်လာနိုင်ပါတယ်။

Data တစ်ခု ရောက်လာတိုင်း သက်ဆိုင်ရာ Rule တွေက အဲဒီ Information ကို အသုံးပြုပြီး နောက်ထပ် Action တွေကို Trigger လုပ်နိုင်ပါတယ်။

ဒါကြောင့် Data ရောက်လာတဲ့ Order ကို အတိအကျ မှီခိုစရာမလိုတော့ပါဘူး။

TIP 43

Workflow ကို Coordinate လုပ်ဖို့ Blackboards အသုံးပြုပါ

Blackboard ကို အသုံးပြုပြီး မတူညီတဲ့ Facts နဲ့ Agents တွေကို Coordinate လုပ်နိုင်ပါတယ်။

တစ်ချိန်တည်းမှာ Participant တစ်ခုချင်းစီရဲ့ Independence နဲ့ Isolation ကိုလည်း ထိန်းထားနိုင်ပါတယ်။

Chapter 5 ရဲ့ အဓိကအယူအဆ

Software ဟာ အနာဂတ်မှာ ပြောင်းလဲရမယ့်အရာဖြစ်တဲ့အတွက် အစကတည်းက Flexible ဖြစ်အောင် Design လုပ်ပါ။

“Bend, or Break” — ကွေးနိုင်အောင်ရေးပါ၊ မဟုတ်ရင် ပြောင်းလဲမှုတွေကြားမှာ Code က ကျိုးပဲ့သွားနိုင်ပါတယ်။

Chapter 6 — While You Are Coding

လူအများစုက Project တစ်ခု Coding အဆင့်ရောက်ပြီဆိုရင် Design ကို Executable Statements တွေအဖြစ် ပြောင်းရေးရုံပဲဖြစ်ပြီး အလုပ်အများစုက Mechanical ဖြစ်သွားပြီလို့ ယူဆကြပါတယ်။

စာအုပ်ကတော့ ဒီအမြင်ဟာ Software တွေကို—

- မလှပခြင်း
- ထိရောက်မှုမရှိခြင်း
- Structure မကောင်းခြင်း
- Maintenance လုပ်ရခက်ခြင်း
- အမှားများခြင်း

တို့ ဖြစ်စေတဲ့ အဓိကအကြောင်းရင်းတစ်ခုလို့ ဆိုပါတယ်။

Coding ဟာ Mechanical အလုပ်မဟုတ်ပါဘူး။

Code ရေးနေစဉ် အချိန်တိုင်းမှာ ဆုံးဖြတ်ချက်တွေ ချရပါတယ်။ အဲဒီဆုံးဖြတ်ချက်တွေကို သေချာစဉ်းစားပြီး Judgment ကောင်းကောင်းနဲ့ ချမှသာ ရေရှည်တည်တံ့ပြီး မှန်ကန်၊ အသုံးဝင်တဲ့ Program တစ်ခု ဖြစ်လာနိုင်ပါတယ်။

ဒီအခန်းမှာ—

1. Programming by Coincidence
2. Algorithm Speed
3. Refactoring
4. Code That's Easy to Test
5. Evil Wizards

ဆိုတဲ့ Section ၅ ခုကို ဆွေးနွေးထားပါတယ်။

31. Programming by Coincidence (တိုက်ဆိုင်မှုကို အားကိုးပြီး Programming မလုပ်ပါနဲ့)

Developer တွေဟာ တစ်ခါတလေ Code တစ်ခု အလုပ်လုပ်နေတာကို မြင်ပြီး “အဲဒါဆို မှန်ပြီ” လို့ ယူဆတတ်ကြပါတယ်။

ဒါပေမယ့် Code က အလုပ်လုပ်နေခြင်းဟာ ကိုယ်စဉ်းစားပြီး သေချာတည်ဆောက်ထားလို့ ဖြစ်တာလား၊ ဒါမှမဟုတ် **တိုက်ဆိုင်မှုကြောင့် အလုပ်လုပ်နေတာလား** ဆိုတာ ခွဲခြားသိဖို့လိုပါတယ်။

စာအုပ်က ဒီအခြေအနေကို မိုင်းကွင်းထဲမှာ လမ်းလျှောက်နေတဲ့ စစ်သားနဲ့ နှိုင်းယှဉ်ထားပါတယ်။

စစ်သားက မြေကို Bayonet နဲ့ စမ်းကြည့်တယ်။

ဘာမှ မပေါက်ကွဲဘူး။

ဒါကြောင့် အဲဒီနေရာမှာ မိုင်းမရှိဘူးလို့ ယူဆတယ်။

နောက်ထပ်လည်း စမ်းတယ်။ ဘာမှမဖြစ်ဘူး။

နောက်ဆုံးမှာ မိုင်းကွင်းမဟုတ်ဘူးလို့ ယုံကြည်ပြီး ရှေ့ကို လွတ်လွတ်လပ်လပ် လျှောက်သွားတဲ့အချိန်မှာ ပေါက်ကွဲသွားပါတယ်။

အစောပိုင်း စမ်းသပ်မှုတွေက ဘာမှမတွေ့တာဟာ “လုံခြုံတယ်” ဆိုတာကို သက်သေပြတာမဟုတ်ပါဘူး။

ကံကောင်းခဲ့တာသာ ဖြစ်နိုင်ပါတယ်။

Developer တွေလည်း အလားတူပဲ။

နေ့စဉ် Code ရေးတဲ့အခါ Trap အများကြီး ရှိပါတယ်။

ဒါကြောင့်—

Luck နဲ့ Accidental Success ကို မအားကိုးဘဲ Deliberately Programming လုပ်သင့်ပါတယ်။

How to Program by Coincidence (တိုက်ဆိုင်မှုကို အားကိုးပြီး Programming လုပ်ပုံ)

ဥပမာ Fred ဆိုတဲ့ Programmer တစ်ယောက်ကို Programming Assignment တစ်ခု ပေးတယ်။

Fred က Code နည်းနည်းရေးတယ်။

Run လုပ်ကြည့်တယ်။

အလုပ်လုပ်တယ်။

နောက်ထပ် Code ရေးတယ်။

ပြန် Run လုပ်တယ်။

အလုပ်လုပ်ပြန်တယ်။

ဒီလိုနဲ့ ရက်သတ္တပတ်အနည်းငယ် Coding လုပ်သွားတယ်။

တစ်နေ့မှာ Program က ရုတ်တရက် အလုပ်မလုပ်တော့ဘူး။

Fred က နာရီပေါင်းများစွာ Debugging လုပ်ပေမယ့် ဘာကြောင့် အလုပ်မလုပ်တော့တာလဲ မသိတော့ဘူး။

အကြောင်းရင်းက Fredဟာ Code ဘာကြောင့် အလုပ်လုပ်သလဲဆိုတာကို နားလည်အောင် မလုပ်ဘဲ—

"အလုပ်လုပ်နေတယ်ဆိုရင် ရပြီ"

ဆိုတဲ့အယူအဆနဲ့ ဆက်ရေးခဲ့လို့ပါ။

Programming by Coincidence ဖြစ်စေတဲ့ အကြောင်းများ

- Compiler က ဒီလိုအလုပ်လုပ်နေတာကို အားကိုးခြင်း
- Operating System ရဲ့ Behavior တစ်ခုကို အမြဲတမ်းတူမယ်လို့ ယူဆခြင်း
- Library တစ်ခုရဲ့ မသတ်မှတ်ထားတဲ့ Behavior ကို အားကိုးခြင်း
- Test Data တစ်ခုတည်းနဲ့ မှန်တယ်လို့ ယူဆခြင်း
- Performance က တိုက်ဆိုင်စွာ ကောင်းနေတာကို အမြဲတမ်းကောင်းမယ်လို့ ယူဆခြင်း
- Environment တစ်ခုမှာ အလုပ်လုပ်တာကို Environment အားလုံးမှာ အလုပ်လုပ်မယ်လို့ ယူဆခြင်း

ဒီလို Assumptions တွေက နောက်ပိုင်းမှာ Bug တွေ ဖြစ်စေနိုင်ပါတယ်။

Deliberate Programming (ရည်ရွယ်ချက်ရှိရှိ Programming လုပ်ခြင်း)

Code ရေးတဲ့အခါ—

“ဒါက ဘာကြောင့် အလုပ်လုပ်တာလဲ?”

ဆိုတဲ့မေးခွန်းကို ကိုယ့်ကိုယ်ကို မေးပါ။

Code ရဲ့ Behavior ကို နားလည်ထားပါ။

ကိုယ်မသိတဲ့အရာကို Assumption မလုပ်ပါနဲ့။

Documentation ကို ဖတ်ပါ။

Experiment လုပ်ပါ။

Boundary Conditions တွေကို စမ်းပါ။

အမှန်တကယ်ဖြစ်နေတဲ့ Environment ထဲမှာ ကိုယ့်ရဲ့ Assumptions တွေကို Verify လုပ်ပါ။

TIP 46

Programming ကို တိုက်ဆိုင်မှုနဲ့ မလုပ်ပါနဲ့

Code အလုပ်လုပ်နေတယ်ဆိုတာနဲ့ လုံလောက်ပြီလို့ မယူဆပါနဲ့။

ဘာကြောင့် အလုပ်လုပ်တယ်ဆိုတာ သိထားပါ။

32. Algorithm Speed

Code အများစုဟာ မြန်မြန်ဆန်ဆန် Run နိုင်ပါတယ်။

ဒါပေမယ့် တချို့ Algorithm တွေက Input Data အရေအတွက် တိုးလာတာနဲ့ Performance ကို အလွန်အမင်း ထိခိုက်စေနိုင်ပါတယ်။

ဒီအတွက် Algorithm ရဲ့ Speed ကို ကြိုတင်ခန့်မှန်းနိုင်ဖို့ လိုပါတယ်။

စာအုပ်က Algorithm တစ်ခုရဲ့ Speed ကို လေ့လာရာမှာ **Big-O Notation** ကို အသုံးပြုပါတယ်။

အဓိကက—

Input Size တိုးလာတဲ့အခါ Algorithm ရဲ့ အလုပ်လုပ်ရတဲ့ပမာဏ ဘယ်လောက်မြန်မြန် တိုးလာသလဲ။

ဆိုတာကို သိဖို့ပါ။

Constant Time — O(1)

Input Size ဘယ်လောက်ပဲကြီးကြီး Operation အရေအတွက်က တူနေတယ်ဆိုရင်—

O(1) ဖြစ်ပါတယ်။

ဥပမာ Array ထဲက Index တစ်ခုကို တိုက်ရိုက် Access လုပ်ခြင်း။

```
item = array[10]
```

Array ထဲမှာ Item 100 ရှိရှိ၊ 1,000,000 ရှိရှိ Index 10 ကို Access လုပ်တာက အကြမ်းဖျင်းအားဖြင့် တူပါတယ်။

Linear Time — O(n)

Input Size တိုးလာတာနဲ့ အလုပ်လုပ်ရတဲ့ပမာဏလည်း အချိုးကျတိုးလာရင်—

O(n) ဖြစ်ပါတယ်။

ဥပမာ List ထဲက Item တစ်ခုကို အစကနေ အဆုံးထိ ရှာခြင်း။

Item အရေအတွက် 10 ဆတိုးရင် အများဆုံး Search လုပ်ရမယ့် အရေအတွက်လည်း 10 ဆတိုးနိုင်ပါတယ်။

Quadratic Time — O(n^2)

Loop တစ်ခုထဲမှာ Loop တစ်ခု ထပ်ရှိတဲ့ Algorithm တွေမှာ—

O(n^2) ဖြစ်နိုင်ပါတယ်။

Input Size နှစ်ဆတိုးသွားရင် အလုပ်လုပ်ရတဲ့ပမာဏက ခန့်မှန်းအားဖြင့် လေးဆတိုးနိုင်ပါတယ်။

ဒါကြောင့် Input ကြီးလာတဲ့အခါ O(n^2) Algorithm တွေဟာ အလွန်နှေးလာနိုင်ပါတယ်။

Logarithmic Time — O(log n)

Binary Search လို Algorithm တွေမှာ Input ကို တစ်ကြိမ်စီ အပိုင်းပိုင်းခွဲပြီး ရှာနိုင်တဲ့အတွက်—

O(log n)
ဖြစ်နိုင်ပါတယ်။

Input Size အလွန်ကြီးလာတာနဲ့တောင် Operation အရေအတွက်က အလွန်မြန်မြန် မတိုးပါဘူး။

Algorithm Speed ကို ခန့်မှန်းပါ

Algorithm ရေးပြီးမှ Performance Problem ဖြစ်လာတာကို စောင့်မနေပါနဲ့။

Coding မစခင်ကတည်းက—

- Input Size
- Algorithm Complexity
- Memory Usage
- Disk Access
- Network Access

စတာတွေကို စဉ်းစားပါ။

အချိန်ကုန်တဲ့နေရာဟာ CPU မဟုတ်ဘဲ Disk၊ Network ဒါမှမဟုတ် Memory ဖြစ်နေနိုင်ပါတယ်။

TIP 47

Algorithm Speed ကို Estimate လုပ်ပါ

Algorithm တစ်ခုကို အသုံးမပြုခင် Input Size ကြီးလာရင် ဘယ်လိုအခြေအနေဖြစ်မလဲဆိုတာ စဉ်းစားပါ။

“ဒီ Code က အခုမြန်တယ်” ဆိုတာနဲ့ မလုံလောက်ပါဘူး။

“Data ၁၀ ဆ၊ ၁၀၀ ဆ တိုးလာရင်ရော?”

ဆိုတာကို မေးရပါမယ်။

စာအုပ်ရဲ့ Exercise တွေမှာ Sort Algorithm တွေကို Machine အမျိုးမျိုးမှာ Run ပြီး Compiler Optimization၊ Memory နဲ့ Background Processes တွေက Result ကို ဘယ်လိုသက်ရောက်သလဲဆိုတာ စမ်းသပ်ဖို့လည်း တောင်းဆိုထားပါတယ်။

33. Refactoring (Code ကို ပြန်လည်ပြုပြင်ခြင်း)

Program တစ်ခုဟာ တစ်ကြိမ်ရေးပြီး ပြီးဆုံးသွားတဲ့အရာ မဟုတ်ပါဘူး။

Project တိုးတက်လာတာနဲ့အမျှ—

- Requirements ပြောင်းနိုင်တယ်
- Design ပြောင်းနိုင်တယ်
- Business Rules ပြောင်းနိုင်တယ်
- New Features ထပ်လာနိုင်တယ်
- အရင်ဆုံးဖြတ်ချက်တွေ မှားနေတယ်ဆိုတာ သိလာနိုင်တယ်

ဒါကြောင့် Code ကို ပြန်လည်စဉ်းစားပြီး ပြုပြင်ရပါမယ်။

စာအုပ်က Software Development ကို Building Construction ထက် **Gardening** နဲ့ ပိုတူတယ်လို့ ပြောပါတယ်။

Garden တစ်ခုကို တစ်ကြိမ်စိုက်ပြီး ပြီးသွားတာမဟုတ်ပါဘူး။

အပင်တွေ ကြီးလာရင်—

- ဖြတ်တောက်ရတယ်
- နေရာပြောင်းရတယ်
- အားနည်းတဲ့အပင်တွေ ဖယ်ရတယ်
- မြေကို ပြုပြင်ရတယ်
- အမြဲစောင့်ကြည့်ရတယ်

Software Code လည်း အတူတူပါပဲ။

Code ကို အမြဲတမ်း ပြုစုပျိုးထောင်ရပါတယ်။

What Is Refactoring?

Refactoring ဆိုတာ Program ရဲ့ **External Behavior ကို မပြောင်းဘဲ Internal Structure ကို တိုးတက်အောင် ပြင်ဆင်ခြင်း**ဖြစ်ပါတယ်။

ဥပမာ—

- Duplicate Code ဖယ်ခြင်း
- Function ကြီးကို Function သေးများအဖြစ် ခွဲခြင်း
- Variable Name ပြောင်းခြင်း
- Class Structure ပြန်စီခြင်း
- Coupling လျှော့ခြင်း
- Cohesion မြှင့်ခြင်း
- ရှုပ်ထွေးတဲ့ Code ကို ရိုးရှင်းအောင်လုပ်ခြင်း

စတာတွေ ဖြစ်ပါတယ်။

When Should You Refactor?

ဘယ်အချိန်မှာ Refactor လုပ်မလဲ

Refactoring ကို Project အဆုံးထိ မစောင့်ပါနဲ့။

Code ရေးနေစဉ်—

“ဒီ Code ကို ဒီထက်ကောင်းအောင် ရေးနိုင်မလား?”

လို့ စဉ်းစားပါ။

Code တစ်ခုကို နားလည်ဖို့ ခက်လာရင် Refactor လုပ်ပါ။

Duplicate တွေ ပေါ်လာရင် Refactor လုပ်ပါ။

Requirement ပြောင်းလဲတဲ့အခါ အရင် Design ကို ပြန်ကြည့်ပါ။

TIP 48

Refactor လုပ်ဖို့ အချိန်မစောင့်ပါနဲ့

Refactoring ကို သီးခြား Project တစ်ခုလို မမြင်ပါနဲ့။

နေ့စဉ် Coding Process ထဲမှာ ထည့်သွင်းပါ။

The Problems of Refactoring

Refactoring လုပ်တဲ့အခါ Risk တွေရှိပါတယ်။

Code ကို ပြင်လိုက်တာနဲ့ မူလအလုပ်လုပ်နေတဲ့ Behavior ပျက်သွားနိုင်ပါတယ်။

ဒါကြောင့် Refactoring မလုပ်ခင်နဲ့ လုပ်ပြီးတဲ့အခါ **Tests** ရှိဖို့ အရေးကြီးပါတယ်။

Automated Tests ရှိရင် Refactoring လုပ်တဲ့အခါ ပိုယုံကြည်စိတ်ချရပါတယ်။

စာအုပ်က Refactoring နဲ့ Testing ကို တိုက်ရိုက်ဆက်စပ်ထားပြီး Refactoring လုပ်တဲ့အခါ Existing Tests တွေက Safety Net အဖြစ် အရေးကြီးတယ်လို့ ပြထားပါတယ်။

The First Step

Refactoring လုပ်မယ်ဆိုရင်—

1. လက်ရှိ Code ရဲ့ Behavior ကို သိပါ။
2. Test တွေ Run ပါ။
3. Structure ကို ပြင်ပါ။
4. Test တွေ ပြန် Run ပါ။
5. Behavior မပြောင်းကြောင်း Verify လုပ်ပါ။

ဒီလိုလုပ်ခြင်းအားဖြင့် Code ရဲ့ Structure ကို တဖြည်းဖြည်း တိုးတက်အောင်လုပ်နိုင်ပါတယ်။

34. Code That’s Easy to Test (Test လုပ်ရလွယ်တဲ့ Code ရေးပါ)

Code ရေးနေတဲ့အချိန်မှာ တစ်နေ့နေ့မှာ အဲဒီ Code ကို Test လုပ်ရမယ်ဆိုတာကို အမြဲစဉ်းစားထားသင့်ပါတယ်။

Test လုပ်ရလွယ်တဲ့ Code ဖြစ်လေလေ တကယ် Test လုပ်ဖြစ်ဖို့ အလားအလာများလေလေ ဖြစ်ပါတယ်။

စာအုပ်က ဒီအခန်းရဲ့ အဓိကအယူအဆကို—

Make code easy to test.
လို့ ရှင်းပြထားပါတယ်။

Testability

Test လုပ်ရလွယ်တဲ့ Code မှာ—

- Module တွေ သီးခြားဖြစ်တယ်
- Dependencies နည်းတယ်
- Input/Output ရှင်းတယ်
- Side Effects နည်းတယ်
- Global State ကို မမှီခိုဘူး
- Component တစ်ခုချင်းစီကို သီးခြားစမ်းနိုင်တယ်

Code တစ်ပိုင်းကို Test လုပ်ဖို့ System တစ်ခုလုံးကို Start လုပ်ရမယ်ဆိုရင် Testability မကောင်းပါဘူး။

Built-In Test

Program ထဲမှာ ကိုယ့်ကိုယ်ကို စစ်ဆေးနိုင်တဲ့ Test Mechanism တွေ ထည့်ထားနိုင်ပါတယ်။

ဥပမာ—

- Assertions
- Diagnostic Code
- Self-tests
- Internal Consistency Checks

စတာတွေ ဖြစ်ပါတယ်။

Program က မိမိရဲ့ State ကို မိမိစစ်ဆေးနိုင်ရင် Error တွေကို စောစောတွေ့နိုင်ပါတယ်။

Unit Testing

Unit တစ်ခုချင်းစီကို သီးခြား Test လုပ်နိုင်အောင် Code Structure ကို ပြုလုပ်ပါ။

Unit Test တစ်ခုက—

Input → Function → Expected Output
လိုမျိုး ရိုးရှင်းသင့်ပါတယ်။

ဒီလိုဖြစ်ရင် Test တွေကို Automatically Run လုပ်ဖို့ လွယ်ကူပါတယ်။

Testing the Boundary

Happy Path တစ်ခုတည်းကို မစမ်းပါနဲ့။

Boundary Conditions တွေကို စမ်းပါ။

ဥပမာ—

- Empty Input
- Zero
- Negative Number
- Maximum Value
- Minimum Value
- Very Large Data
- Invalid Data

စတာတွေ ဖြစ်ပါတယ်။

TIP 49

Test လုပ်ရလွယ်တဲ့ Code ကို အစကတည်းက ရေးပါ

Testing ကို နောက်ဆုံးမှာ ထည့်ဖို့ မကြိုးစားပါနဲ့။

Code Design လုပ်တဲ့အချိန်ကတည်းက Testability ကို ထည့်သွင်းစဉ်းစားပါ။

35. Evil Wizards (အန္တရာယ်ရှိနိုင်တဲ့ Wizard များ)

Programming Tools တွေက အလွန်အစွမ်းထက်လာပါတယ်။

တချို့ Tool တွေက Programmer အစား Code အများကြီးကို Automatically Generate လုပ်ပေးနိုင်ပါတယ်။

ဒါဟာ အချိန်အများကြီး သက်သာစေနိုင်ပါတယ်။

ဒါပေမယ့် အန္တရာယ်လည်း ရှိပါတယ်။

စာအုပ်က—

Tool တစ်ခုက ကိုယ့်အတွက် Code အများကြီးရေးပေးနိုင်တယ်ဆိုတာနဲ့ အဲဒီ Tool ကို နားမလည်ဘဲ အသုံးမပြုသင့်ပါဘူး။

လို့ သတိပေးပါတယ်။

Magic ကို မယုံပါနဲ့

Wizard တစ်ခုကို အသုံးပြုပြီး—

Button တစ်ချက်နှိပ်လိုက်တာနဲ့ Code ထောင်ပေါင်းများစွာ ထွက်လာတယ်။

ဆိုရင် အဲဒီ Code ဘာလုပ်နေတယ်ဆိုတာ သိဖို့လိုပါတယ်။

မသိဘဲ အသုံးပြုရင်—

- Generated Code က ဘယ်လိုအလုပ်လုပ်လဲ မသိတော့ခြင်း
- Debugging ခက်ခြင်း
- Performance Problem မသိနိုင်ခြင်း
- Unnecessary Code အများကြီး ထွက်လာခြင်း
- Tool ပြောင်းရင် Project ကို ထိခိုက်ခြင်း
- Tool ရဲ့ Limitations မသိခြင်း

စတဲ့ ပြဿနာတွေ ဖြစ်နိုင်ပါတယ်။

Understand the Tools

Code Generator တစ်ခု အသုံးပြုမယ်ဆိုရင်—

1. ဘာ Code ထုတ်ပေးလဲ သိပါ။
2. Generated Code ကို ဖတ်ပါ။
3. Generated Code ကို ပြင်နိုင်ဖို့ သိပါ။
4. Tool မရှိရင် ဘာဖြစ်မလဲ သိပါ။
5. Tool ရဲ့ Limitations ကို သိပါ။

Tool က Programmer ကို မအစားထိုးသင့်ပါဘူး။

Tool ကို Programmer က ထိန်းချုပ်နိုင်ရပါမယ်။

Evil Wizards ရဲ့ အဓိကသင်ခန်းစာ

Automation နဲ့ Code Generation က အလွန်အသုံးဝင်ပါတယ်။ ဒါပေမယ့်—

နားမလည်တဲ့အရာကို အလွယ်တကူ မယုံပါနဲ့။

Generated Code ကို ကိုယ်တိုင် နားလည်နိုင်ရပါမယ်။

Tool ကို အသုံးပြုတာဟာ Problem ကို ဖြေရှင်းဖို့ ဖြစ်ရမယ်။

Tool ကို အသုံးပြုနေရင်း Tool ကိုယ်တိုင်က Problem မဖြစ်သွားအောင် သတိထားရပါမယ်။

Chapter 6 ရဲ့ အဓိကအယူအဆက — Coding ဟာ Mechanical အလုပ်မဟုတ်ဘဲ အမြဲတမ်း စဉ်းစား၊ စစ်ဆေး၊ ပြုပြင်ပြီး တာဝန်ယူလုပ်ဆောင်ရတဲ့ အလုပ်ဖြစ်ပါတယ်။

Chapter 7 — Before the Project

ပရောဂျက်တစ်ခုကို မစတင်ခင်ကတည်းက "ဒီပရောဂျက်က အောင်မြင်မှာမဟုတ်ဘူး" လို့ ခံစားဖူးပါသလား။ တစ်ခါတလေမှာ အဲဒီလိုဖြစ်နိုင်ပါတယ်။ အခြေခံစည်းမျဉ်းတွေကို အစကတည်းက မသတ်မှတ်ထားဘူးဆိုရင် ပရောဂျက်ကို အခုကတည်းက ရပ်လိုက်ပြီး ပရောဂျက်အတွက် ထောက်ပံ့ပေးနေသူရဲ့ ငွေကြေးကို ချွေတာတာက ပိုကောင်းနိုင်ပါတယ်။

ပရောဂျက်တစ်ခုရဲ့ အစမှာ Requirements တွေကို သတ်မှတ်ရပါမယ်။ User တွေကို နားထောင်ရုံနဲ့ မလုံလောက်ပါဘူး။ **The Requirements Pit** မှာ ဒီအကြောင်းကို ပိုပြီးလေ့လာနိုင်ပါတယ်။

Solving Impossible Puzzles မှာ သမားရိုးကျအမြင်တွေနဲ့ Constraints တွေကို ဘယ်လိုစီမံမလဲဆိုတာ ဆွေးနွေးထားပါတယ်။ Requirements ရေးဆွဲတာပဲဖြစ်ဖြစ်၊ Analysis လုပ်တာပဲဖြစ်ဖြစ်၊ Coding ဒါမှမဟုတ် Testing လုပ်တာပဲဖြစ်ဖြစ် ခက်ခဲတဲ့ပြဿနာတွေ ပေါ်လာမှာပါ။ ဒါပေမယ့် အများအားဖြင့် အစမှာထင်ရသလောက် မခက်ခဲပါဘူး။

ပြဿနာတွေကို ဖြေရှင်းပြီးပြီလို့ထင်ရပေမယ့် စတင်လုပ်ဆောင်ဖို့ စိတ်ထဲမှာ မသက်မသာဖြစ်နေတတ်ပါတယ်။ အဲဒါဟာ ရိုးရိုးအလုပ်ရွှေ့ဆိုင်းချင်တာလား၊ ဒါမှမဟုတ် တကယ်သတိထားသင့်တဲ့အရာတစ်ခုရှိနေတာလား။ **Not Until You're Ready** က ဘယ်အချိန်မှာ ကိုယ့်အတွင်းစိတ်က "စောင့်ပါဦး" လို့ပြောတာကို နားထောင်သင့်တယ်ဆိုတာ အကြံပြုထားပါတယ်။

စောလွန်းပြီး စတင်တာက ပြဿနာတစ်ခုဖြစ်သလို၊ အချိန်အရမ်းကြာအောင် စောင့်နေတာကလည်း ပိုဆိုးနိုင်ပါတယ်။ **The Specification Trap** မှာ Specification ကို အသေးစိတ်ရေးဆွဲရာမှာ ဖြစ်လာနိုင်တဲ့ ပြဿနာတွေနဲ့ Specification by Example ရဲ့ အကျိုးကျေးဇူးတွေကို ဆွေးနွေးထားပါတယ်။

နောက်ဆုံးမှာ **Circles and Arrows** မှာ Formal Development Process နဲ့ Methodology တွေကို အသုံးပြုတဲ့အခါ ဖြစ်နိုင်တဲ့ ပြဿနာတွေကို လေ့လာပါမယ်။ ဘယ်လောက်ပဲ သေချာစွာ စဉ်းစားထားတဲ့ Method ဖြစ်ပါစေ၊ ဘယ်လောက်ပဲ "Best Practice" တွေ ပါဝင်ပါစေ—**ဘယ် Method ကမှ စဉ်းစားခြင်းကို အစားထိုးလို့မရပါဘူး။**

ပရောဂျက်မစခင် ဒီအရေးကြီးတဲ့အချက်တွေကို ဖြေရှင်းထားနိုင်ရင် **Analysis Paralysis** ကို ရှောင်ရှားနိုင်ပြီး ပရောဂျက်ကို အောင်မြင်စွာ စတင်နိုင်ဖို့ ပိုကောင်းတဲ့အနေအထားမှာ ရှိလာပါလိမ့်မယ်။

36. The Requirements Pit (Requirements တွေရဲ့ ထောင်ချောက်)

"Perfection is achieved, not when there is nothing left to add, but when there is nothing left to take away."
"ပြီးပြည့်စုံခြင်းဆိုတာ ထပ်ထည့်စရာ ဘာမှမရှိတော့တဲ့အချိန်မှာ မဟုတ်ဘဲ၊ ထပ်ဖယ်စရာ ဘာမှမရှိတော့တဲ့အချိန်မှာ ရရှိတာဖြစ်တယ်။"
— Antoine de St. Exupery

စာအုပ်တွေနဲ့ Tutorials အများစုမှာ Requirements Gathering ကို Project ရဲ့ အစပိုင်း Phase တစ်ခုအဖြစ် ဖော်ပြကြပါတယ်။

"Gathering" ဆိုတဲ့စကားလုံးကို ကြည့်လိုက်ရင် Requirements တွေက မြေပြင်ပေါ်မှာ ရှိပြီးသားဖြစ်ပြီး Analysts တွေက လိုက်လံရှာဖွေပြီး စုဆောင်းရုံပဲလိုတယ်လို့ ထင်ရပါတယ်။

ဒါပေမယ့် တကယ့်လက်တွေ့မှာ အဲဒီလို မဟုတ်ပါဘူး။

Requirements တွေဟာ များသောအားဖြင့် မျက်နှာပြင်ပေါ်မှာ ရှင်းရှင်းလင်းလင်း မရှိပါဘူး။ Assumptions တွေ၊ အထင်အမြင်လွဲမှားမှုတွေ၊ နိုင်ငံရေးဆန်တဲ့ အကျိုးစီးပွားတွေ စတဲ့ အလွှာများစွာအောက်မှာ မြှုပ်နှံထားတတ်ပါတယ်။

TIP 51

Requirements တွေကို "စုဆောင်း" မနေပါနဲ့ — တူးဖော်ရှာဖွေပါ

Digging for Requirements (Requirements တွေကို တူးဖော်ရှာဖွေခြင်း)

ဒီလိုပတ်ဝန်းကျင်အညစ်အကြေးတွေကြားထဲကနေ တကယ့် Requirement ကို ဘယ်လိုခွဲခြားသိနိုင်မလဲ။

အဖြေက ရိုးရှင်းသလို ရှုပ်ထွေးလည်းပါတယ်။

ရိုးရှင်းတဲ့အဖြေကတော့ Requirement ဆိုတာ **ပြီးမြောက်အောင် လုပ်ဆောင်ရမယ့်အရာတစ်ခုကို ဖော်ပြထားတဲ့ Statement** ဖြစ်ပါတယ်။

ဥပမာ—

- Employee Record တစ်ခုကို သတ်မှတ်ထားတဲ့ လူအုပ်စုကသာ ကြည့်ရှုနိုင်ရမယ်။
- Cylinder-head ရဲ့ အပူချိန်ဟာ Engine တစ်ခုချင်းစီအတွက် သတ်မှတ်ထားတဲ့ Critical Value ကို မကျော်ရဘူး။
- Editor က Keyword တွေကို Highlight လုပ်ရမယ်။ Highlight လုပ်မယ့် Keyword တွေက ပြင်ဆင်နေတဲ့ File အမျိုးအစားပေါ်မူတည်ရမယ်။

ဒါပေမယ့် Requirements အများစုဟာ ဒီလိုရှင်းလင်းတိကျမနေပါဘူး။ အဲဒါကြောင့် Requirements Analysis က ရှုပ်ထွေးလာတာဖြစ်ပါတယ်။

Requirement, Policy နဲ့ Implementation

ဥပမာ User က—

“Employee တစ်ယောက်ရဲ့ Record ကို သူ့ရဲ့ Supervisor တွေနဲ့ Personnel Department ပဲ ကြည့်နိုင်ရမယ်။”

လို့ ပြောတယ်ဆိုပါစို့။

ဒါဟာ တကယ့် Requirement ဟုတ်ပါသလား။

ဒီနေ့အတွက်တော့ ဟုတ်နိုင်ပါတယ်။ ဒါပေမယ့် Business Policy တစ်ခုကို Absolute Statement အနေနဲ့ ထည့်သွင်းထားပါတယ်။

Policy တွေဟာ ပုံမှန်ပြောင်းလဲတတ်ပါတယ်။ ဒါကြောင့် Policy ကို Requirement ထဲမှာ တိုက်ရိုက် Hard-code မလုပ်သင့်ပါဘူး။

Policy ကို Requirement နဲ့ သီးခြား Document လုပ်ပြီး နှစ်ခုကို ချိတ်ဆက်ထားတာ ပိုကောင်းပါတယ်။

Requirement ကို General Statement အနေနဲ့ ရေးပြီး Policy ကို Implementation မှာ Support လုပ်ရမယ့် အရာတစ်ခုအဖြစ် ဥပမာပေးနိုင်ပါတယ်။

နောက်ပိုင်းမှာ Policy ကို Application ထဲမှာ **Metadata** အဖြစ် သိမ်းဆည်းနိုင်ပါတယ်။

ဒီကွာခြားချက်ဟာ သေးငယ်သလိုထင်ရပေမယ့် Developer တွေအတွက် အရေးကြီးတဲ့ အကျိုးသက်ရောက်မှုရှိပါတယ်။

“Only personnel can view an employee record” လို့ Requirement ရေးထားရင် Developer က Application က File တစ်ခုကို Access လုပ်တိုင်း Explicit Test တစ်ခုထည့်ဖို့ ဖြစ်နိုင်ပါတယ်။

ဒါပေမယ့်—

“Only authorized users may access an employee record.”
လို့ ရေးထားရင် Developer က Access Control System တစ်ခုကို Design လုပ်ပြီး Implement လုပ်နိုင်ပါတယ်။

Policy ပြောင်းသွားတဲ့အခါ Code ကို ပြန်ပြင်စရာမလိုဘဲ System ရဲ့ Metadata ကိုပဲ ပြင်နိုင်ပါတယ်။
ဒီလို Requirements တွေကို စုဆောင်းတဲ့ပုံစံက Metadata ကို Support လုပ်နိုင်တဲ့ System Design တစ်ခုဆီကို သဘာဝအတိုင်း ဦးတည်စေပါတယ်။

User Interface နဲ့ Requirements

User Interface ကို ဆွေးနွေးတဲ့အခါ Requirement, Policy နဲ့ Implementation ကြားက ကွာခြားချက်တွေ ပိုပြီး မရှင်းလင်းတော့တတ်ပါတယ်။
ဥပမာ—
"System က Loan Term ကို ရွေးချယ်နိုင်အောင် လုပ်ပေးရမယ်။"
ဒါဟာ Requirement ဖြစ်ပါတယ်။
ဒါပေမယ့်—
"Loan Term ရွေးဖို့ List Box တစ်ခု လိုတယ်။"
ဆိုတာက Requirement ဟုတ်ချင်မှ ဟုတ်ပါလိမ့်မယ်။
User တွေက တကယ်ပဲ List Box ကို မဖြစ်မနေလိုအပ်တယ်ဆိုရင် Requirement ဖြစ်နိုင်ပါတယ်။
ဒါပေမယ့် User က "ရွေးချယ်နိုင်ရမယ်" ဆိုတဲ့အဓိပ္ပာယ်နဲ့ List Box ကို ဥပမာအနေနဲ့ပဲ ပြောတာဆိုရင် Requirement မဟုတ်နိုင်ပါဘူး။
အရေးကြီးတာက User တွေက တစ်ခုခုကို **ဘာကြောင့်** လုပ်နေတာလဲဆိုတာ ရှာဖွေဖို့ပါ။
သူတို့ လက်ရှိလုပ်နေတဲ့နည်းလမ်းကိုပဲ ကူးယူဖို့မဟုတ်ပါဘူး။
နောက်ဆုံးမှာ Development က User ရဲ့ **Business Problem** ကို ဖြေရှင်းပေးရမှာဖြစ်ပြီး User ပြောထားတဲ့ Requirements တွေကို စာသားအတိုင်း ဖြည့်ဆည်းပေးရုံနဲ့ မပြီးပါဘူး။
Requirement တစ်ခုရဲ့ နောက်ကွယ်က အကြောင်းရင်းကို Document လုပ်ထားခြင်းက နေ့စဉ် Implementation Decision တွေ ချမှတ်ရာမှာ Team အတွက် အလွန်တန်ဖိုးရှိတဲ့ အချက်အလက်ဖြစ်လာပါတယ်။

TIP 52 -User တစ်ယောက်နဲ့အတူ အလုပ်လုပ်ပြီး User လို စဉ်းစားပါ

User တွေရဲ့ Requirements ကို အမှန်တကယ် နားလည်ဖို့ အကောင်းဆုံးနည်းလမ်းတွေထဲက တစ်ခုက **ကိုယ်တိုင် User ဖြစ်ကြည့်ခြင်း** ပါ။
Help Desk အတွက် System တစ်ခု ရေးနေတယ်ဆိုရင် Experienced Support Person တစ်ယောက်နဲ့အတူ ဖုန်းတွေကို ရက်အနည်းငယ် စောင့်ကြည့်ပါ။
Manual Stock Control System တစ်ခုကို Automation လုပ်နေတယ်ဆိုရင် Warehouse ထဲမှာ တစ်ပတ်လောက် အလုပ်လုပ်ကြည့်ပါ။
ဒီလိုလုပ်ခြင်းက System ကို တကယ့်လက်တွေ့မှာ ဘယ်လိုအသုံးပြုမလဲဆိုတာ နားလည်စေတဲ့အပြင် User တွေနဲ့ ယုံကြည်မှုတည်ဆောက်ဖို့လည်း ကူညီပေးပါတယ်။

Documenting Requirements

User တွေနဲ့ထိုင်ပြီး တကယ့် Requirements တွေကို ရှာဖွေပြီးပြီဆိုပါစို့။

Application က ဘာတွေလုပ်ပေးရမလဲဆိုတဲ့ Scenario တချို့ကိုလည်း ရလာပါပြီ။

Professional တစ်ယောက်အနေနဲ့ ဒီအရာတွေကို Document လုပ်ပြီး Developers၊ End Users နဲ့ Project Sponsors အားလုံး ဆွေးနွေးနိုင်မယ့် Document တစ်ခု ပြုလုပ်ချင်ပါလိမ့်မယ်။

ဒါပေမယ့် ဒီလူအုပ်စုတွေဟာ လိုအပ်ချက် မတူကြပါဘူး။

Ivar Jacobson က Requirements တွေကို ဖော်ပြဖို့ **Use Cases** ဆိုတဲ့ Concept ကို အဆိုပြုခဲ့ပါတယ်။

Use Case တွေက System ကို ဘယ်လိုအသုံးပြုမလဲဆိုတာ User Interface အဆင့်နဲ့မဟုတ်ဘဲ ပိုမို Abstract ဖြစ်တဲ့ပုံစံနဲ့ ဖော်ပြနိုင်ပါတယ်။

ဒါပေမယ့် Use Case ဘယ်လိုရေးရမလဲဆိုတဲ့ အသေးစိတ်အချက်တွေမှာ အမြင်ကွဲပြားမှုများစွာ ရှိလာပါတယ်။

Formal ဖြစ်ရမလား၊ Informal ဖြစ်ရမလား။

ရိုးရိုး Prose နဲ့ရေးရမလား၊ Form လို Structured Document နဲ့ရေးရမလား။

ဘယ်လောက်အသေးစိတ် ရေးသင့်သလဲ။

Audience က လူအမျိုးမျိုးဖြစ်နေတာကိုလည်း မမေ့သင့်ပါဘူး။

Sometimes the Interface Is the System

Wired Magazine မှာ Producer နဲ့ Musician ဖြစ်တဲ့ Brian Eno က အလွန်အံ့သြစရာကောင်းတဲ့ Technology တစ်ခု အကြောင်း ဖော်ပြခဲ့ပါတယ်။

အဲဒါက အသံနဲ့ပတ်သက်ပြီး လုပ်နိုင်သမျှအရာအားလုံးကို လုပ်နိုင်တဲ့ Mixing Board တစ်ခုပါ။

ဒါပေမယ့် Musician တွေကို ပိုကောင်းတဲ့ Music ဖန်တီးနိုင်အောင်၊ Recording ကို ပိုမြန်မြန်နဲ့ စရိတ်သက်သက်သာသာ လုပ်နိုင်အောင် မကူညီဘဲ သူတို့ရဲ့ Creative Process ကို အနှောင့်အယှက်ပေးနေပါတယ်။

Recording Engineer တွေဟာ အသံတွေကို အလိုလို Balance လုပ်တတ်ကြပါတယ်။

နှစ်ပေါင်းများစွာ အတွေ့အကြုံကြောင့် သူတို့ရဲ့ နားနဲ့ လက်ချောင်းတွေကြားမှာ Feedback Loop တစ်ခု ရှိနေပါတယ်။

Fader တွေကို ရွှေ့ခြင်း၊ Knob တွေကို လှည့်ခြင်း စတာတွေဟာ သူတို့အတွက် သဘာဝကျပါတယ်။

ဒါပေမယ့် Mixer အသစ်ရဲ့ Interface က ဒီ Skill တွေကို အသုံးမချဘဲ Keyboard နဲ့ ရိုက်တာ၊ Mouse နဲ့ Click လုပ်တာတွေကို အတင်းအကျပ် လိုအပ်စေခဲ့ပါတယ်။

Function တွေ အပြည့်အစုံပါဝင်ပေမယ့် User တွေအတွက် မရင်းနှီးတဲ့ပုံစံနဲ့ ထည့်သွင်းထားပါတယ်။

ဒီဥပမာက Requirement တစ်ခုကို ဖော်ပြပါတယ်—

ရှိပြီးသား User Skill တွေကို အသုံးချနိုင်ရမယ်။

လက်ရှိနည်းလမ်းကို အတိအကျ Copy လုပ်တာက တိုးတက်မှုကို တားဆီးနိုင်ပါတယ်။

ဒါပေမယ့် Future ကို သွားနိုင်ဖို့အတွက် Familiar ဖြစ်တဲ့ အတွေ့အကြုံကနေ အသစ်ဆီကို ပြောင်းရွှေ့နိုင်မယ့် Transition တစ်ခုတော့ ပေးရပါမယ်။

အောင်မြင်တဲ့ Tools တွေဟာ သူတို့ကို အသုံးပြုမယ့် လူတွေရဲ့ လက်နဲ့ လိုက်လျောညီထွေဖြစ်ရပါတယ်။

သင် တခြားသူတွေအတွက် တည်ဆောက်ပေးတဲ့ Tools တွေဟာလည်း Adaptable ဖြစ်ရပါမယ်။

Overspecifying (Specification ကို လိုတာထက်ပိုပြီး အသေးစိတ်သတ်မှတ်ခြင်း)

Requirements Document တစ်ခုရေးရာမှာ ဖြစ်နိုင်တဲ့ အန္တရာယ်ကြီးတစ်ခုက **အရမ်းအသေးစိတ်ရေးခြင်း** ဖြစ်ပါတယ်။

ကောင်းမွန်တဲ့ Requirements Document တွေဟာ Abstract ဖြစ်သင့်ပါတယ်။

Requirement တွေနဲ့ပတ်သက်လာရင် Business Need ကို တိတိကျကျ ဖော်ပြနိုင်တဲ့ အရိုးရှင်းဆုံး Statement က အကောင်းဆုံးဖြစ်ပါတယ်။

ဒါဟာ မရှင်းမလင်းရေးရမယ်လို့ ဆိုလိုတာမဟုတ်ပါဘူး။

Requirement အနေနဲ့ အခြေခံလိုအပ်ချက်တွေကို တိကျစွာ ဖမ်းယူရမယ်။

လက်ရှိလုပ်ငန်းစဉ်တွေ၊ လက်ရှိအလုပ်လုပ်ပုံတွေကိုတော့ Policy အနေနဲ့ Document လုပ်ထားသင့်ပါတယ်။

Requirements ဟာ Architecture မဟုတ်ပါဘူး။

Requirements ဟာ Design မဟုတ်ပါဘူး။

Requirements ဟာ User Interface မဟုတ်ပါဘူး။

Requirements ဆိုတာ Need ဖြစ်ပါတယ်။

TIP 53
Abstraction တွေဟာ Details တွေထက် ပိုကြာရှည်ခံတယ်

Requirements တွေမှာ Future ကို တိတိကျကျ ကြိုတင်ခန့်မှန်းဖို့ မလိုပါဘူး။

ဥပမာ—

“System သည် DATE များကို Abstraction တစ်ခုအဖြစ် အသုံးပြုမည်။ DATE Services များဖြစ်သည့် Formatting၊ Storage နှင့် Math Operations များကို တစ်သမတ်တည်း အသုံးပြုမည်။”

ဒီလိုရေးထားရင် DATE ကို အသုံးပြုရမယ်ဆိုတာ Requirement အဖြစ် သတ်မှတ်ထားပေမယ့် DATE ကို ဘယ်လို Implement လုပ်မလဲဆိုတာကို အလွန်အကျွံ မသတ်မှတ်ထားပါဘူး။

Just One More Wafer-Thin Mint...

"Feature တစ်ခုလောက် ထပ်ထည့်လိုက်ဦးမယ်..."

Project Failure အများစုကို Scope တိုးလာခြင်းနဲ့ ဆက်စပ်ပြီး အပြစ်တင်ကြပါတယ်။

ဒါကို—

- Feature Bloat
- Creeping Featurism
- Requirements Creep

စတဲ့ နာမည်တွေနဲ့ ခေါ်ကြပါတယ်။

Requirements တွေ တဖြည်းဖြည်းတိုးလာတာကို ဘယ်လိုကာကွယ်နိုင်မလဲ။

Project တစ်ခုမှာ Bugs Reported and Fixed၊ Defect Density၊ Cohesion၊ Coupling၊ Function Points၊ Lines of Code စတဲ့ Metrics တွေကို Track လုပ်နိုင်ပါတယ်။

ဒါပေမယ့် Requirements ကိုတော့ တကယ်ကြွစွာ Track လုပ်တဲ့ Project တွေ မများပါဘူး။

အဲဒါကြောင့် Scope ပြောင်းလဲမှုတွေကို ဘယ်သူက Feature တောင်းဆိုခဲ့သလဲ၊ ဘယ်သူက Approve လုပ်ခဲ့သလဲ၊ စုစုပေါင်း Request ဘယ်နှခု Approve ဖြစ်ခဲ့သလဲ စတာတွေကို မသိနိုင်တော့ပါဘူး။

Requirements Growth ကို စီမံဖို့ အဓိကအချက်က Feature အသစ်တစ်ခုချင်းစီက Schedule ပေါ်မှာ ဘယ်လောက်သက်ရောက်မှုရှိသလဲဆိုတာ Project Sponsors တွေကို ပြသပေးဖို့ပါ။

"Feature တစ်ခုလောက်ပဲ ထပ်ထည့်မယ်" ဆိုတာကို လက်ခံနေရင်းနဲ့ ဒီလအတွင်း Feature အသစ် ၁၅ ခုတောင် ထပ်ထည့်ထားပြီးဖြစ်နိုင်ပါတယ်။

ဒါကြောင့် Requirements တွေကို Track လုပ်ပါ။

Maintain a Glossary

Requirements တွေကို စတင်ဆွေးနွေးတဲ့အချိန်မှာ User တွေနဲ့ Domain Experts တွေဟာ သူတို့အတွက် သီးခြားအဓိပ္ပာယ်ရှိတဲ့ Terms တွေကို အသုံးပြုကြပါလိမ့်မယ်။

ဥပမာ "Client" နဲ့ "Customer" ကို ကွဲကွဲပြားပြား သုံးနိုင်ပါတယ်။

Project ထဲမှာ ဒီစကားလုံးတွေကို မတူညီတဲ့အဓိပ္ပာယ်နဲ့ အလွတ်သဘော အသုံးမပြုသင့်ပါဘူး။

Project ထဲမှာ အသုံးပြုတဲ့ Terms နဲ့ Vocabulary အားလုံးကို အဓိပ္ပာယ်သတ်မှတ်ပေးထားတဲ့ **Project Glossary** တစ်ခု ဖန်တီးပြီး ထိန်းသိမ်းထားသင့်ပါတယ်။

End Users ကနေ Support Staff အထိ Project ထဲမှာ ပါဝင်သူအားလုံးက Glossary ကို အသုံးပြုသင့်ပါတယ်။

TIP 54

Project Glossary တစ်ခု အသုံးပြုပါ

User နဲ့ Developer တွေက အရာတစ်ခုတည်းကို နာမည်မတူဘဲ ခေါ်တာ၊ ဒါမှမဟုတ် နာမည်တစ်ခုတည်းနဲ့ အရာမတူတာတွေကို ခေါ်နေရင် Project အောင်မြင်ဖို့ အလွန်ခက်ခဲပါတယ်။

Get the Word Out
အချက်အလက်တွေကို လူတိုင်းသိအောင် ဖြန့်ဝေပါ

Project Documents တွေကို Project ထဲက ပါဝင်သူအားလုံး အလွယ်တကူ ဝင်ကြည့်နိုင်အောင် Internal Web Site တွေမှာ ထားနိုင်ပါတယ်။

Requirements Document တွေအတွက် ဒီနည်းလမ်းက အထူးအသုံးဝင်ပါတယ်။

Requirements ကို Hypertext Document အနေနဲ့ တင်ထားရင် Audience အမျိုးမျိုးရဲ့ လိုအပ်ချက်ကို ဖြည့်ဆည်းပေးနိုင်ပါတယ်။

Project Sponsors တွေက Business Objectives တွေ ပြည့်မီမမီကို High Level မှာ ကြည့်နိုင်ပါတယ်။

Programmers တွေကတော့ Hyperlink တွေကို အသုံးပြုပြီး Detail Level တွေထဲကို ဆင်းကြည့်နိုင်ပါတယ်။

Web-based Distribution က စာရွက်နဲ့ထုတ်ထားတဲ့ Requirements Analysis စာအုပ်ကြီးတွေကိုလည်း ရှောင်ရှားနိုင်ပါတယ်။

အဲဒီလိုစာအုပ်တွေကို ဘယ်သူမှ မဖတ်ဘဲ စာရွက်ပေါ် Ink မခြောက်ခင်ကတည်းက Outdated ဖြစ်သွားတတ်ပါတယ်။

Challenges

- သင်ရေးနေတဲ့ Software ကို သင်ကိုယ်တိုင် အသုံးပြုနိုင်ပါသလား။
- ကိုယ်တိုင် Software ကို အသုံးမပြုနိုင်ဘဲ Requirements ကို ကောင်းကောင်းနားလည်နိုင်ပါသလား။
- လက်ရှိဖြေရှင်းရမယ့် Computer မဟုတ်တဲ့ Problem တစ်ခုကို ရွေးပြီး Non-computer Solution အတွက် Requirements တွေ ရေးကြည့်ပါ။

37. Solving Impossible Puzzles (မဖြစ်နိုင်ဘူးလို့ထင်ရတဲ့ ပဟေဠိတွေကို ဖြေရှင်းခြင်း)

Gordius ဆိုတဲ့ Phrygia ဘုရင်က ဘယ်သူမှ ဖြေလို့မရတဲ့ Knot တစ်ခု ချည်ထားခဲ့ပါတယ်။

အဲဒီ Knot ကို ဖြေနိုင်တဲ့သူဟာ Asia တစ်ခုလုံးကို အုပ်ချုပ်နိုင်မယ်လို့ ဆိုခဲ့ကြပါတယ်။

Alexander the Great ရောက်လာတဲ့အခါ Knot ကို ဖြေဖို့မကြိုးစားဘဲ သူ့ရဲ့ဓားနဲ့ အပိုင်းပိုင်း ဖြတ်လိုက်ပါတယ်။

Requirements ကို နည်းနည်းကွဲပြားစွာ နားလည်လိုက်တာပဲ ဖြစ်ပါတယ်။

Project တစ်ခုထဲမှာ တစ်ခါတလေ အလွန်ခက်ခဲတဲ့ Problem တစ်ခုနဲ့ ကြုံရပါလိမ့်မယ်။

Engineering ပိုင်းက ကိုင်တွယ်လို့မရတဲ့ အရာတစ်ခု ဖြစ်နိုင်သလို၊ ထင်ထားတာထက် အများကြီးခက်ခဲနေတဲ့ Code တစ်ခုလည်း ဖြစ်နိုင်ပါတယ်။

မဖြစ်နိုင်ဘူးလို့တောင် ထင်ရနိုင်ပါတယ်။

ဒါပေမယ့် တကယ်ပဲ အဲဒီလောက်ခက်ခဲတာ ဟုတ်ပါသလား။

Degrees of Freedom - လွတ်လပ်စွာ ရွေးချယ်နိုင်တဲ့ နယ်ပယ်

"Think outside the box" ဆိုတဲ့ စကားလုံးက မသက်ဆိုင်တဲ့ Constraints တွေကို ဖယ်ရှားဖို့ အားပေးပါတယ်။

ဒါပေမယ့် ဒီစကားလုံးဟာ အပြည့်အဝ တိကျတယ်လို့ မဆိုနိုင်ပါဘူး။

"Box" ဆိုတာ Constraints နဲ့ Conditions တွေရဲ့ နယ်နိမိတ်ဆိုရင် အဓိကက **Box ကို ရှာဖို့** ဖြစ်ပါတယ်။

သင်ထင်ထားတာထက် Box က ပိုကြီးနေနိုင်ပါတယ်။

Puzzle တစ်ခုကို ဖြေရှင်းဖို့အတွက်—

1. သင့်အပေါ် ချမှတ်ထားတဲ့ Constraints တွေကို သိရမယ်။
2. သင် လွတ်လပ်စွာ ရွေးချယ်နိုင်တဲ့ နယ်ပယ်တွေကို သိရမယ်။

အဲဒီနှစ်ခုကြားမှာပဲ Solution ကို ရှာတွေ့နိုင်ပါတယ်။

TIP 55
"Box အပြင်ဘက်မှာ စဉ်းစားပါ" မဟုတ်ဘဲ "Box ကို ရှာပါ"

မဖြေရှင်းနိုင်လောက်အောင် ခက်ခဲတဲ့ Problem တစ်ခုနဲ့ ကြုံလာရင် ရနိုင်သမျှ လမ်းကြောင်းအားလုံးကို စာရင်းပြုစုပါ။

မဖြစ်နိုင်ဘူး၊ မအသုံးဝင်ဘူး၊ မိုက်မဲတယ်လို့ ထင်ရတဲ့နည်းလမ်းကိုတောင် မဖယ်လိုက်ပါနဲ့။

ပြီးရင် လမ်းကြောင်းတစ်ခုချင်းစီကို ဘာကြောင့် မသုံးနိုင်တာလဲဆိုတာ ရှင်းပြပါ။

သေချာပါသလား။

သက်သေပြနိုင်ပါသလား။

ပြီးတော့ Constraints တွေကို Category ခွဲပြီး Priority သတ်မှတ်ပါ။

Woodworker တစ်ယောက်က သစ်သားကို ဖြတ်တဲ့အခါ အရှည်ဆုံးအပိုင်းကို အရင်ဖြတ်ပြီး ကျန်တဲ့သစ်သားထဲကနေ ပိုသေးတဲ့အပိုင်းတွေကို ဖြတ်ပါတယ်။

ဒီလိုပဲ Project မှာလည်း အတင်းကျပ်ဆုံး Constraint တွေကို အရင်သတ်မှတ်ပြီး ကျန်တဲ့ Constraints တွေကို အဲဒီအတွင်းမှာ လိုက်လျောညီထွေဖြစ်အောင် စီစဉ်သင့်ပါတယ်။

There Must Be an Easier Way! - "ဒီထက် လွယ်တဲ့နည်းလမ်းတော့ ရှိရမယ်!"

တစ်ခါတလေ သင်ထင်ထားတာထက် အများကြီးခက်ခဲတဲ့ Problem တစ်ခုကို လုပ်နေရနိုင်ပါတယ်။

လမ်းမှားကို လိုက်နေသလို ခံစားရနိုင်ပါတယ်။

"ဒီထက် လွယ်တဲ့နည်းလမ်းတော့ ရှိရမယ်" လို့ တွေးမိနိုင်ပါတယ်။

အဲဒီအချိန်မှာ နောက်တစ်လှမ်းပြန်ဆုတ်ပြီး ဒီမေးခွန်းတွေကို ကိုယ့်ကိုယ်ကို မေးပါ—

- **ပိုလွယ်တဲ့နည်းလမ်း ရှိပါသလား။**
- **သင်ဖြေရှင်းနေတာက မှန်ကန်တဲ့ Problem ဟုတ်ပါသလား။ ဒါမှမဟုတ် အရေးမကြီးတဲ့ Technical Detail တစ်ခုက သင့်ကို အာရုံလွှဲနေပါသလား။**
- **ဒီအရာက ဘာကြောင့် Problem ဖြစ်နေတာလဲ။**
- **ဘာက ဒီ Problem ကို အရမ်းခက်ခဲအောင် လုပ်နေတာလဲ။**
- **ဒီနည်းလမ်းအတိုင်းပဲ လုပ်ရမှာလား။**
- **တကယ်ပဲ ဒီအရာကို လုပ်ဖို့ လိုအပ်ရဲ့လား။**

ဒီမေးခွန်းတွေထဲက တစ်ခုကို ဖြေဖို့ကြိုးစားရင်း အံ့အားသင့်စရာအဖြေတစ်ခု ရလာနိုင်ပါတယ်။

Requirements ကို နည်းနည်းပြန်အဓိပ္ပာယ်ဖွင့်လိုက်ရုံနဲ့ Problem အများကြီး ပျောက်ကွယ်သွားနိုင်ပါတယ်။

Gordian Knot ကို ဖြေရှင်းခဲ့သလိုပေါ့။

သင်လိုအပ်တာက—

တကယ့် Constraints၊ အမှန်မဟုတ်တဲ့ Constraints နဲ့ နှစ်ခုကို ခွဲခြားသိနိုင်တဲ့ ဉာဏ်ပညာ ပါပဲ။

38. Not Until You're Ready (အဆင်သင့်မဖြစ်မချင်း မစတင်ပါနဲ့)

"He who hesitates is sometimes saved."
"တစ်ခါတလေ တွန့်ဆုတ်တတ်သူကပဲ ဘေးအန္တရာယ်ကနေ လွတ်မြောက်တတ်တယ်။"
— James Thurber

ထူးချွန်တဲ့ Performer တွေမှာ တူညီတဲ့ အရည်အချင်းတစ်ခုရှိပါတယ်။

သူတို့ဟာ **ဘယ်အချိန်မှာ စတင်ရမလဲ၊ ဘယ်အချိန်မှာ စောင့်ရမလဲ** ဆိုတာ သိကြပါတယ်။

Diver တစ်ယောက်က High Board ပေါ်မှာ ခုန်ချဖို့ အချိန်ကောင်းကို စောင့်နေတတ်ပါတယ်။

Conductor တစ်ယောက်ကလည်း Orchestra ရှေ့မှာ လက်မြှောက်ထားပြီး စတင်ဖို့ အချိန်မှန်ပြီလို့ ခံစားရတဲ့အထိ စောင့်တတ်ပါတယ်။

Developer တစ်ယောက်အနေနဲ့လည်း အတွင်းစိတ်က "စောင့်ပါဦး" လို့ တိုးတိုးပြောလာတဲ့အခါ နားထောင်သင့်ပါတယ်။

Coding စတင်ဖို့ ထိုင်လိုက်တဲ့အချိန်မှာ စိတ်ထဲမှာ မသက်မသာဖြစ်စေတဲ့ သံသယတစ်ခုရှိနေတယ်ဆိုရင် အဲဒါကို အလေးထားပါ။

TIP 56 - ကိုယ့်ရဲ့ သံသယတွေကို နားထောင်ပါ — အဆင်သင့်ဖြစ်တဲ့အချိန်မှာ စတင်ပါ

Developer တစ်ယောက်အနေနဲ့ သင့် Career တစ်လျှောက်မှာလည်း အလားတူအရာတွေကို လုပ်နေခဲ့တာပါ။

ဘာက အလုပ်ဖြစ်တယ်၊ ဘာက အလုပ်မဖြစ်ဘူးဆိုတာ စမ်းသပ်ပြီး အတွေ့အကြုံနဲ့ ဉာဏ်ပညာတွေ စုဆောင်းလာခဲ့ပါတယ်။

Task တစ်ခုကို လုပ်ရမယ့်အချိန်မှာ စိတ်ထဲက သံသယဖြစ်လာတယ်၊ မလုပ်ချင်သလို ခံစားလာတယ်ဆိုရင် အဲဒီ Feeling ကို အလေးထားပါ။

ဘာကမှားနေတာလဲဆိုတာ ချက်ချင်း မပြောနိုင်သေးရင်တောင် အချိန်နည်းနည်းပေးပါ။

နောက်ပိုင်းမှာ အဲဒီသံသယက ပိုရှင်းလင်းတဲ့အရာတစ်ခုအဖြစ် ပြောင်းလဲလာပြီး သင်ဖြေရှင်းနိုင်တဲ့ Problem တစ်ခု ဖြစ်လာနိုင်ပါတယ်။

Software Development ဟာ ယနေ့တိုင် Science တစ်ခုအဖြစ် အပြည့်အဝ မဖြစ်သေးပါဘူး။

ဒါကြောင့် ကိုယ့်ရဲ့ Instinct တွေကိုလည်း ကိုယ့်ရဲ့ Performance ထဲမှာ ပါဝင်ခွင့်ပေးပါ။

Good Judgment or Procrastination?
(ကောင်းမွန်တဲ့ဆုံးဖြတ်ချက်လား၊ အလုပ်ရွှေ့ဆိုင်းနေတာလား)

လူတိုင်းဟာ စာရွက်ဖြူကြီးတစ်ရွက်ကို ကြည့်ပြီး စတင်ရေးရမှာကို ကြောက်တတ်ကြပါတယ်။

Project အသစ်တစ်ခု စတင်တာ၊ ဒါမှမဟုတ် ရှိပြီးသား Project ထဲမှာ Module အသစ်တစ်ခု စတင်တာကလည်း စိတ်မသက်သာစရာ ဖြစ်နိုင်ပါတယ်။

ဒါကြောင့် အစပြုဖို့ Initial Commitment ကို ရွှေ့ဆိုင်းချင်ကြပါတယ်။

ဒါဆို ကိုယ့်မှာ အဆင်သင့်ဖြစ်ဖို့ စောင့်နေတာလား၊ ရိုးရိုး Procrastination လုပ်နေတာလားဆိုတာ ဘယ်လိုသိနိုင်မလဲ။

ဒီလိုအခြေအနေမှာ အသုံးဝင်တဲ့နည်းလမ်းတစ်ခုက **Prototype စတင်လုပ်ခြင်း** ဖြစ်ပါတယ်။

ခက်ခဲမယ်လို့ ထင်တဲ့ Area တစ်ခုကို ရွေးပြီး Proof of Concept တစ်ခု စတင်တည်ဆောက်ပါ။

အများအားဖြင့် အခြေအနေ နှစ်မျိုးထဲက တစ်ခု ဖြစ်လာပါလိမ့်မယ်။

အစလုပ်ပြီး မကြာခင်မှာ "အချိန်ဖြုန်းနေတာပဲ" လို့ ခံစားလာနိုင်ပါတယ်။

အဲဒီလို Bored ဖြစ်လာတာက သင်စတင်ဖို့ တွန့်ဆုတ်ခဲ့တာဟာ တကယ်တော့ Commitment ကို ရွှေ့ဆိုင်းချင်တာသာ ဖြစ်နိုင်တယ်ဆိုတဲ့ လက္ခဏာပါ။

Prototype ကို ရပ်လိုက်ပြီး တကယ့် Development ကို စတင်ပါ။

တစ်ဖက်မှာတော့ Prototype လုပ်နေစဉ်မှာ ရုတ်တရက် အရေးကြီးတဲ့ အချက်တစ်ခုကို နားလည်သွားနိုင်ပါတယ်။

အခြေခံ Premise တစ်ခု မှားနေတယ်ဆိုတာ သိလာနိုင်ပြီး ဘယ်လိုပြင်ရမလဲဆိုတာလည်း ရှင်းရှင်းလင်းလင်း မြင်လာနိုင်ပါတယ်။

အဲဒီအချိန်မှာ Prototype ကို စွန့်လွှတ်ပြီး တကယ့် Project ကို စတင်ဖို့ စိတ်အေးသွားပါလိမ့်မယ်။

ဒီအခြေအနေမှာ သင့် Instinct က မှန်ကန်ခဲ့တာဖြစ်ပြီး သင်နဲ့ Team အတွက် အချိန်နဲ့ အားစိုက်ထုတ်မှု အများကြီးကို ချွေတာပေးလိုက်တာ ဖြစ်ပါတယ်။

Prototype ကို သင့်စိတ်ထဲက မသက်မသာဖြစ်မှုကို စူးစမ်းဖို့ အသုံးပြုနေတယ်ဆိုတာ မမေ့ပါနဲ့။

Prototype စလုပ်ပြီး ရက်သတ္တပတ်အနည်းငယ်ကြာမှ "ဒါက Prototype အနေနဲ့ စခဲ့တာပဲ" ဆိုတာ ပြန်သတိရသွားတာမျိုး မဖြစ်စေပါနဲ့။

39. The Specification Trap (Specification ရဲ့ ထောင်ချောက်)

Program Specification ဆိုတာ Requirement တစ်ခုကို Programmer ရဲ့ Skill က ဆက်လက်လုပ်ဆောင်နိုင်တဲ့ အဆင့်အထိ လျှော့ချသတ်မှတ်ပေးတဲ့ Process ဖြစ်ပါတယ်။

ဒါဟာ ကမ္ဘာကြီးကို ရှင်းလင်းဖော်ပြပြီး အဓိက Ambiguities တွေကို ဖယ်ရှားပေးတဲ့ Communication လုပ်ငန်းစဉ်တစ်ခု ဖြစ်ပါတယ်။

Specification ဟာ လက်ရှိ Developer အတွက်သာ မဟုတ်ပါဘူး။

အနာဂတ်မှာ Code ကို Maintain လုပ်မယ့် Developer တွေအတွက်လည်း မှတ်တမ်းတစ်ခု ဖြစ်ပါတယ်။

User နဲ့လည်း Agreement တစ်ခုဖြစ်ပြီး နောက်ဆုံး System ဟာ သူတို့လိုအပ်ချက်နဲ့ ကိုက်ညီရမယ်ဆိုတဲ့ Implicit Contract တစ်ခုလည်း ဖြစ်ပါတယ်။

ဒါကြောင့် Specification ရေးခြင်းဟာ တာဝန်ကြီးတစ်ခု ဖြစ်ပါတယ်။

ဒါပေမယ့် Designer အများစုဟာ ဘယ်အချိန်မှာ ရပ်ရမလဲဆိုတာ ခက်ခဲတတ်ပါတယ်။

အသေးစိတ်အရာတိုင်းကို အလွန်အမင်း တိကျအောင် မသတ်မှတ်နိုင်ရင် သူတို့ရဲ့ အလုပ်ကို မပြီးမြောက်သေးဘူးလို့ ထင်တတ်ကြပါတယ်။

ဒါဟာ မှားယွင်းတဲ့အမြင် ဖြစ်ပါတယ်။

Specification တစ်ခုက System တစ်ခုရဲ့ Detail နဲ့ Nuance အားလုံးကို အပြည့်အစုံ ဖော်ပြနိုင်မယ်လို့ ယူဆတာဟာ မသင့်တော်ပါဘူး။

Project မစခင် User တွေကိုယ်တိုင်က သူတို့တကယ်လိုချင်တာကို အတိအကျ သိပြီးသားဖြစ်တယ်လို့လည်း ယူဆလို့မရပါဘူး။

သူတို့က Requirement ကို နားလည်တယ်လို့ ပြောပြီး Page 200 ပါတဲ့ Document ကိုလည်း Approve လုပ်နိုင်ပါတယ်။

ဒါပေမယ့် Running System ကို တကယ်မြင်လိုက်တဲ့အချိန်မှာ Change Request တွေ အများကြီး ထပ်လာမှာ သေချာပါတယ်။

TIP 57

တချို့အရာတွေက ဖော်ပြရေးသားတာထက် တကယ်လုပ်ပြတာ ပိုကောင်းတယ်

Language ရဲ့ Expressive Power မှာလည်း ကန့်သတ်ချက်တွေ ရှိပါတယ်။

Diagramming Techniques နဲ့ Formal Methods တွေတောင် လုပ်ဆောင်ရမယ့် Operation တွေကို Natural Language နဲ့ ဖော်ပြရပါသေးတယ်။

Natural Language ဟာ ဒီအလုပ်အတွက် အမြဲတမ်း မလုံလောက်ပါဘူး။

ဖိနပ်ကြိုးချည်နည်းကို တခြားသူတစ်ယောက် နားလည်အောင် စာနဲ့ တိုတောင်းစွာ ရေးကြည့်ပါ။

လက်တွေ့မှာ ကျွန်တော်တို့အားလုံး ဖိနပ်ကြိုးချည်တတ်ပေမယ့် စာနဲ့ တစ်ဆင့်ချင်း ရှင်းပြဖို့က အလွန်ခက်ခဲပါတယ်။

ဒါကြောင့် တချို့အရာတွေကို စာနဲ့ဖော်ပြတာထက် **တကယ်လုပ်ပြတာက ပိုကောင်းပါတယ်။**

Specification က Programmer ကို Interpretation လုပ်ဖို့ လုံးဝနေရာမပေးတော့ဘူးဆိုရင် Programming ရဲ့ Skill နဲ့ Creativity ကို ဖယ်ရှားလိုက်သလို ဖြစ်နိုင်ပါတယ်။

Coding လုပ်နေစဉ်မှာပဲ Option အသစ်တွေကို တွေ့လာနိုင်ပါတယ်။

ဥပမာ—

“ဒီ Routine ကို ဒီလိုရေးထားတာကြောင့် Function အသစ်တစ်ခုကို အားထုတ်မှုနည်းနည်းနဲ့ ထည့်နိုင်တယ်”

ဒါမှမဟုတ်—

“Specification က ဒီလိုလုပ်ဖို့ပြောထားပေမယ့် အလားတူ Result ကို တခြားနည်းလမ်းနဲ့ တစ်ဝက်လောက်အချိန်နဲ့ ရနိုင်တယ်”

လို့ တွေးမိနိုင်ပါတယ်။

အဲဒီအခါ Specification က အလွန်အမင်း တင်းကျပ်နေခဲ့ရင် ဒီလိုအခွင့်အရေးတွေကိုတောင် သတိမထားမိနိုင်ပါဘူး။

Pragmatic Programmer တစ်ယောက်အနေနဲ့ Requirements Gathering၊ Design နဲ့ Implementation တွေကို တစ်ခုနဲ့တစ်ခု သီးခြားလုပ်ငန်းစဉ်တွေလို မမြင်သင့်ပါဘူး။

အားလုံးဟာ Quality System တစ်ခုကို Delivery လုပ်ဖို့အတွက် တစ်ခုတည်းသော Process ရဲ့ မတူညီတဲ့ အစိတ်အပိုင်းတွေ ဖြစ်ပါတယ်။

Requirements စုဆောင်းခြင်း၊ Specification ရေးခြင်း၊ Coding စတင်ခြင်းတို့ကို သီးခြားအဖွဲ့တွေက သီးခြားလုပ်နေတဲ့ Environment တွေကို သတိထားပါ။

Specification နဲ့ Implementation ဟာ တစ်ခုနဲ့တစ်ခု တိုက်ရိုက်ဆက်စပ်နေသင့်ပါတယ်။

Implementation နဲ့ Testing ကနေ ရလာတဲ့ Feedback တွေဟာ Specification Process ထဲကို ပြန်ဝင်သင့်ပါတယ်။

Specification ကို လုံးဝမရေးသင့်ဘူးလို့ ဆိုလိုတာ မဟုတ်ပါဘူး။

Contractual Reasons၊ အလုပ်လုပ်နေတဲ့ Environment၊ ဒါမှမဟုတ် Product ရဲ့ သဘောသဘာဝကြောင့် အလွန်အသေးစိတ် Specification လိုအပ်တဲ့ အခြေအနေတွေ ရှိပါတယ်။

ဒါပေမယ့် Specification က ပိုပြီးအသေးစိတ်လာတာနဲ့အမျှ အကျိုးကျေးဇူး လျော့ကျလာနိုင်ပြီး တစ်ချိန်မှာ Negative Return တောင် ဖြစ်လာနိုင်တာကို သတိပြုပါ။

Implementation သို့မဟုတ် Prototype မရှိဘဲ Specification တစ်ခုအပေါ် Specification တစ်ခု ထပ်ဆင့်တည်ဆောက်နေရင် တကယ်တည်ဆောက်လို့မရတဲ့အရာကိုတောင် Specification လုပ်မိနိုင်ပါတယ်။

Specification တွေကို Developer တွေအတွက် Security Blanket လို အသုံးပြုပြီး Coding စတင်ရမှာကို ရှောင်နေမိတာ ကြာလာလေလေ တကယ့် Code ရေးဖို့ ပိုခက်လာလေလေ ဖြစ်ပါလိမ့်မယ်။

Specification Spiral ထဲ မကျပါနဲ့။

တစ်ချိန်ချိန်မှာတော့ Coding စတင်ရပါမယ်။

Team တစ်ခုလုံး Specification တွေထဲမှာ ပျော်ဝင်နေပြီဆိုရင် အဲဒီနေရာကနေ ထွက်လာအောင်လုပ်ပါ။

Prototype ဒါမှမဟုတ် Tracer Bullet Development ကို စဉ်းစားပါ။

40. Circles and Arrows

Programming ကို Engineering နဲ့ ပိုတူလာအောင် ပြုလုပ်ဖို့ နည်းလမ်းတွေ အများကြီး ပေါ်ပေါက်ခဲ့ပါတယ်။

Structured Programming၊ Chief Programmer Teams၊ CASE Tools၊ Waterfall Development၊ Spiral Model၊ Jackson၊ ER Diagrams၊ Booch Clouds၊ OMT၊ Objectory၊ Coad/Yourdon နဲ့ ယနေ့ခေတ် UML အထိ Methodology အမျိုးမျိုး ရှိခဲ့ပါတယ်။

Method တစ်ခုချင်းစီမှာ သူ့ရဲ့ Followers တွေ ရှိလာပြီး ခေတ်စားတဲ့အချိန်တစ်ခု ရှိခဲ့ပါတယ်။

နောက်ဆုံးမှာတော့ Method အသစ်တစ်ခုနဲ့ အစားထိုးခံရပါတယ်။

တချို့ Developer တွေက Project တွေ ကျရှုံးနေတာကို သိရက်နဲ့ Methodology အသစ်တစ်ခု ပေါ်လာတိုင်း အဲဒါက ပိုကောင်းမယ်လို့ မျှော်လင့်ပြီး လိုက်ပြောင်းနေကြပါတယ်။

ဒါပေမယ့် ဘယ် Method က ဘယ်လောက်ကောင်းကောင်း Developer တွေက သူတို့ရဲ့ Development Practice နဲ့ Capability တွေကို မစဉ်းစားဘဲ Blindly Adopt လုပ်နေမယ်ဆိုရင် စိတ်ပျက်စရာ ရလဒ်ကို ရစေနိုင်ပါတယ်။

TIP 58

Formal Methods တွေရဲ့ ကျွန်မဖြစ်ပါနဲ့

Formal Methods တွေမှာ အားနည်းချက်တွေ အများကြီးရှိပါတယ်။

Formal Methods အများစုဟာ Requirements ကို Diagram တွေနဲ့ Supporting Words တွေ ပေါင်းစပ်ပြီး ဖော်ပြကြပါတယ်။

ဒီ Diagram တွေဟာ Designer ရဲ့ Requirement အပေါ် နားလည်ထားမှုကို ကိုယ်စားပြုပါတယ်။

ဒါပေမယ့် End User အများစုအတွက် ဒီ Diagram တွေဟာ အဓိပ္ပာယ်မရှိနိုင်ပါဘူး။

အဲဒါကြောင့် Designer က ပြန်လည်ရှင်းပြပေးရပါတယ်။

ဒီလိုဖြစ်တဲ့အခါ Actual User ကိုယ်တိုင်က Requirement ကို Formal Check လုပ်နိုင်ခြင်း မရှိတော့ပါဘူး။

Designer ရဲ့ Explanation ကိုပဲ အားကိုးရပါတယ်။

Requirements ကို ဒီလိုဖမ်းယူထားတာမှာ အကျိုးရှိတာတွေရှိပေမယ့် ဖြစ်နိုင်ရင် User ကို Prototype ပြပြီး ကိုယ်တိုင် စမ်းသုံးခိုင်းတာကို ပိုနှစ်သက်ပါတယ်။

Formal Methods တွေက Specialization ကိုလည်း အားပေးတတ်ပါတယ်။

တစ်ဖွဲ့က Data Model ကို လုပ်တယ်။

တစ်ဖွဲ့က Architecture ကို ကြည့်တယ်။

နောက်တစ်ဖွဲ့က Use Cases တွေ စုဆောင်းတယ်။

ဒီလိုခွဲခြားလုပ်ဆောင်ခြင်းက Communication အားနည်းလာပြီး အားထုတ်မှုတွေ ဖြုန်းတီးသွားစေနိုင်ပါတယ်။

Designer တွေနဲ့ Coder တွေကို "သူတို့" နဲ့ "ငါတို့" ဆိုပြီး ခွဲခြားတဲ့ စိတ်ဓာတ်လည်း ဖြစ်လာနိုင်ပါတယ်။

ကျွန်ုပ်တို့အနေနဲ့ ကိုယ်လုပ်နေတဲ့ System တစ်ခုလုံးကို နားလည်ဖို့ ကြိုးစားသင့်ပါတယ်။

System ရဲ့ အစိတ်အပိုင်းတိုင်းကို အသေးစိတ် သိနိုင်ဖို့ မဖြစ်နိုင်ပေမယ့်—

- Components တွေ ဘယ်လို အပြန်အလှန်ဆက်သွယ်တယ်

- Data တွေ ဘယ်မှာရှိတယ်
- Requirements တွေက ဘာတွေလဲ

ဆိုတာတော့ သိထားသင့်ပါတယ်။

ကျွန်ုပ်တို့က Adaptable ဖြစ်ပြီး Dynamic ဖြစ်တဲ့ System တွေကို ရေးသားချင်ပါတယ်။

Metadata ကို အသုံးပြုပြီး Runtime မှာ Application ရဲ့ Character ကို ပြောင်းလဲနိုင်အောင် လုပ်ချင်ပါတယ်။

ဒါပေမယ့် Formal Methods အများစုက Static Object/Data Model နဲ့ Event/Activity Chart တွေကို ပေါင်းစပ်အသုံးပြုကြပါတယ်။

System တွေရဲ့ Dynamic Nature ကို အပြည့်အဝ ဖော်ပြနိုင်တဲ့ Formal Method ကိုတော့ မတွေ့ရသေးပါဘူး။

တကယ်တော့ Formal Methods အများစုဟာ Runtime မှာ Dynamic ဖြစ်သင့်တဲ့ Object တွေကြားမှာ Static Relationship တွေ တည်ဆောက်ဖို့ အားပေးပြီး လမ်းမှားကို ပို့နိုင်ပါတယ်။

Do Methods Pay Off? (Method တွေက တကယ်အကျိုးရှိသလား)

Robert Glass က Software Development Technologies အမျိုးမျိုးရဲ့ Productivity နဲ့ Quality တိုးတက်မှုတွေကို လေ့လာခဲ့ပါတယ်။

သူ့ရဲ့ သုံးသပ်ချက်အရ Methodology တွေရဲ့ အစောပိုင်း Hype ဟာ အလွန်အကျွံ ချဲ့ကားထားတာ ဖြစ်ပါတယ်။

တချို့ Method တွေမှာ အကျိုးကျေးဇူးရှိတယ်ဆိုတဲ့ အထောက်အထားတွေ ရှိပေမယ့် အဲဒီအကျိုးကျေးဇူးတွေက Method အသစ်ကို လေ့လာအသုံးချနေတဲ့ ပထမအဆင့်မှာ Productivity နဲ့ Quality သိသိသာသာ ကျဆင်းပြီးမှသာ ပေါ်လာတတ်ပါတယ်။

Tool အသစ်နဲ့ Method အသစ်တွေကို Adopt လုပ်ရာမှာ ကုန်ကျစရိတ်ကို ဘယ်တော့မှ လျှော့တွက်မထားပါနဲ့။

ဒီနည်းလမ်းအသစ်တွေကို အသုံးပြုတဲ့ ပထမဆုံး Project တွေကို **Learning Experience** အနေနဲ့ သဘောထားဖို့ ပြင်ဆင်ထားပါ။

Should We Use Formal Methods?

အသုံးပြုသင့်ပါတယ်။

ဒါပေမယ့် Formal Development Methods တွေဟာ Toolbox ထဲက Tool တစ်ခုသာ ဖြစ်တယ်ဆိုတာ အမြဲသတိရပါ။

သေချာစွာ Analysis လုပ်ပြီး Formal Method တစ်ခု လိုအပ်တယ်လို့ ဆုံးဖြတ်ရင် အသုံးပြုပါ။

ဒါပေမယ့် **ဘယ်သူက အုပ်ချုပ်နေတာလဲ** ဆိုတာ မမေ့ပါနဲ့။

Methodology ရဲ့ ကျွန်မဖြစ်ပါနဲ့။

စက်ဝိုင်းတွေနဲ့ မြားတွေက သင့်အတွက် Master မဖြစ်သင့်ပါဘူး။

Pragmatic Programmers တွေဟာ Methodology တွေကို Critical ဖြစ်စွာ လေ့လာပြီး Method တစ်ခုချင်းစီထဲက အကောင်းဆုံးအရာတွေကို ရွေးထုတ်ကာ ကိုယ့်ရဲ့ Working Practices ထဲမှာ ပေါင်းစပ်အသုံးပြုကြပါတယ်။

ပြီးတော့ အဲဒီ Practices တွေကို လစဉ်နီးပါး ပိုကောင်းအောင် ပြုပြင်ကြပါတယ်။

ဒါဟာ အလွန်အရေးကြီးပါတယ်။

သင့် Process တွေကို အမြဲတမ်း Refine လုပ်ပြီး Improve လုပ်နေသင့်ပါတယ်။

Methodology တစ်ခုရဲ့ တင်းကျပ်တဲ့ နယ်နိမိတ်တွေကို ကိုယ့်ကမ္ဘာရဲ့ အကန့်အသတ်အဖြစ် ဘယ်တော့မှ လက်မခံပါနဲ့။

Method တစ်ခုရှိတာနဲ့ Authority ရှိတယ်လို့ မယူဆပါနဲ့။

Class Diagram တွေ အများကြီးနဲ့ Use Cases ရာချီပြီး Meeting ထဲဝင်လာတာကို တွေ့ရနိုင်ပါတယ်။

ဒါပေမယ့် အဲဒီစာရွက်တွေဟာ Requirement နဲ့ Design အပေါ် လူတွေရဲ့ မပြည့်စုံနိုင်တဲ့ Interpretation တွေပဲ ဖြစ်နေပါသေးတယ်။

Tool တစ်ခုက ဘယ်လောက်ဈေးကြီးတယ်ဆိုတာကြောင့် သူထုတ်ပေးတဲ့ Output က ပိုကောင်းတယ်လို့ မယူဆပါနဲ့။

TIP 59
ဈေးကြီးတဲ့ Tools တွေက ပိုကောင်းတဲ့ Design ကို မထုတ်ပေးပါဘူး

Formal Methods တွေဟာ Development ထဲမှာ သူ့နေရာနဲ့သူ အသုံးဝင်ပါတယ်။

ဒါပေမယ့် Project တစ်ခုမှာ—

"Class Diagram က Application ဖြစ်ပြီး ကျန်တာက Mechanical Coding ပဲ"

ဆိုတဲ့ Philosophy ကို တွေ့ရပြီဆိုရင် အဲဒီ Project Team ဟာ ပြဿနာကြီးတစ်ခုထဲ ရောက်နေပြီလို့ သိနိုင်ပါတယ်။

Chapter 8 — Pragmatic Projects

Project တစ်ခု စတင်အကောင်အထည်ဖော်လာတဲ့အခါ Individual Philosophy နဲ့ Coding ပိုင်းကနေ ပိုမိုကျယ်ပြန့်တဲ့ Project-level Issues တွေကို ပြောင်းပြီး စဉ်းစားဖို့လိုလာပါတယ်။

ဒီအခန်းမှာ Project Management ရဲ့ အသေးစိတ်အချက်အလက်အားလုံးကို မဆွေးနွေးဘဲ Project တစ်ခု အောင်မြင်ခြင်း၊ ကျရှုံးခြင်းကို အဓိက သက်ရောက်နိုင်တဲ့ အရေးကြီးတဲ့ နယ်ပယ်အချို့ကို ဆွေးနွေးထားပါတယ်။

လူတစ်ယောက်ထက်ပိုပြီး Project တစ်ခုမှာ အလုပ်လုပ်လာတာနဲ့ Ground Rules တွေ သတ်မှတ်ပြီး Project ရဲ့ အစိတ်အပိုင်းတွေကို သင့်တော်သလို တာဝန်ခွဲဝေပေးရပါမယ်။

Pragmatic Teams မှာ Pragmatic Philosophy ကို မပျက်စေဘဲ Team တစ်ခုအနေနဲ့ ဘယ်လိုလုပ်ဆောင်မလဲဆိုတာ ဆွေးနွေးထားပါတယ်။

Project-level Activities တွေကို တစ်သမတ်တည်း၊ ယုံကြည်စိတ်ချရစွာ လုပ်ဆောင်နိုင်ဖို့ အရေးကြီးဆုံးအချက်က **Procedures တွေကို Automation လုပ်ခြင်း** ဖြစ်ပါတယ်။

အဲဒါကို **Ubiquitous Automation** မှာ ဆက်လက်ရှင်းပြထားပါတယ်။

အရင်အခန်းတွေမှာ Coding လုပ်ရင်း Testing လုပ်ဖို့ ဆွေးနွေးခဲ့ပါတယ်။ **Ruthless Testing** မှာတော့ Project တစ်ခုလုံးအတွက် Testing Philosophy နဲ့ Testing Tools တွေအထိ ဆက်လက်တိုးချဲ့ဆွေးနွေးထားပါတယ်။

Developer တွေ Testing ထက်တောင် ပိုမကြိုက်တာက Documentation ဖြစ်ပါတယ်။ **It's All Writing** မှာ Documentation ကို ပိုမိုလွယ်ကူပြီး အကျိုးရှိအောင် ဘယ်လိုလုပ်မလဲဆိုတာ ဆွေးနွေးထားပါတယ်။

Project အောင်မြင်တယ်ဆိုတာကို Project Sponsor က ဘယ်လိုမြင်သလဲဆိုတာက အရေးကြီးပါတယ်။ **Great Expectations** မှာ Sponsor တွေရဲ့ မျှော်လင့်ချက်ကို ဘယ်လိုကျော်လွန်ပြီး သူတို့စိတ်ကျေနပ်အောင်လုပ်မလဲဆိုတာ ဆွေးနွေးထားပါတယ်။

နောက်ဆုံး Tip ကတော့ ဒီစာအုပ်တစ်အုပ်လုံးရဲ့ အယူအဆတွေနဲ့ တိုက်ရိုက်ဆက်စပ်နေပါတယ်။ **Pride and Prejudice** မှာ ကိုယ်လုပ်တဲ့အလုပ်အပေါ် တာဝန်ယူပြီး ဂုဏ်ယူဖို့ တိုက်တွန်းထားပါတယ်။

41. Pragmatic Teams

အခုအချိန်အထိ ဒီစာအုပ်မှာ Individual Programmer တစ်ယောက် ပိုကောင်းလာအောင် ကူညီပေးနိုင်တဲ့ Pragmatic Techniques တွေကို လေ့လာခဲ့ပါတယ်။

ဒီနည်းလမ်းတွေကို Team တစ်ခုလုံးအနေနဲ့ အသုံးချလို့ရပါသလား။

အဖြေက ရှင်းရှင်းလင်းလင်း "ရပါတယ်" ဖြစ်ပါတယ်။

Pragmatic Individual တစ်ယောက်ဖြစ်ခြင်းမှာ အကျိုးကျေးဇူးတွေရှိပါတယ်။ ဒါပေမယ့် အဲဒီ Individual ဟာ Pragmatic Team တစ်ခုထဲမှာ အလုပ်လုပ်တဲ့အခါ အကျိုးကျေးဇူးတွေက အဆများစွာ တိုးလာပါတယ်။

ဒီအခန်းမှာ Pragmatic Techniques တွေကို Team တစ်ခုလုံးအတွက် ဘယ်လိုအသုံးချနိုင်မလဲဆိုတာ အကျဉ်းချုပ်လေ့လာပါမယ်။

ဒီအချက်တွေက အစပဲရှိပါသေးတယ်။ Pragmatic Developer တွေကို ကောင်းမွန်တဲ့ Environment တစ်ခုထဲမှာ စုစည်းပေးလိုက်ရင် သူတို့ကိုယ်တိုင် သူတို့အတွက် သင့်တော်တဲ့ Team Dynamics တွေကို အလျင်အမြန် တိုးတက်အောင် ပြုပြင်နိုင်ကြပါလိမ့်မယ်။

Broken Windows မထားပါနဲ့

Quality ဟာ Team တစ်ခုလုံးရဲ့ တာဝန်ဖြစ်ပါတယ်။

အလုပ်ကို အလွန်ဂရုစိုက်တဲ့ Developer တစ်ယောက်ကို အလုပ်အရည်အသွေးကို ဂရုမစိုက်တဲ့ Team တစ်ခုထဲ ထည့်ထားလိုက်ရင် အသေးစားပြဿနာတွေကို ပြင်ဖို့အတွက် လိုအပ်တဲ့ စိတ်အားထက်သန်မှုကို ထိန်းသိမ်းဖို့ ခက်ခဲပါလိမ့်မယ်။

Team တစ်ခုလုံးအနေနဲ့ ဘယ်သူမှမပြင်တဲ့ အသေးစားအပြစ်အနာအဆာတွေ၊ **Broken Windows** တွေကို လက်မခံသင့်ပါဘူး။

Product ရဲ့ Quality အတွက် Team တစ်ခုလုံးက တာဝန်ယူရပါမယ်။

Quality Officer တစ်ယောက်ကို သီးခြားခန့်ထားပြီး Quality တာဝန်ကို သူ့ထံပဲ လွှဲအပ်ထားတာဟာ မှားယွင်းပါတယ်။

Quality ဆိုတာ Team Member တစ်ယောက်ချင်းစီရဲ့ ပါဝင်ပံ့ပိုးမှုကနေသာ ရလာနိုင်ပါတယ်။

Boiled Frogs
အပြုတ်ခံရသော ဖားများ

ပတ်ဝန်းကျင်က တဖြည်းဖြည်း ပြောင်းလဲသွားတာကို သတိမထားမိတဲ့ ဖားအကြောင်းကို အရင်က ပြောခဲ့ပါတယ်။

Project Development ရဲ့ အပူအပင်များတဲ့ အခြေအနေထဲမှာ ကိုယ့်ပတ်ဝန်းကျင်မှာ ဘာတွေပြောင်းလဲနေလဲဆိုတာ မမြင်နိုင်တာက လူတစ်ဦးချင်းအတွက် ဖြစ်နိုင်သလို Team တစ်ခုလုံးအတွက်လည်း ပိုလွယ်ကူစွာ ဖြစ်နိုင်ပါတယ်။

တစ်ယောက်က "တခြားတစ်ယောက်က ဒီကိစ္စကို ကိုင်တွယ်နေမှာပဲ" လို့ ထင်နိုင်ပါတယ်။

Team Leader က User တောင်းဆိုတဲ့ Change ကို Approve လုပ်ပြီးသားဖြစ်မယ်လို့ ထင်နိုင်ပါတယ်။

ဒါကြောင့် Team တစ်ခုလုံးအနေနဲ့ ပတ်ဝန်းကျင်ပြောင်းလဲမှုတွေကို တက်ကြွစွာ စောင့်ကြည့်သင့်ပါတယ်။

Scope တိုးလာခြင်း၊ Schedule အချိန်လျော့သွားခြင်း၊ Feature အသစ်တွေ တိုးလာခြင်း၊ Environment အသစ်တွေ ပေါ်လာခြင်း စတဲ့ Original Agreement ထဲမှာ မပါဝင်ခဲ့တဲ့ အရာတွေကို စောင့်ကြည့်ပါ။

Change တွေကို မဖြစ်မနေ ပယ်ချရမယ်လို့ ဆိုလိုတာမဟုတ်ပါဘူး။

Change ဖြစ်နေတယ်ဆိုတာကို သိရှိနေဖို့လိုတာပါ။

မဟုတ်ရင် နောက်ဆုံးမှာ အပူထဲရောက်သွားမှာ Team ကိုယ်တိုင် ဖြစ်ပါလိမ့်မယ်။

Communicate (ဆက်သွယ်ပြောဆိုပါ)

Team ထဲမှာ Developer တွေ အချင်းချင်း ပြောဆိုဆက်သွယ်ဖို့ လိုအပ်တာ သိသာပါတယ်။

ဒါပေမယ့် Team ကိုယ်တိုင်ဟာ Organization ထဲမှာ Entity တစ်ခုအဖြစ် ရှိနေတယ်ဆိုတာ မမေ့သင့်ပါဘူး။

Team တစ်ခုလုံးက Organization ရဲ့ အပြင်ဘက်လူတွေနဲ့လည်း ရှင်းလင်းစွာ ဆက်သွယ်နိုင်ရပါမယ်။

မကောင်းတဲ့ Project Team တွေဟာ အပြင်လူတွေကြည့်ရင် မပျော်မရွှင်၊ စကားမပြောချင်တဲ့ Team တွေလို မြင်ရတတ်ပါတယ်။

Meeting တွေမှာ Structure မရှိဘူး။

Document တွေကလည်း မတူညီဘူး။

Terminology တွေကိုလည်း လူတစ်ယောက်နဲ့တစ်ယောက် မတူအောင် သုံးကြပါတယ်။

ကောင်းမွန်တဲ့ Project Team တွေမှာ ကိုယ်ပိုင် Personality ရှိပါတယ်။

Meeting တစ်ခုလာရင် ကောင်းမွန်စွာ ပြင်ဆင်ထားတဲ့ Presentation ကို တွေ့ရမယ်ဆိုတာ လူတွေသိကြပါတယ်။

Documentation တွေက ရှင်းလင်း၊ တိကျပြီး တစ်သမတ်တည်းဖြစ်ပါတယ်။

Team က အသံတစ်သံတည်းနဲ့ ပြောပါတယ်။

Team Brand

Team တစ်ခုလုံး တစ်သံတည်းဖြစ်အောင် ကူညီပေးနိုင်တဲ့ ရိုးရှင်းတဲ့ Marketing Trick တစ်ခုက **Brand တစ်ခု ဖန်တီးခြင်း** ဖြစ်ပါတယ်။

Project စတဲ့အခါ Project အတွက် နာမည်တစ်ခု သတ်မှတ်ပါ။

ထူးဆန်းပြီး မှတ်မိလွယ်တဲ့ နာမည်ဖြစ်ရင် ပိုကောင်းပါတယ်။

Logo တစ်ခုကို အချိန်နည်းနည်းပေးပြီး ဖန်တီးပါ။

Memos နဲ့ Reports တွေမှာ Logo ကို အသုံးပြုပါ။

လူတွေနဲ့ ဆက်သွယ်တဲ့အခါ Team Name ကို မကြာခဏ အသုံးပြုပါ။

အနည်းငယ် ရယ်စရာကောင်းသလို ထင်ရပေမယ့် Team အတွက် Identity တစ်ခု တည်ဆောက်ပေးနိုင်ပါတယ်။

Don't Repeat Yourself (ကိုယ့်ကိုယ်ကို မထပ်ပါစေနဲ့)

Team Member တွေကြားမှာ အလုပ်တူတူ ထပ်လုပ်နေခြင်းဟာ အချိန်နဲ့ အားစိုက်မှုကို ဖြုန်းတီးစေပါတယ်။

နောက်ပိုင်း Maintenance လုပ်တဲ့အခါလည်း အလွန်ရှုပ်ထွေးစေနိုင်ပါတယ်။

Communication က ဒီပြဿနာကို ကူညီနိုင်ပါတယ်။

တချို့ Team တွေမှာ **Project Librarian** တစ်ယောက်ထားပြီး Documentation နဲ့ Code Repository တွေကို စီမံခိုင်းပါတယ်။

တခြား Team Member တွေက တစ်ခုခုရှာဖွေရင် Librarian ကို ပထမဆုံးမေးနိုင်ပါတယ်။

Project ကြီးလာရင် Functional Area တစ်ခုချင်းစီအတွက် Focal Point တစ်ယောက်စီ သတ်မှတ်နိုင်ပါတယ်။

ဥပမာ—

- Date Handling → Mary
- Database Schema → Fred

စသဖြင့် တာဝန်ပေးနိုင်ပါတယ်။

Groupware Systems နဲ့ Local Newsgroups တွေကလည်း မေးခွန်းနဲ့အဖြေတွေကို ဆက်သွယ်ပြီး သိမ်းဆည်းထားဖို့ အသုံးဝင်ပါတယ်။

Orthogonality

Traditional Team Organization တွေဟာ Waterfall Development ရဲ့ အဟောင်းအယူအဆပေါ်မှာ အခြေခံထားပါတယ်။

လူတွေကို Job Function အလိုက် ခွဲထားပါတယ်—

- Business Analysts
- Architects
- Designers
- Programmers
- Testers
- Documenters

စသဖြင့် ဖြစ်ပါတယ်။

Analysis၊ Design၊ Coding နဲ့ Testing တွေကို သီးခြားကမ္ဘာတွေလို ခွဲထားတာဟာ မှားယွင်းပါတယ်။

ဒါတွေဟာ Problem တစ်ခုတည်းကို ရှုထောင့်မတူဘဲ ကြည့်နေတဲ့ လုပ်ငန်းစဉ်တွေပါ။

User နဲ့ အဆင့် ၂ ဆင့်၊ ၃ ဆင့်လောက် ဝေးနေတဲ့ Programmer တွေဟာ သူတို့ရေးတဲ့ Code ကို User က ဘယ်လိုသုံးနေသလဲဆိုတာ မသိနိုင်တော့ပါဘူး။

အဲဒါကြောင့် မှန်ကန်တဲ့ဆုံးဖြတ်ချက်တွေ ချဖို့ ခက်လာပါတယ်။

TIP 60

Job Function အလိုက်မဟုတ်ဘဲ Functionality အလိုက် Team ဖွဲ့ပါ

လူတွေကို Small Teams အဖြစ် ခွဲပြီး Final System ရဲ့ Functional Aspect တစ်ခုစီအတွက် တာဝန်ပေးပါ။

Team တစ်ခုချင်းစီက ကိုယ့်အတွင်းမှာ ကိုယ့်နည်းကိုယ်ဟန်နဲ့ စီမံနိုင်ပါတယ်။

Database Access Layer လို End-user Feature မဟုတ်တဲ့အရာတွေတောင် Functionality အဖြစ် သတ်မှတ်နိုင်ပါတယ်။

Help Subsystem လည်း အတူတူပါပဲ။

ရည်ရွယ်ချက်က Code ကို Modularize လုပ်တဲ့အခါ သုံးတဲ့ Criteria အတိုင်း **Cohesive, Self-contained Teams** တွေ ဖန်တီးဖို့ပါ။

Team Organization မှားနေတဲ့ လက္ခဏာတစ်ခုက Program Module တစ်ခုတည်း ဒါမှမဟုတ် Class တစ်ခုတည်းကို Subteam နှစ်ဖွဲ့က တစ်ပြိုင်တည်း လုပ်နေခြင်း ဖြစ်ပါတယ်။

Functionality အလိုက် Team ဖွဲ့ခြင်းက—

- Contract
- Decoupling
- Orthogonality

တို့လို Code Organization နည်းလမ်းတွေကို Team Organization မှာပါ အသုံးချနိုင်စေပါတယ်။

ဥပမာ Database Vendor ပြောင်းသွားရင် Database Team ပဲ အဓိကသက်ရောက်ပါမယ်။

Calendar Function အတွက် Off-the-shelf Tool အသစ်သုံးမယ်ဆိုရင် Calendar Team ပဲ အဓိကသက်ရောက်ပါမယ်။

ဒီလိုနည်းနဲ့ Team တွေရဲ့ အပြန်အလှန်သက်ရောက်မှုကို လျှော့ချနိုင်ပြီး Development Time ကို လျှော့၊ Quality ကို မြှင့်၊ Defects ကို လျှော့နိုင်ပါတယ်။

Project ကြီးတွေမှာ အနည်းဆုံး **Heads နှစ်ယောက်** လိုပါတယ်—

1. **Technical Head** — Development Philosophy, Style, Team Responsibilities နဲ့ Technical Decisions ကို ကြည့်မယ်။
2. **Administrative Head / Project Manager** — Schedule, Resources, Progress နဲ့ Business Priorities ကို ကြည့်မယ်။

Project ကြီးလာရင် Librarian၊ Tool Builder၊ Operational Support စတဲ့ အထောက်အပံ့တွေကိုလည်း ထည့်သွင်းနိုင်ပါတယ်။

Automation

Team လုပ်ငန်းစဉ်တွေကို တစ်သမတ်တည်းနဲ့ တိကျစွာ လုပ်ဆောင်နိုင်အောင် သေချာစေဖို့ အကောင်းဆုံးနည်းလမ်းတစ်ခုက

အရာအားလုံးကို Automation လုပ်ခြင်း ဖြစ်ပါတယ်။

Editor က Code ကို အလိုအလျောက် Format လုပ်ပေးနိုင်ရင် ကိုယ်တိုင် Manual လုပ်စရာ ဘာကြောင့်လိုမလဲ။

Nightly Build က Test တွေကို အလိုအလျောက် Run ပေးနိုင်ရင် Test Form တွေကို ကိုယ်တိုင်ဖြည့်စရာ ဘာကြောင့်လိုမလဲ။

Automation ဟာ Project Team တိုင်းအတွက် မရှိမဖြစ် အစိတ်အပိုင်းတစ်ခုပါ။

Team Member တစ်ယောက် ဒါမှမဟုတ် တချို့ကို **Tool Builders** အဖြစ် တာဝန်ပေးပြီး—

- Makefiles
- Shell Scripts
- Editor Templates
- Utility Programs

စတာတွေ ဖန်တီးခိုင်းနိုင်ပါတယ်။

Know When to Stop Adding Paint

Team တွေဟာ လူတစ်ယောက်ချင်းစီနဲ့ ဖွဲ့စည်းထားတာကို မမေ့ပါနဲ့။

Member တစ်ယောက်ချင်းစီရဲ့ ကိုယ်ပိုင်အရည်အချင်းနဲ့ ထူးချွန်နိုင်ခွင့်ကို ပေးပါ။

Project ကို လိုအပ်ချက်အတိုင်း အောင်မြင်စွာ Delivery လုပ်နိုင်ဖို့ လုံလောက်တဲ့ Structure ပဲ ပေးပါ။

ပြီးရင် **ထပ်ပြီး Paint သုတ်ချင်တဲ့ ဆန္ဒကို ထိန်းပါ။**

Challenge

Software Development မဟုတ်တဲ့ နယ်ပယ်တွေမှာ အောင်မြင်တဲ့ Team တွေကို လေ့လာကြည့်ပါ။

သူတို့ ဘာကြောင့် အောင်မြင်တာလဲ။

ဒီအခန်းမှာ ဆွေးနွေးထားတဲ့ Process တွေထဲက ဘာတွေကို သူတို့ အသုံးပြုကြသလဲ။

42. Ubiquitous Automation (နေရာတိုင်းမှာ Automation အသုံးပြုခြင်း)

"Civilization advances by extending the number of important operations we can perform without thinking."
လူသားယဉ်ကျေးမှု တိုးတက်လာတာဟာ မစဉ်းစားဘဲ လုပ်ဆောင်နိုင်တဲ့ အရေးကြီး Operation တွေကို တိုးပွားလာအောင်

ပြုလုပ်နိုင်ခြင်းကြောင့် ဖြစ်ပါတယ်။

ကားခေတ်အစမှာ Model-T Ford ကို Start လုပ်ဖို့ Instructions က စာမျက်နှာ ၂ မျက်နှာကျော် ရှည်ပါတယ်။

ယနေ့ခေတ်ကားတွေမှာတော့ Key လှည့်လိုက်ရုံပါပဲ။

Computing Industry ဟာ Model-T ခေတ်လို ဖြစ်နေသေးပေမယ့် Common Operation တစ်ခုလုပ်တိုင်း စာမျက်နှာ ၂

မျက်နှာစာ Instructions ကို ထပ်ခါထပ်ခါ လိုက်လုပ်ဖို့ မတတ်နိုင်ပါဘူး။

Build၊ Release၊ Code Review Paperwork နဲ့ Project ထဲမှာ ထပ်ခါထပ်ခါ လုပ်ရတဲ့အရာတိုင်းကို Automatic ဖြစ်အောင်

လုပ်သင့်ပါတယ်။

Manual Procedure တွေက Consistency ကို Chance အပေါ် မူတည်စေပါတယ်။

လူတစ်ယောက်နဲ့တစ်ယောက် အဓိပ္ပာယ်ဖွင့်ဆိုပုံ မတူနိုင်တာကြောင့် Repeatability ကိုလည်း အာမခံလို့မရပါဘူး။

TIP 61

Manual Procedure မသုံးပါနဲ့

လူတွေဟာ Computer လောက် တစ်သမတ်တည်း မလုပ်နိုင်ကြပါဘူး။

Shell Script ဒါမှမဟုတ် Batch File ကတော့ Instructions တူတူကို အစီအစဉ်တူတူနဲ့ အကြိမ်ကြိမ် Run နိုင်ပါတယ်။

Source Control ထဲမှာ ထည့်ထားနိုင်တာကြောင့် Procedure ပြောင်းလဲမှုကိုလည်း Track လုပ်နိုင်ပါတယ်။

cron လို Tool တွေကို အသုံးပြုပြီး—

- Backup
- Nightly Build
- Website Maintenance
- Periodic Reports

စတာတွေကို လူမပါဘဲ အလိုအလျောက် Run နိုင်ပါတယ်။

Project ကို Compile လုပ်ခြင်း

Project Compile လုပ်တာဟာ Reliable ဖြစ်ပြီး Repeatable ဖြစ်ရမယ့် အလုပ်တစ်ခုပါ။

IDE သုံးနေတယ်ဆိုရင်တောင် Makefile တွေ အသုံးပြုတာကို နှစ်သက်ပါတယ်။

Makefile က—

- Scripted ဖြစ်တယ်
- Automatic ဖြစ်တယ်
- Code Generation ထည့်နိုင်တယ်
- Regression Tests Run နိုင်တယ်

အကောင်းဆုံးကတော့—

Checkout → Build → Test → Ship
ကို Command တစ်ခုတည်းနဲ့ လုပ်နိုင်ရပါမယ်။

Code ကို အလိုအလျောက် Generate လုပ်ခြင်း

Common Source တစ်ခုကနေ Knowledge တစ်ခုကို Derive လုပ်နိုင်ရင် Code Generation ကို အသုံးပြုပါ။

Make ရဲ့ Dependency Analysis ကို အသုံးပြုပြီး File တစ်ခုကနေ နောက် File တစ်ခုကို အလိုအလျောက် Generate လုပ်နိုင်ပါတယ်။

ဥပမာ XML File ကနေ Java File ထုတ်ပြီး Compile လုပ်နိုင်ပါတယ်။

```
.SUFFIXES: .java .class .xml

.xml.java:
   perl convert.pl $< > $@

.java.class:
   $(JAVAC) $(JAVAC_FLAGS) $<
```

ဒါလိုပဲ Source Code၊ Header File၊ Documentation တွေကိုလည်း အလိုအလျောက် Generate လုပ်နိုင်ပါတယ်။

Regression Tests

Makefile ကို Regression Test တွေ Run ဖို့လည်း အသုံးပြုနိုင်ပါတယ်။

Module တစ်ခုတည်းကိုဖြစ်စေ၊ Subsystem တစ်ခုလုံးကိုဖြစ်စေ Command တစ်ခုနဲ့ Test လုပ်နိုင်ပါတယ်။

Project Source Tree ရဲ့ အပေါ်ဆုံး Directory မှာ Command တစ်ခု Run လုပ်ရုံနဲ့ Project တစ်ခုလုံးကို Test လုပ်နိုင်ရပါမယ်။

Build Automation

Build ဆိုတာ Empty Directory တစ်ခုကနေ Known Environment ကို အသုံးပြုပြီး Project တစ်ခုလုံးကို Scratch ကနေ ပြန်တည်ဆောက်ပြီး Final Deliverable ထုတ်ပေးတဲ့ Procedure ဖြစ်ပါတယ်။

ပုံမှန် Build မှာ—

1. Repository ကနေ Source Code Checkout လုပ်ပါ။
2. Project ကို Scratch ကနေ Build လုပ်ပါ။
3. Release/Version Number သတ်မှတ်ပါ။
4. Distributable Image ဖန်တီးပါ။
5. Documentation၊ README နဲ့ Examples တွေကို ထည့်ပါ။
6. Test တွေ Run ပါ။

Project အများစုမှာ ဒီ Build ကို Nightly Build အဖြစ် အလိုအလျောက် Run သင့်ပါတယ်။

Nightly Build မှာ Available Tests အားလုံးကို Run သင့်ပါတယ်။

ဒီလိုလုပ်ထားရင် ဒီနေ့ Code Change ကြောင့် Regression Test Fail ဖြစ်တာကို ချက်ချင်း သိနိုင်ပါတယ်။

ပြဿနာကို အရင်းအမြစ်နားနီးနီးမှာ ရှာနိုင်လေလေ Fix လုပ်ရတာ ပိုလွယ်လေလေ ဖြစ်ပါတယ်။

Administrative အလုပ်တွေကိုလည်း Automation လုပ်ပါ

Programmer တွေက Coding တစ်ခုတည်းပဲ တစ်နေ့လုံး လုပ်နိုင်ရင် ကောင်းမှာပါ။

ဒါပေမယ့် လက်တွေ့မှာ—

- Email ပြန်ခြင်း
- Paperwork ဖြည့်ခြင်း
- Web Documentation တင်ခြင်း
- Reports ပြုလုပ်ခြင်း

စတာတွေ ရှိပါတယ်။

ဒီအလုပ်တွေကို Shell Script တွေကနေ Automation လုပ်နိုင်ပါတယ်။

အဓိကရည်ရွယ်ချက်က—

Automatic, Unattended, Content-driven Workflow
တစ်ခု တည်ဆောက်ဖို့ပါ။

Web Site ကို အလိုအလျောက် ဖန်တီးခြင်း

Development Team တော်တော်များများမှာ Internal Web Site တစ်ခုထားတာက ကောင်းပါတယ်။

ဒါပေမယ့် Web Site ကို Manual Maintenance အရမ်းမလုပ်သင့်ပါဘူး။

မမှန်ကန်တော့တဲ့ Information က Information မရှိတာထက် ပိုဆိုးပါတယ်။

Code၊ Requirements၊ Design Documents၊ Charts၊ Graphs စတာတွေကနေ Documentation ကို အလိုအလျောက်

Generate လုပ်ပြီး Web Site ပေါ်တင်နိုင်ပါတယ်။

Nightly Build ဒါမှမဟုတ် Source Code Check-in Procedure နဲ့ ချိတ်ထားနိုင်ပါတယ်။

Web Content ကို Repository ထဲက Information ကနေ အလိုအလျောက် Generate လုပ်သင့်ပါတယ်။

ဒါဟာ **DRY Principle** ရဲ့ နောက်ထပ်အသုံးချမှုတစ်ခုပါ။

Information ဟာ Source Code/Document အဖြစ် တစ်နေရာတည်းမှာရှိပြီး Web Browser က View တစ်ခုသာ ဖြစ်သင့်ပါတယ်။

Web Page ကို လက်နဲ့ ပြန်ပြင်နေရတာ မဖြစ်သင့်ပါဘူး။

Nightly Build ကနေ ထုတ်ပေးတဲ့—

- Build Results
- Compiler Warnings
- Errors
- Regression Test Results
- Performance Statistics
- Coding Metrics
- Static Analysis Results

စတာတွေကို Development Web Site မှာ ရရှိနိုင်သင့်ပါတယ်။

Approval Procedures - Approval Process တွေကို Automation လုပ်ခြင်း

Code Review၊ Design Review စတဲ့ Administrative Workflow တွေရှိရင် Web Site နဲ့ Automation ကို အသုံးပြုပြီး

Paperwork ကို လျှော့ချနိုင်ပါတယ်။

ဥပမာ Source File တစ်ခုထဲမှာ—

```
/* Status: needs_review */
```

လို့ ထည့်ထားနိုင်ပါတယ်။

Script တစ်ခုက needs_review ပါတဲ့ File တွေအားလုံးကို ရှာပြီး Review လုပ်ရန် List တစ်ခုအဖြစ် Web Site ပေါ်မှာ

ထုတ်ပေးနိုင်ပါတယ်။

Review ပြီးရင် Status ကို အလိုအလျောက် reviewed ပြောင်းနိုင်ပါတယ်။

Code Walk-through ကို လူတွေနဲ့ ကိုယ်တိုင်လုပ်မလားဆိုတာက သီးခြားကိစ္စဖြစ်ပေမယ့် Paperwork ကိုတော့ Automation

လုပ်နိုင်ပါတယ်။

The Cobbler's Children (ဖိနပ်ချုပ်သမားရဲ့ ကလေးတွေ)

Software ရေးတဲ့သူတွေဟာ တစ်ခါတလေ အလုပ်လုပ်ဖို့ အညံ့ဆုံး Tools တွေကိုပဲ အသုံးပြုကြပါတယ်။

ဒါပေမယ့် Better Tools ဖန်တီးဖို့ လိုအပ်တဲ့ Raw Materials တွေ ကျွန်တော်တို့မှာ ရှိပြီးသားပါ—

- cron
- make
- Ant
- CruiseControl
- Ruby
- Perl
- Other Scripting Languages

Computer ကို ထပ်ခါထပ်ခါ လုပ်ရတဲ့ Mundane Work တွေ လုပ်ခိုင်းပါ။

Computer က ကျွန်တော်တို့ထက် ပိုကောင်းအောင် လုပ်ပေးနိုင်ပါတယ်။

ကျွန်တော်တို့မှာ ပိုအရေးကြီးပြီး ပိုခက်ခဲတဲ့ အလုပ်တွေ လုပ်စရာရှိပါတယ်။

43. Ruthless Testing (မညှာမတာ Testing လုပ်ခြင်း)

Developer အများစုဟာ Testing ကို မကြိုက်ကြပါဘူး။

Code ဘယ်နေရာမှာ ပျက်မယ်ဆိုတာ ကိုယ်တိုင်သိနေပြီး အားနည်းတဲ့နေရာတွေကို ရှောင်ကာ ပေါ့ပေါ့ပါးပါး Test လုပ်တတ်ကြပါတယ်။

Pragmatic Programmer တွေက မတူပါဘူး။

Bug ကို အခုရှာချင်ပါတယ်။

နောက်ပိုင်း တခြားသူက ကိုယ့် Bug ကို ရှာတွေ့ပြီး အရှက်ရတာထက် အခုတည်းက ကိုယ်တိုင် ရှာချင်ပါတယ်။

Bug ရှာတာကို ငါးဖမ်းပိုက်နဲ့ နှိုင်းယှဉ်နိုင်ပါတယ်။

- Unit Test = ပိုက်သေးသေးနဲ့ ငါးသေးတွေဖမ်းခြင်း
- Integration Test = ပိုက်ကြီးနဲ့ ငါးကြီးတွေဖမ်းခြင်း

တစ်ခါတလေ ငါးတွေ လွတ်ထွက်သွားပါမယ်။

အဲဒီအခါ ပိုက်မှာ အပေါက်တွေရှိရင် ပြန်ဖာပြီး နောက်ထပ် Defects တွေကို ဖမ်းနိုင်အောင်လုပ်ရပါမယ်။

TIP 62 - စောစော Test လုပ်ပါ။ မကြာခဏ Test လုပ်ပါ။ Automatic Test လုပ်ပါ။

Code ရလာတာနဲ့ Testing စလုပ်သင့်ပါတယ်။

Bug သေးသေးလေးတွေဟာ အချိန်တိုအတွင်းမှာ ကြီးမားတဲ့ Bug တွေ ဖြစ်လာတတ်ပါတယ်။

Manual Testing အားလုံးကို ကိုယ်တိုင်လုပ်ဖို့ မလိုချင်ပါဘူး။

Test Plan အကြီးကြီးရေးထားပြီး Shelf ပေါ်တင်ထားတာထက် Build တိုင်းမှာ Automatically Run ဖြစ်တဲ့ Test တွေက ပိုထိရောက်ပါတယ်။

Bug ကို စောစောရှာနိုင်လေလေ ပြင်ရတာ ဈေးသက်သာလေလေ ဖြစ်ပါတယ်။

"Code a little, test a little."

Production Code ရေးတဲ့အချိန်နဲ့အတူ Test Code ကိုလည်း ရေးသင့်ပါတယ်။

တကယ်တော့ ကောင်းမွန်တဲ့ Project တစ်ခုမှာ Production Code ထက် Test Code ပိုများနေနိုင်ပါတယ်။

ဒါဟာ အချိန်ကုန်ပေမယ့် ရေရှည်မှာ အများကြီး သက်သာပါတယ်။

Test အားလုံး Pass ဖြစ်ပြီဆိုတာကို သိရခြင်းက Code တစ်ပိုင်းကို **"Done"** လို့ သတ်မှတ်နိုင်ဖို့ ယုံကြည်မှု ပိုပေးပါတယ်။

TIP 63

Test အားလုံး Run ပြီးမှ Coding ပြီးတယ်လို့ သတ်မှတ်ပါ

Code တစ်ပိုင်းရေးပြီးသွားတာနဲ့ Boss ဒါမှမဟုတ် Client ကို "Done" လို့ မပြောသင့်ပါဘူး။

Code ဟာ တကယ်တော့ ဘယ်တော့မှ လုံးဝပြီးသွားတာ မဟုတ်ပါဘူး။

အရေးကြီးတာက Available Tests အားလုံး Pass မဖြစ်သေးရင် အသုံးပြုနိုင်ပြီလို့ မပြောနိုင်ပါဘူး။

Project-wide Testing မှာ အဓိက မေးခွန်း ၃ ခုရှိပါတယ်—

1. ဘာကို Test လုပ်မလဲ။
2. ဘယ်လို Test လုပ်မလဲ။
3. ဘယ်အချိန်မှာ Test လုပ်မလဲ။

What to Test

ဘာတွေကို Test လုပ်မလဲ

အဓိက Testing အမျိုးအစားတွေက—

- Unit Testing
- Integration Testing
- Validation and Verification
- Resource Exhaustion / Errors / Recovery
- Performance Testing
- Usability Testing

တို့ ဖြစ်ပါတယ်။

Unit Testing

Unit Test ဆိုတာ Module တစ်ခုကို စမ်းသပ်တဲ့ Code ဖြစ်ပါတယ်။

Unit Test ဟာ Testing အမျိုးအစားအားလုံးရဲ့ အခြေခံပါ။

Module တစ်ခုချင်းစီက ကိုယ့်ဘာသာကိုယ် မအလုပ်လုပ်နိုင်ရင် အားလုံးပေါင်းတဲ့အခါ ကောင်းကောင်းအလုပ်လုပ်ဖို့ မဖြစ်နိုင်ပါဘူး။

Integration Testing

Integration Test က Project ထဲက Major Subsystems တွေ အချင်းချင်း ကောင်းကောင်းအလုပ်လုပ်ကြောင်း စမ်းသပ်တာပါ။

Contract တွေ ကောင်းကောင်းသတ်မှတ်ပြီး Unit Test တွေ Pass ထားရင် Integration Issue တွေကို အလွယ်တကူရှာနိုင်ပါတယ်။

မဟုတ်ရင် Integration က Bug တွေ ပေါက်ဖွားဖို့ အကောင်းဆုံးနေရာ ဖြစ်လာနိုင်ပါတယ်။

Validation and Verification

Executable UI ဒါမှမဟုတ် Prototype တစ်ခုရလာတာနဲ့ အရေးကြီးတဲ့ မေးခွန်းတစ်ခုကို ဖြေရပါမယ်—

User က သူတို့လိုချင်တာကို ပြောခဲ့တယ်။ ဒါပေမယ့် သူတို့တကယ်လိုအပ်တာ ဟုတ်ပါသလား။

Functional Requirements ကို ပြည့်မီပါသလား။

Bug မရှိပေမယ့် Problem မှားကို ဖြေရှင်းထားတဲ့ System ဆိုရင် အသုံးမဝင်ပါဘူး။

Resource Exhaustion, Errors, and Recovery

Ideal Conditions မှာ System က မှန်ကန်စွာအလုပ်လုပ်တာကို သိပြီးပြီဆိုရင် Real-world Conditions မှာ ဘယ်လိုအလုပ်လုပ်မလဲဆိုတာ စမ်းသပ်ရပါမယ်။

Resource တွေဟာ အကန့်အသတ်မဲ့ မဟုတ်ပါဘူး။

ဥပမာ—

- Memory
- Disk Space
- CPU Bandwidth
- Time
- Network Bandwidth

စတာတွေ ရှိပါတယ်။

Memory မလုံလောက်တဲ့အခါ System က ဘယ်လိုတုံ့ပြန်မလဲ။

Disk မလုံလောက်တဲ့အခါ ဘာဖြစ်မလဲ။

System Fail သွားတဲ့အခါ Gracefully Fail ဖြစ်ပါသလား။

State ကို Save ပြီး User ရဲ့ Work မပျောက်အောင် လုပ်ပေးပါသလား။

ဒါမှမဟုတ် Crash ဖြစ်ပြီး User ရှေ့မှာ အမှားပြသွားပါသလား။

Performance Testing

Real-world Conditions မှာ Expected Users၊ Connections နဲ့ Transactions per Second အတိုင်း Performance Requirement ကို ပြည့်မီပါသလား။

System က Scalable ဖြစ်ပါသလား။

တချို့ Application တွေအတွက် Realistic Load ကို Simulate လုပ်ဖို့ Special Hardware/Software လိုနိုင်ပါတယ်။

Usability Testing

Usability Testing ဟာ အခြား Testing တွေနဲ့ မတူပါဘူး။

Real Users တွေနဲ့ Real Environment မှာ စမ်းသပ်ရပါတယ်။

Requirements Analysis အတွင်းမှာ User တွေကို နားလည်မှုလွဲခဲ့တာရှိပါသလား။

Software က User ရဲ့ လက်တစ်စိတ်တစ်ပိုင်းလို သဘာဝကျစွာ အလုပ်လုပ်နိုင်ပါသလား။

Usability ကို စောစော Test လုပ်သင့်ပါတယ်။

Correction လုပ်ဖို့ အချိန်ကျန်နေသေးတဲ့အချိန်မှာ Test လုပ်ရပါမယ်။

Usability မပြည့်မီတာဟာ Divide-by-zero Bug တစ်ခုလိုပဲ Bug တစ်ခု ဖြစ်ပါတယ်။

Design / Methodology Testing

Code ရဲ့ Design ကိုယ်တိုင်နဲ့ Software တည်ဆောက်ခဲ့တဲ့ Methodology ကို Test လုပ်လို့ရပါသလား။

တစ်နည်းနဲ့ ရပါတယ်။

Metrics တွေကို အသုံးပြုပြီး Code ရဲ့ အမျိုးမျိုးသော Aspect တွေကို တိုင်းတာနိုင်ပါတယ်။

ဥပမာ—

- Lines of Code
- Cyclomatic Complexity
- Inheritance Fan-in / Fan-out
- Response Set
- Class Coupling Ratios

စတာတွေပါ။

Module တစ်ခုရဲ့ Metrics က တခြား Module တွေနဲ့ အလွန်ကွာနေတယ်ဆိုရင် အဲဒီကွာခြားမှုက သင့်တော်ရဲ့လားဆိုတာ စဉ်းစားရပါမယ်။

Regression Testing

Regression Test ဆိုတာ လက်ရှိ Test Result ကို အရင် Result နဲ့ နှိုင်းယှဉ်တာပါ။

ဒီနေ့ Fix လုပ်လိုက်တဲ့ Bug ကြောင့် မနေ့က အလုပ်လုပ်နေတဲ့ Function တစ်ခု မပျက်သွားကြောင်း သေချာစေနိုင်ပါတယ်။

Regression Test ဟာ Development တစ်လျှောက်မှာ အရေးကြီးတဲ့ Safety Net တစ်ခုပါ။

Test Data

Testing အတွက် Data နှစ်မျိုးရှိပါတယ်—

1. **Real-world Data**
2. **Synthetic Data**

နှစ်မျိုးလုံးကို အသုံးပြုသင့်ပါတယ်။

Real-world Data က Actual User Data ဖြစ်ပါတယ်။

Synthetic Data က Artificially Generated Data ဖြစ်ပါတယ်။

Synthetic Data ကို—

- Data အများကြီးလိုတဲ့အခါ
- Boundary Condition စမ်းသပ်ချင်တဲ့အခါ
- Statistical Property တစ်ခုခုနဲ့ စမ်းသပ်ချင်တဲ့အခါ

အသုံးပြုနိုင်ပါတယ်။

ဥပမာ—

- February 29
- အလွန်ကြီးမားတဲ့ Record
- Foreign Postal Code
- Transaction သုံးခုမှာ တစ်ခု Fail ဖြစ်တဲ့ Data

စတာတွေကို အသုံးပြုနိုင်ပါတယ်။

Testing the Tests (Test တွေကိုတောင် Test လုပ်ပါ)

Perfect Software မရေးနိုင်သလို Perfect Test Software လည်း မရေးနိုင်ပါဘူး။

ဒါကြောင့် **Test ကိုပါ Test လုပ်ဖို့လိုပါတယ်။**

Bug တစ်ခုကို ဖမ်းဖို့ Test ရေးပြီးပြီဆိုရင် အဲဒီ Bug ကို တမင်တကာ ဖြစ်အောင်လုပ်ကြည့်ပါ။

Test က တကယ် Alarm ပေးသလား စစ်ပါ။

TIP 64

Testing ကို Test ဖို့ Saboteur အသုံးပြုပါ

Project Saboteur တစ်ယောက်ထားနိုင်ပါတယ်။

သူက Source Tree ရဲ့ သီးခြား Copy တစ်ခုကို ယူပြီး Bug တွေကို တမင်တကာ ထည့်ပါမယ်။

ပြီးတော့ Test တွေက အဲဒီ Bug တွေကို ဖမ်းနိုင်သလား စစ်ပါမယ်။

Testing Thoroughly

Testing လုံလောက်ပြီလို့ ဘယ်လိုသိနိုင်မလဲ။

အတိုချုပ်အဖြေက—

မသိနိုင်ပါဘူး။

Coverage Analysis Tools တွေက ဘယ် Code Line တွေ Execute ဖြစ်ပြီး ဘယ်ဟာတွေ မဖြစ်သေးလဲဆိုတာ ပြပေးနိုင်ပါတယ်။

ဒါပေမယ့် 100% Code Coverage ရတာနဲ့ System မှန်တယ်လို့ မဆိုနိုင်ပါဘူး။

အရေးကြီးတာက Code ရဲ့ **States** ဖြစ်ပါတယ်။

Code Line အနည်းငယ်ပဲရှိပေမယ့် Logical States အလွန်များနိုင်ပါတယ်။

TIP 65 - Code Coverage မဟုတ်ဘဲ State Coverage ကို Test လုပ်ပါ

When to Test

ဘယ်အချိန်မှာ Test လုပ်မလဲ

Testing ကို Project နောက်ဆုံးအချိန်အထိ မထားပါနဲ့။

Production Code တစ်ကြောင်းရှိလာတာနဲ့ Test လုပ်ရပါမယ်။

Testing အများစုကို Automatically လုပ်သင့်ပါတယ်။

Test Result ကိုပါ Automatically Interpret လုပ်နိုင်ရပါမယ်။

Source Repository ထဲ Code Check-in မလုပ်ခင် Test Run လုပ်ပါ။

Stress Test လို Test တွေက Setup နဲ့ Special Equipment လိုနိုင်တာကြောင့် Weekly/Monthly လုပ်နိုင်ပါတယ်။

ဒါပေမယ့် Regular Schedule ထဲမှာ မဖြစ်မနေ ထည့်ထားရပါမယ်။

Tightening the Net

Test တွေက Bug တစ်ခုကို မဖမ်းနိုင်ဘဲ Bug တစ်ခု Production ထဲရောက်သွားခဲ့ရင်—

နောက်တစ်ကြိမ် အဲဒီ Bug ကို ဖမ်းနိုင်မယ့် Test အသစ်တစ်ခု ထည့်ပါ။

TIP 66

Bug တစ်ခုကို တစ်ကြိမ်ပဲ ရှာတွေ့ပါစေ

Human Tester တစ်ယောက်က Bug တစ်ခုကို ရှာတွေ့ပြီဆိုရင် အဲဒီ Bug ကို Human Tester က နောက်တစ်ကြိမ် မတွေ့သင့်တော့ပါဘူး။

Automated Test ထဲမှာ အဲဒီ Bug ကို ဖမ်းနိုင်အောင် Test ထည့်ပါ။

နောက်တစ်ကြိမ်ကစပြီး အမြဲတမ်း Automatically စစ်ဆေးနိုင်ရပါမယ်။

"ဒီ Bug က နောက်ထပ် ဘယ်တော့မှ မဖြစ်တော့ပါဘူး" လို့ Developer က ပြောနိုင်ပါတယ်။

ဒါပေမယ့် အဲဒီ Bug က ပြန်ဖြစ်လာနိုင်ပါတယ်။

Automated Test က ဖမ်းပေးနိုင်တဲ့ Bug တွေကို လူက ထပ်ခါထပ်ခါ လိုက်ရှာဖို့ အချိန်မရှိပါဘူး။

Challenges

Project ကို Automatically Test လုပ်နိုင်ပါသလား။

မလုပ်နိုင်ရင် ဘာကြောင့်လဲ။

Acceptable Result သတ်မှတ်ဖို့ ခက်ခဲလို့လား။

GUI မပါဘဲ Application Logic ကို သီးခြား Test လုပ်ဖို့ ခက်ခဲလို့လား။

အဲဒါက GUI ရဲ့ Coupling နဲ့ပတ်သက်ပြီး ဘာကို ပြောပြနေလဲ။

44. It's All Writing

"The palest ink is better than the best memory."
စာရေးထားတဲ့ မှင်ဖျော့ဖျော့တောင် အကောင်းဆုံး Memory ထက် ပိုကောင်းပါတယ်။

Developer အများစုဟာ Documentation ကို သိပ်မစဉ်းစားကြပါဘူး။

အကောင်းဆုံးအခြေအနေမှာ မဖြစ်မနေ လုပ်ရတဲ့ အလုပ်တစ်ခုလိုပဲ မြင်ကြပါတယ်။

အဆိုးဆုံးအခြေအနေမှာတော့ Project အဆုံးမှာ Management က Documentation ကို မေ့သွားမယ်လို့ မျှော်လင့်ပြီး Low-priority Task လို သတ်မှတ်ကြပါတယ်။

Pragmatic Programmer တွေက Documentation ကို Development Process ရဲ့ Integral Part တစ်ခုအဖြစ် လက်ခံပါတယ်။

Documentation ကို Code နားမှာထားနိုင်ရင် ပိုကောင်းပါတယ်။

Code နဲ့ Documentation ကို သီးခြားအရာနှစ်ခုလို မမြင်ဘဲ **Model တစ်ခုတည်းရဲ့ View နှစ်ခု** လို့ မြင်သင့်ပါတယ်။

TIP 67

English ကို Programming Language တစ်ခုလို သဘောထားပါ

Project Documentation မှာ အဓိကအားဖြင့် နှစ်မျိုးရှိပါတယ်—

Internal Documentation

- Source Code Comments
- Design Documents
- Test Documents

External Documentation

- User Manuals
- Public Documentation
- Published Materials

Audience ဘယ်သူပဲဖြစ်ဖြစ် Documentation ဟာ Code ရဲ့ Mirror ဖြစ်ပါတယ်။

Documentation နဲ့ Code မကိုက်ရင် Code က အမှန်တရား ဖြစ်ပါတယ်။

TIP 68

Documentation ကို အစကတည်းက Build ထဲထည့်ပါ။ နောက်မှ ထပ်ကပ်မထည့်ပါနဲ့။

Comments in Code

Code မှာ Comments ရှိသင့်ပါတယ်။

ဒါပေမယ့် Comments များလွန်းတာက Comments နည်းလွန်းတာလိုပဲ မကောင်းနိုင်ပါတယ်။

Comment က—

ဘာကြောင့် ဒီလိုလုပ်ထားတာလဲ၊ ရည်ရွယ်ချက်က ဘာလဲ၊ Goal က ဘာလဲ

ဆိုတာကို ရှင်းပြသင့်ပါတယ်။

Code က "How" ကို ပြပြီးသားဖြစ်တဲ့အတွက် How ကို ထပ်ရှင်းပြတဲ့ Comment က Redundancy ဖြစ်ပြီး DRY Principle ကို ချိုးဖောက်ပါတယ်။

Comments တွေက—

- Engineering Trade-offs
- ဘာကြောင့် ဒီ Decision ကို ချခဲ့တာလဲ
- ဘယ် Alternative တွေကို ပယ်ခဲ့တာလဲ

စတာတွေကို မှတ်တမ်းတင်ဖို့ အလွန်ကောင်းပါတယ်။

Variable Name တွေကိုလည်း အဓိပ္ပာယ်ရှိအောင် ပေးပါ။

foo၊ doit၊ manager၊ stuff လို နာမည်တွေက မရှင်းလင်းပါဘူး။

connectionPool လို့ ရေးတာက cp လို့ ရေးတာထက် ပိုကောင်းပါတယ်။

Meaningless Name ထက် Misleading Name က ပိုဆိုးပါတယ်။

Code ထဲမှာ ရှုပ်ထွေးစေတဲ့ နာမည်တွေ မပေးပါနဲ့။

What Not to Put in Source Comments

Source Comment ထဲမှာ မထည့်သင့်တာတွေက—

- Code က Export လုပ်တဲ့ Function List
- Revision History
- File Name

စတာတွေ ဖြစ်ပါတယ်။

အဲဒီ Information တွေကို Automatic Tools တွေက ထုတ်ပေးနိုင်ပါတယ်။

Source File ထဲမှာတော့ **Author's Name** လို အရေးကြီးတဲ့ Ownership Information ရှိသင့်ပါတယ်။

တာဝန်ယူမှုကို Code နဲ့ ချိတ်ဆက်ထားခြင်းက လူတွေကို ပိုမိုတာဝန်ယူစေပါတယ်။

Executable Documents

ဥပမာ Database Table တစ်ခုရဲ့ Column တွေကို Specification ထဲမှာ ရေးထားတယ်။

အဲဒီနောက် Database Table ဖန်တီးဖို့ SQL Commands သီးခြားရှိတယ်။

Programming Language ထဲမှာလည်း Row ကို ကိုယ်စားပြုမယ့် Record Structure တစ်ခုရှိတယ်။

တူညီတဲ့ Information ကို နေရာ ၃ ခုမှာ ထပ်ရေးထားတာ ဖြစ်ပါတယ်။

တစ်ခုကို ပြောင်းလိုက်တာနဲ့ ကျန်တဲ့နှစ်ခု Out-of-date ဖြစ်သွားပါမယ်။

ဒါဟာ DRY Principle ကို ချိုးဖောက်တာပါ။

ဖြေရှင်းနည်းက **Authoritative Source တစ်ခု** သတ်မှတ်ပြီး အဲဒီ Source ကနေ အခြား View တွေကို Automatically Generate လုပ်ခြင်းပါ။

ဒါဆို Specification၊ Schema နဲ့ Code တွေဟာ တစ်သမတ်တည်း ကိုက်ညီနေပါမယ်။

What if My Document Isn't Plain Text?

Word Processor တွေက Proprietary Format သုံးရင် Document ကို Automatically Process လုပ်ဖို့ ခက်ပါတယ်။

နည်းလမ်းနှစ်ခုရှိပါတယ်—

1. **Macros ရေးပါ။**
2. Document ကို Definitive Source မထားဘဲ အခြား Representation ကို Authoritative Source လုပ်ပါ။

အဓိကက Model တစ်ခုတည်းကို ပြုပြင်ပြီး View အားလုံးကို Automatically Update လုပ်နိုင်ဖို့ပါ။

Technical Writers

Professional Technical Writers ပါလာတဲ့ Project တွေမှာ Programmer တွေက Material တွေကို "Wall အပြင်ဘက်" ကို ပစ်ပေးပြီး Writer တွေကို ကိုယ့်ဘာသာကိုယ် လုပ်ခိုင်းတာဟာ မှားပါတယ်။

Technical Writer တွေကလည်း Pragmatic Principles ကို လိုက်နာသင့်ပါတယ်—

- DRY
- Orthogonality
- Model/View
- Automation
- Scripting

စတာတွေကို အသုံးပြုသင့်ပါတယ်။

Print It or Weave It

Paper Documentation ဟာ Print ထုတ်ပြီးတာနဲ့ Out-of-date ဖြစ်သွားနိုင်ပါတယ်။

Documentation ဟာ ဘယ်ပုံစံပဲဖြစ်ဖြစ် Snapshot တစ်ခုသာ ဖြစ်ပါတယ်။

ဒါကြောင့် Documentation ကို Web ပေါ်တင်နိုင်တဲ့ Format နဲ့ ထုတ်ထားတာ ပိုကောင်းပါတယ်။

Web Documentation မှာ—

- Hyperlinks
- Date Stamp
- Version Number

တွေ ထည့်ထားသင့်ပါတယ်။

Documentation တစ်ခုတည်းကို—

- Printed Document
- Web Page
- Online Help
- Slide Show

စတဲ့ Format မျိုးစုံနဲ့ ထုတ်ဖို့လိုရင် Content နဲ့ Presentation ကို ခွဲထားသင့်ပါတယ်။

Markup System သုံးထားရင် Source တစ်ခုတည်းကနေ Output Format မျိုးစုံ ထုတ်နိုင်ပါတယ်။

Markup Languages

Large-scale Documentation Project တွေအတွက် DocBook လို Markup System တွေ အသုံးပြုနိုင်ပါတယ်။

Source တစ်ခုတည်းကနေ—

- Online Help
- Manuals
- Website Content
- Tip Calendar

စတာတွေကို Automatically Generate လုပ်နိုင်ပါတယ်။

Documentation နဲ့ Code ဟာ Underlying Model တစ်ခုတည်းရဲ့ View မတူတာသာ ဖြစ်ပါတယ်။

Documentation ကို Code လိုပဲ ဂရုစိုက်ပြီး Main Project Workflow ထဲမှာ ထည့်ထားသင့်ပါတယ်။

Challenge

သင်ရေးထားတဲ့ Source Code အတွက် Explanatory Comment တစ်ခု ရေးထားပါသလား။

မရေးထားရင် ဘာကြောင့်လဲ။

အချိန်မရှိလို့လား။

Code က အလုပ်ဖြစ်မဖြစ် မသေချာသေးလို့လား။

Prototype ဖြစ်လို့ နောက်မှပစ်မယ်လို့ ထင်လို့လား။

ဒါမှမဟုတ် Design ကိုယ်တိုင် မရှင်းလင်းသေးလို့ Documentation ရေးရတာ အဆင်မပြေလို့လား။

ဒီအခြေအနေဟာ **Programming by Coincidence** နဲ့ ဆက်စပ်နေနိုင်ပါတယ်။

45. Great Expectations

Company တစ်ခုက Record Profit ကြေညာပေမယ့် Share Price က 20% ကျသွားနိုင်ပါတယ်။

ဘာကြောင့်လဲ။

Analyst တွေရဲ့ Expectations ကို မပြည့်မီလို့ပါ။

ကလေးတစ်ယောက်က ဈေးကြီးတဲ့ Christmas Present ရပေမယ့် သူမျှော်လင့်ထားတဲ့ ဈေးပေါတဲ့ Doll မဟုတ်လို့ ငိုနိုင်ပါတယ်။

Project Team တစ်ခုက အလွန်ရှုပ်ထွေးတဲ့ Application တစ်ခုကို အံ့သြစရာကောင်းလောက်အောင် တည်ဆောက်နိုင်ပါတယ်။

ဒါပေမယ့် Help System မပါလို့ User တွေက အသုံးမပြုချင်ရင် Project က အောင်မြင်တယ်လို့ မဆိုနိုင်ပါဘူး။

Abstract အနေနဲ့ Application တစ်ခုဟာ Specification ကို မှန်ကန်စွာ Implement လုပ်ရင် အောင်မြင်တယ်လို့ ပြောနိုင်ပါတယ်။

ဒါပေမယ့် လက်တွေ့မှာ Project Success ကို **User Expectations ကို ဘယ်လောက်ကောင်းကောင်း ဖြည့်ဆည်းပေးနိုင်သလဲ** ဆိုတာနဲ့ တိုင်းတာပါတယ်။

Expectations ထက် နိမ့်ရင် Failure ဖြစ်ပါတယ်။

ဒါပေမယ့် Expectations ကို အလွန်အမင်း ကျော်လွန်သွားရင်လည်း ပြဿနာရှိနိုင်ပါတယ်။

Communicating Expectations

User တွေဟာ သူတို့လိုချင်တဲ့ System အကြောင်း Vision တစ်ခုနဲ့ လာကြပါတယ်။

အဲဒီ Vision က—

- မပြည့်စုံနိုင်တယ်
- တစ်ခုနဲ့တစ်ခု မကိုက်ညီနိုင်တယ်
- Technical အရ မဖြစ်နိုင်နိုင်တယ်

ဒါပေမယ့် အဲဒီ Vision က User ရဲ့ Vision ဖြစ်တဲ့အတွက် သူတို့ရဲ့ Emotion တွေနဲ့ ချိတ်ဆက်နေပါတယ်။

ဒါကြောင့် လျစ်လျူရှုလို့မရပါဘူး။

Development တိုးတက်လာတာနဲ့ User Expectations ကို မဖြည့်ဆည်းနိုင်တဲ့နေရာတွေကို တွေ့လာပါမယ်။

တချို့နေရာမှာတော့ User Expectations က လိုတာထက် နည်းနေနိုင်ပါတယ်။

Developer ရဲ့ တာဝန်တစ်ခုက ဒီအချက်တွေကို User နဲ့ ဆက်သွယ်ပေးဖို့ပါ။

Development Process တစ်လျှောက်လုံးမှာ User နဲ့ ဆက်သွယ်ပြီး Final Deliverable အကြောင်း နားလည်မှုတူညီအောင် လုပ်ပါ။

အရေးကြီးဆုံးက Application က ဖြေရှင်းဖို့ ရည်ရွယ်ထားတဲ့ **Business Problem** ကို မမေ့ဖို့ပါ။

“Managing Expectations” လို့ ခေါ်တဲ့ Approach ရှိပေမယ့် User ရဲ့ Hope ကို Developer က ထိန်းချုပ်ဖို့ မဟုတ်ပါဘူး။

Developer နဲ့ User နှစ်ဖက်စလုံးက Development Process နဲ့ Final Deliverable အပေါ် **Common Understanding** တစ်ခု ရရှိအောင် ပူးပေါင်းလုပ်ဆောင်သင့်ပါတယ်။

Tracer Bullets နဲ့ Prototypes တွေဟာ ဒီလို Communication အတွက် အလွန်အသုံးဝင်ပါတယ်။

User က ကိုယ်တည်ဆောက်ထားတာကို ကိုယ်တိုင်မြင်နိုင်ပါတယ်။

ကိုယ်နားလည်ထားတဲ့ Requirements မှန်မမှန်ကိုလည်း အတူတူ စမ်းသပ်နိုင်ပါတယ်။

The Extra Mile (အပိုတစ်လှမ်း သွားပါ)

User နဲ့ အနီးကပ်အလုပ်လုပ်ပြီး Expectations တွေကို နားလည်ထားရင် Project Delivery အချိန်မှာ Surprise နည်းပါမယ်။

စာရေးသူတွေကတော့ ဒါကို **မကောင်းတဲ့အရာ** လို့ ဆိုပါတယ်။

User ကို Surprise လုပ်ပါ။

ဒါပေမယ့် ကြောက်အောင် မလုပ်ပါနဲ့။

ပျော်ရွှင်အံ့အားသင့်အောင် လုပ်ပါ။

User မျှော်လင့်ထားတာထက် အနည်းငယ်ပိုပြီး ပေးပါ။

User-oriented Feature သေးသေးလေးတစ်ခု ထည့်ပေးဖို့ အားထုတ်မှုဟာ ရေရှည်မှာ Goodwill အများကြီး ပြန်ပေးနိုင်ပါတယ်။

User တွေကို နားထောင်ပြီး သူတို့ကို တကယ် Delight ဖြစ်စေမယ့် Feature တွေကို ရှာပါ။

ဥပမာ—

- Balloon / Tooltip Help
- Keyboard Shortcuts
- Quick Reference Guide
- Colorization
- Log File Analyzers
- Automated Installation
- System Integrity Checking Tools
- Training အတွက် System Version အများကြီး Run နိုင်ခြင်း
- Organization အလိုက် Customized Splash Screen

စတာတွေကို အလွယ်တကူ ထည့်နိုင်ပါတယ်။

ဒီ Feature တွေဟာ Superficial ဖြစ်ပေမယ့် Feature Bloat အများကြီး မဖြစ်စေပါဘူး။

အရေးကြီးတာက User တွေကို—

“ဒီ Development Team က ကျွန်တော်တို့အတွက် ဂရုစိုက်ပြီး တကယ်အသုံးပြုဖို့ ရည်ရွယ်ထားတဲ့ System တစ်ခု ဖန်တီးထားတယ်”

လို့ ခံစားစေပါတယ်။

ဒါပေမယ့် Feature အသစ်ထည့်ရင်း System ကို မပျက်စီးစေဖို့ သတိထားပါ။

Challenges

ကိုယ်တိုင်လုပ်ထားတဲ့ Project ကို ကိုယ်တိုင်ပဲ အပြင်းထန်ဆုံး ဝေဖန်မိဖူးပါသလား။

ကိုယ်မျှော်လင့်ထားသလောက် ကိုယ်လုပ်ထားတဲ့အရာက မကောင်းတဲ့အခါ ဘာကြောင့်လဲ။

User တွေ Software ကို ရတဲ့အခါ ဘာအကြောင်းအရာတွေကို အများဆုံး Comment လုပ်ကြသလဲ။

သူတို့ အာရုံစိုက်တဲ့နေရာဟာ သင်အားထုတ်ထားတဲ့ အချိန်နဲ့ အချိုးကျပါသလား။

သူတို့ကို ဘာက Delight ဖြစ်စေသလဲ။

46. Pride and Prejudice (ဂုဏ်ယူမှုနှင့် အမြင်ဘက်လိုက်မှု)

"You have delighted us long enough."
Pragmatic Programmer တွေဟာ Responsibility ကနေ မရှောင်ကြပါဘူး။

အခက်အခဲတွေကို လက်ခံပြီး ကိုယ့်ရဲ့ Expertise ကို ထင်ရှားစွာ အသုံးချကြပါတယ်။

Design တစ်ခု ဒါမှမဟုတ် Code တစ်ခုအတွက် ကိုယ်တိုင်တာဝန်ရှိတယ်ဆိုရင်—

ကိုယ်ဂုဏ်ယူနိုင်မယ့် အလုပ်တစ်ခုကို လုပ်ပါ။

TIP 70
ကိုယ့်အလုပ်မှာ ကိုယ့်နာမည် ထည့်ပါ

ရှေးခေတ် Craftsmen တွေဟာ သူတို့လုပ်ထားတဲ့အလုပ်မှာ ကိုယ့်နာမည်ကို ထည့်ပြီး ဂုဏ်ယူခဲ့ကြပါတယ်။

သင်လည်း အဲဒီလို ဖြစ်သင့်ပါတယ်။

ဒါပေမယ့် Project Team တွေဟာ လူတွေနဲ့ ဖွဲ့စည်းထားတဲ့အတွက် ဒီ Rule က ပြဿနာတချို့ ဖြစ်စေနိုင်ပါတယ်။

Code Ownership ကို တစ်ဦးတည်းပိုင်ဆိုင်တယ်လို့ သတ်မှတ်လိုက်ရင် Cooperation Problem ဖြစ်နိုင်ပါတယ်။

လူတွေက သူတို့ Code ကို ပိုင်နက်လို ကာကွယ်လာနိုင်ပါတယ်။

Common Foundation Element တွေကို တခြားသူတွေနဲ့ အတူမလုပ်ချင်တော့နိုင်ပါတယ်။

နောက်ဆုံး Project တစ်ခုလုံးက သီးခြားသေးငယ်တဲ့ Kingdom တွေလို ဖြစ်သွားနိုင်ပါတယ်။

ကိုယ့် Code ကို အလွန်ဘက်လိုက်ပြီး Coworker ရဲ့ Code ကို မကြိုက်တော့နိုင်ပါတယ်။

ဒါဟာ ကျွန်တော်တို့လိုချင်တဲ့အရာ မဟုတ်ပါဘူး။

ကိုယ့် Code ကို တခြားသူတွေ ဝင်မထိအောင် မနာလိုစွာ ကာကွယ်မနေပါနဲ့။

တစ်ချိန်တည်းမှာပဲ တခြားသူတွေရဲ့ Code ကိုလည်း လေးစားပါ။

Golden Rule ကို လိုက်နာပါ—

ကိုယ်က တခြားသူတွေ ကိုယ့်အပေါ် ပြုမူစေချင်သလို ကိုယ်ကလည်း တခြားသူတွေအပေါ် ပြုမူပါ။

Developer တွေအကြား Mutual Respect ရှိဖို့ အရေးကြီးပါတယ်။

Anonymity - အမည်မသိမှု

အထူးသဖြင့် Project ကြီးတွေမှာ Anonymity က—

- Carelessness
- Mistakes
- Laziness
- Bad Code

တို့အတွက် အခြေအနေကောင်းတစ်ခု ဖြစ်လာနိုင်ပါတယ်။

ကိုယ့်ကိုယ်ကို “Machine ထဲက Gear တစ်ခု” လို မြင်လာပြီး ကောင်းမွန်တဲ့ Code ရေးမယ့်အစား Status Report တွေထဲမှာ Excuse တွေ ရေးလာနိုင်ပါတယ်။

Code မှာ Ownership ရှိသင့်ပါတယ်။

ဒါပေမယ့် Individual တစ်ယောက်တည်းက Ownership ရှိရမယ်လို့ မဆိုလိုပါဘူး။

eXtreme Programming မှာ **Communal Ownership of Code** ကို အကြံပြုထားပါတယ်။

ဒါပေမယ့် Anonymity ရဲ့ အန္တရာယ်ကို ကာကွယ်ဖို့ Pair Programming လို Practice တွေလည်း လိုအပ်ပါတယ်။

Pride of Ownership - ကိုယ့်အလုပ်အပေါ် ဂုဏ်ယူမှု

ကျွန်တော်တို့မြင်ချင်တာက **Pride of Ownership** ပါ။

“ဒီအလုပ်ကို ငါရေးခဲ့တယ်။ ဒီအလုပ်အတွက် ငါ တာဝန်ယူတယ်။”

ကိုယ့်ရဲ့ Signature ဟာ Quality ရဲ့ အမှတ်အသားတစ်ခု ဖြစ်လာသင့်ပါတယ်။

လူတွေက Code တစ်ခုမှာ သင့်နာမည်ကို မြင်လိုက်တာနဲ့—

- Solid ဖြစ်မယ်
- Well-written ဖြစ်မယ်
- Tested ဖြစ်မယ်
- Documented ဖြစ်မယ်
- Professional ဖြစ်မယ်

လို့ ယုံကြည်နိုင်ရပါမယ်။

အဲဒါဟာ **Pragmatic Programmer** တစ်ယောက်ရဲ့ အလုပ်ဖြစ်ပါတယ်။